



KDT Backend

2. REST API와 Springboot

2026.07

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

객체지향 프로그래밍

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

절차적 프로그래밍 vs 객체 지향 프로그래밍

- 절차적 프로그래밍(Procedural Programming)과 객체 지향 프로그래밍(Object-Oriented Programming, OOP)은 소프트웨어 개발의 두 가지 대표적인 패러다임

구분	절차적 프로그래밍	객체 지향 프로그래밍
기본 개념	순차적으로 명령어를 실행하여 문제 해결	객체라는 단위로 데이터를 추상화하고, 그 객체 간 상호작용으로 문제 해결
구조	함수 중심 (Function-based)	클래스와 객체 중심 (Class-based)
데이터 처리	데이터를 함수 외부에서 처리	데이터를 객체 내부에서 캡슐화하여 처리
재사용성	함수 재사용에 초점	클래스/객체 단위로 재사용 및 상속
보안/은닉성	데이터 보호 기능 부족 (전역 변수 사용 가능성 높음)	정보 은닉(Encapsulation) 제공
유지보수성	복잡한 프로그램에서 유지보수 어려움	캡슐화, 상속, 다형성으로 유지보수 용이
주요 개념	순차, 조건, 반복	캡슐화, 상속, 다형성, 추상화
대표 언어	C, Pascal, Fortran 등	Java, C++, Python, C# 등
실행 흐름	위에서 아래로 순차적으로 흐름 제어	객체 간 메시지 전달을 통해 흐름 제어
예제	main() 함수에서 순차적으로 함수 호출	클래스 생성 → 객체 생성 → 메서드 호출

[참고] 프로그래밍 예시(C 스타일)

절차적 프로그래밍 (C 스타일)

```
#include <stdio.h>

void greet() {
    printf("Hello, World!\n");
}

int main() {
    greet();
    return 0;
}
```

절차적 프로그래밍 (Java 스타일)

```
public class Greeter {
    public void greet() {
        System.out.println("Hello, World!");
    }

    public static void main(String[] args) {
        Greeter g = new Greeter();
        g.greet();
    }
}
```

항목	절차적 프로그래밍	객체 지향 프로그래밍
적합 대상	소규모, 단순한 프로그램	중대형, 복잡한 시스템
초점	기능 중심	데이터 및 객체 중심
코드 재사용	함수 단위	클래스, 상속, 다형성 기반
복잡성 관리	낮음 (작은 프로그램에 한함)	높음 (잘 설계하면 유지보수 용이)

Java OOP 핵심 용어

개념	설명	Java 예제 코드
클래스 (Class)	객체를 만들기 위한 설계도. 속성과 동작을 정의.	class Student
객체 (Object)	클래스를 기반으로 생성된 실체(인스턴스).	Student s1 = new Student();
속성 (Field)	객체가 가지는 상태/정보 (변수).	String name; double grade;
메서드 (Method)	객체가 수행하는 동작 (함수).	void study();
생성자 (Constructor)	객체 생성 시 초기화 메서드. 클래스 이름과 동일.	Student(String name, int id)
캡슐화 (Encapsulation)	필드를 private로 숨기고 메서드로 접근.	private double grade;
추상화 (Abstraction)	불필요한 정보를 숨기고 핵심만 공개.	abstract class Person
다형성 (Polymorphism)	같은 메서드가 다양한 형태로 동작.	Student vs Scholar
상속 (Inheritance)	부모 클래스의 속성과 기능을 자식이 물려받음.	class Scholar extends Student

Java OOP – 클래스 (Class)

- Java에서 클래스는 객체를 생성하기 위한 설계도로 속성과 동작을 포함하며 생성자를 가짐

구성 요소	설명	예시
필드(Field)	객체의 상태를 저장하는 변수	String name; double price;
메서드(Method)	객체의 동작을 정의하는 함수	void updatePrice(double price) {...}
생성자(Constructor)	객체 생성시 초기화(클래스 이름과 동일)	public Stock(String name, double price) {...}
this	현재 객체 자기 자신을 참조	this.name = name;
super	부모 클래스의 멤버에 접근	super(name, price);

```
public class Stock {
    String name;
    double price;

    public Stock(String name, double price) {
        this.name = name;    // this: 현재 객체
        this.price = price;
    }

    public void updatePrice(double newPrice) {
        this.price = newPrice;
    }
}
```

```
Stock s = new Stock("스칼라 AI", 17000)
```

Java OOP – 객체 (Object)

- 객체는 클래스(Class)로부터 생성된 실체(Instance)로 속성(필드)과 동작(메서드)을 갖는 메모리 상의 실제 존재

구성 요소	설명	예시
클래스	객체를 만들기 위한 설계도	class Stock
인스턴스 생성	객체를 메모리에 생성	Stock s = new Stock("스칼라 AI", 17000);
필드	객체가 가지고 있는 상태/정보	s.name = "스칼라 AI";
메서드	객체가 수행할 수 있는 기능/행동	s.updatePrice(17000);
this	현재 객체 자기 자신을 참조	this.name = name;
생성자	객체가 생성될 때 초기값 설정	new Stock("스칼라 AI", 17000);

```

public class Main {
    public static void main(String[] args) {
        // 객체 생성 (인스턴스화)
        Stock scalaEdu = new Stock("스칼라 에듀", 15000);
        Stock scalaAI = new Stock("스칼라 AI", 17500);

        // 객체 상태 변경
        scalaEdu.updatePrice(15800);
        scalaEdu.printInfo();
    }
}

```

Java OOP – 필드 (Field)

- 클래스 내부에 선언된 변수로, 객체의 고유한 "속성" 또는 "상태"를 저장
- 필드는 객체가 생성될 때 메모리에 할당되며, 각 객체는 독립적인 자신의 필드 값을 가짐

종류	설명	예시
인스턴스 필드	객체마다 독립적인 값을 가짐	String name;
정적 필드 (static)	클래스 전체에서 공유되는 값	static int count;
상수 필드 (final)	한 번 값이 정해지면 변경 불가	final String STOCK_TYPE = "우선주";

Java OOP – 메서드 (Method)

- 메서드는 객체의 동작(기능)을 정의하는 코드 블록

```
[접근제어자] [반환형] [메서드이름](매개변수 목록) {
    // 메서드 본문
    return 값; // 반환형이 void가 아닌 경우
}
```

구성 요소	설명	예시
접근제어자	외부에서 메서드 접근 가능 여부	public, private, protected, (default)
반환형	메서드 실행 후 반환하는 값의 타입	int, String, void 등
메서드명	동작을 나타내는 이름 (소문자로 시작)	getPrice, calculateTax
매개변수(Parameter)	메서드에 전달되는 입력 값들	(int a, int b)
return문	결과를 반환하고 메서드 종료	return a + b;

Java OOP – 메서드 (Method) 예시

```
public class StockUtils {  
  
    // 1. 반환값도 없고, 매개변수도 없음  
    public void printWelcomeMessage() {  
        System.out.println("주식 정보 시스템에 오신 것을 환영합니다.");  
    }  
  
    // 2. 매개변수만 있고, 반환값 없음  
    public void printStockPrice(String stockName, double price) {  
        System.out.println(stockName + "의 현재 가격은 " + price + "원입니다.");  
    }  
  
    // 3. 반환값만 있고, 매개변수 없음  
    public String getMarketStatus() {  
        return "장 마감";  
    }  
  
    // 4. 매개변수도 있고, 반환값도 있음  
    public double calculatePriceChange(double yesterday, double today) {  
        return today - yesterday;  
    }  
  
    // 5. 오버로딩(Overloading): 이름은 같지만 매개변수가 다름  
    public void printStockInfo(String stockName) {  
        System.out.println "[" + stockName + "] 종목 정보 조회");  
    }  
}
```

Java OOP – 캡슐화(Encapsulation)

- 객체의 속성(필드)을 외부에서 직접 접근하지 못하도록 숨기고, 공개된 메서드(getter/setter)를 통해서만 간접적으로 접근하도록 하는 것

캡슐화 원칙	예시
필드는 private으로 선언	private double price;
접근은 getter/setter를 통해 제공	public double getPrice()
필요 시 setter에 유효성 검사 추가	if (price > 0) this.price = price;

❖ 현장에서는 대부분 Lombok을 활용하여 자동으로 getter와 setter를 생성

[참고] 필드 공개와 캡슐화 비교

항목	public 필드 직접 접근	private + getter/setter (캡슐화)
접근 방식	stock.price = 10000;	stock.setPrice(10000);
간결성	간단하고 빠름	코드가 약간 길어짐
유효성 검사	불가능 (값 제한 못함)	setter에 검사 로직 가능
데이터 보호	불안전 (외부에서 자유롭게 변경 가능)	안전 (변경 제어 가능)
유지보수	변경 시 외부 코드 모두 영향 받음	내부 변경 시 외부 영향 최소화
기능 확장성	어려움 (로직 삽입 불가)	쉬움 (getter/setter에 로직 추가 가능)
OOP 원칙 준수	객체지향 위반	정보 은닉 원칙 충실
테스트 및 디버깅	예기치 않은 변경 가능성 존재	추적과 제어 쉬움

```
public class Stock {
    public String name;
    public double price; // 누구나 직접 수정 가능
}

Stock s = new Stock();
s.price = -1000; // 유효하지 않은 값도 설정됨
```

```
public class Stock {
    private double price;

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        if (price > 0) this.price = price;
        else System.out.println("❌ 잘못된 가격");
    }
}

Stock s = new Stock();
s.setPrice(-1000); // → 경고 출력
```

Java OOP – 추상화(Abstraction)

- 추상화란 복잡한 시스템에서 핵심적인 개념만을 추출하고, 불필요한 세부사항은 숨기는 것
- Java에서는 abstract class 또는 interface를 사용하여 추상화를 구현

항목	추상 클래스 (abstract class)	인터페이스 (interface)
용도	상태와 일부 공통 구현을 자식들과 공유하기 위한 설계 도구	구현 없이 동작 규약만 선언하여, 다양한 객체들이 동일한 역할을 수행할 수 있게 함
다중 상속	불가능	가능
필드	인스턴스 변수, 생성자 포함 가능	상수만 가능 (public static final)
메서드	일부 구현 가능	default, static 메서드 구현 가능

```
// 공통 추상 클래스: 자산
abstract class Asset {
    protected String name;
    protected double price;

    public Asset(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public abstract void printInfo();
}
```

```
// 인터페이스: 가치 있는 자산이라는 역할을 부여
public interface Valuable {
    void printInfo();

    default void updatePrice(double price) {
        System.out.println("가격은 " + price + "원");
    }
}
```

Java OOP – 다형성(Polymorphism)

- 다형성이란 "하나의 타입으로 여러 형태의 동작을 표현할 수 있는 능력"을 의미
- 같은 메서드 호출이라도 객체의 실제 타입에 따라 다르게 동작하게 만드는 개념

다형성의 조건	설명
상속(Inheritance)	자식 클래스가 부모 클래스를 상속해야 함
오버라이딩(Overriding)	자식 클래스가 부모의 메서드를 재정의해야 함
*업캐스팅(Upcasting)	자식 객체를 부모 타입으로 참조

❖ 업캐스팅 / 다운캐스팅 예시

```
public class Main {
    public static void main(String[] args) {
        // 업캐스팅
        Stock s = new PreferredStock(); // PreferredStock → Stock

        s.printInfo();           // 자식의 오버라이딩된 메서드 실행
        // s.showDividend();     // 오류: 부모 타입에서는 보이지 않음

        // 다시 자식 타입으로 캐스팅 가능 (다운캐스팅)
        if (s instanceof PreferredStock) {
            PreferredStock ps = (PreferredStock) s;
            ps.dividendRate = 5.0;
            ps.showDividend(); // 가능
        }
    }
}
```

Java OOP – 상속(Inheritance)

- 상속이란 기존(부모) 클래스의 속성과 메서드를 새로운(자식) 클래스가 물려받아 사용하는 객체지향 개념
- 상속은 extends 키워드를 사용하며, 코드 재사용성 향상과 유지보수 편의성을 제공

캡슐화 원칙	예시
필드는 private으로 선언	private double price;
접근은 getter/setter를 통해 제공	public double getPrice()
필요 시 setter에 유효성 검사 추가	if (price > 0) this.price = price;

```
// 부모 클래스
public class Stock {
    protected String name;
    protected double price;

    public Stock(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public void printInfo() {
        System.out.println("[일반주] 종목: " + name + ",
가격: " + price + "원");
    }
}
```

```
// 자식 클래스 (상속)
public class PreferredStock extends Stock {
    private double dividendRate;

    // 자식 생성자에서 super()로 부모 생성자 호출
    public PreferredStock(String name, double price,
double dividendRate) {
        super(name, price); // 부모 생성자 호출
        this.dividendRate = dividendRate;
    }

    // 메서드 오버라이딩
    @Override
    public void printInfo() {
        System.out.println("[우선주] 종목: " + name + ", 가
격: " + price + "원, 배당률: " + dividendRate + "%");
    }
}
```

SOLID 원칙

- 유지보수성과 확장성을 높이기 위한 5가지 설계 원칙으로 *Clean Architecture의 핵심

원칙	이름	핵심 개념
S	단일 책임 원칙 (SRP) Single Responsibility Principle	하나의 클래스는 하나의 책임만 가져야 한다 "클래스는 하나의 책임만 가져라"
O	개방-폐쇄 원칙 (OCP) Open/Closed Principle	소프트웨어 요소(클래스, 모듈, 함수 등)는 확장에 대해 열려 있어야 하고, 수정에 대해서는 닫혀 있어야 한다. 새로운 기능은 기존 코드 수정 없이 추가할 수 있어야 함. "기능 확장은 허용, 기존 코드 수정은 최소화"
L	리스코프 치환 원칙 (LSP) Liskov Substitution Principle	자식 클래스는 부모 클래스를 대체할 수 있어야 한다. 상위 타입 객체를 하위 타입 객체로 치환해도 프로그램이 정상 동작해야 한다. "하위 클래스는 상위 클래스의 행위를 해치면 안 된다." "부모 타입을 대신할 수 있어야 한다"
I	인터페이스 분리 원칙 (ISP) Interface Segregation Principle	하나의 범용적인 인터페이스보다는, 여러 개의 구체적인 인터페이스를 만드는 것이 좋다. 클라이언트는 자신이 사용하지 않는 메서드에 의존하지 않아야 한다. 인터페이스는 작게, 역할 단위로 나누어야 한다. "작은 인터페이스로 쪼개라"
D	의존 역전 원칙 (DIP) Dependency Inversion Principle	고수준 모듈은 저수준 모듈에 의존하면 안 된다. 상위 모듈은 하위 모듈에 의존하지 말고, 추상화(인터페이스)에 의존 해야 한다. "구현체가 아닌 추상화에 의존하라."

- ❖ Clean Architecture는 계층 구조와 의존성 규칙을 통해, 비즈니스 로직을 프레임워크나 DB, UI 등 외부 요소로부터 독립시키는 설계 구조 (Robert Martin이 제안)

SOLID 원칙 – SRP (단일 책임 원칙)

- 한 클래스는 하나의 책임만 가져야 한다.

[나쁜 구조]

```
class Employee {  
    public void getEmployeeInfo() { /* 직원 정보 조회 로직 */ }  
    public void calculatePay() { /* 급여 계산 로직 */ }  
}
```

[좋은 구조]

```
class Employee {  
    public void getEmployeeInfo() { /* 직원 정보 조회 로직 */ }  
}  
  
class PayCalculator {  
    public void calculatePay(Employee employee) { /* 급여 계산 로직 */ }  
}
```


SOLID 원칙 – OCP (개방-폐쇄 원칙)

- 소프트웨어 요소(클래스, 모듈, 함수 등)는 확장에 대해 열려 있어야 하고, 수정에 대해서는 닫혀 있어야 한다.

[나쁜 구조]

```
class AnimalSound {  
    public void makeSound(Animal animal) {  
        if (animal instanceof Dog) {  
            System.out.println("멍멍");  
        } else if (animal instanceof Tiger) {  
            System.out.println("어흥");  
        }  
    }  
}
```

[좋은 구조]

```
interface Animal {  
    void makeSound();  
}  
  
class Dog implements Animal {  
    public void makeSound() { System.out.println("멍멍"); }  
}  
  
class Tiger implements Animal {  
    public void makeSound() { System.out.println("어흥"); }  
}  
  
class AnimalSound {  
    public void makeSound(Animal animal) {  
        animal.makeSound();  
    }  
}
```

SOLID 원칙 – LSP (리스코프 치환 원칙)

- 자식 클래스는 언제나 부모 클래스의 역할을 대체할 수 있어야 한다.

[나쁜 구조]

```
class Bird {  
    void fly() { /* 나는 로직 */ }  
}  
  
class Penguin extends Bird {  
    @Override  
    void fly() {  
        throw new UnsupportedOperationException("펭귄은 날 수 없어요.");  
    }  
}
```

[좋은 구조]

```
// LSP 적용 사례  
class Bird { /* 기본적인 새의 특징 */ }  
  
interface Flyable {  
    void fly();  
}  
  
class Eagle extends Bird implements Flyable {  
    public void fly() { /* 날아가는 로직 */ }  
}  
  
class Penguin extends Bird { /* 펭귄은 날아가는 로직 구현 필요 없음 */ }
```

SOLID 원칙 – ISP(인터페이스 분리 원칙)

- 하나의 범용적인 인터페이스보다는, 여러 개의 구체적인 인터페이스를 만드는 것이 좋다.

[나쁜 구조]

```
interface SmartMachine {
    void print();
    void fax();
    void scan();
}

class Printer implements SmartMachine {
    public void print() {}
    public void fax() {} // 필요 없음
    public void scan() {} // 필요 없음
}
```

[좋은 구조]

```
interface Printable {
    void print();
}

interface Scannable {
    void scan();
}

class Printer implements Printable {
    public void print() {}
}
```

SOLID 원칙 – DIP(의존 역전 원칙)

- 고수준 모듈은 저수준 모듈에 의존하면 안 된다.

[나쁜 구조]

```
class SnowTire {
    public void roll() { /* 눈길 타이어 로직 */ }
}

class Car {
    private SnowTire tire;

    public Car() {
        this.tire = new SnowTire();
    }

    public void drive() {
        tire.roll();
    }
}
```

[좋은 구조]

```
interface Tire {
    void roll();
}

class SnowTire implements Tire {
    public void roll() { /* 눈길 타이어 로직 */ }
}

class NormalTire implements Tire {
    public void roll() { /* 일반 타이어 로직 */ }
}

class Car {
    private Tire tire;

    // 외부에서 의존성 주입 (타이어 설정)
    public Car(Tire tire) {
        this.tire = tire;
    }

    public void drive() {
        tire.roll();
    }
}
```

[샘플코드] 객체지향 프로그래밍(OOP)

```
// 캡슐화 + 상속 + 다형성 예시
public class Product {                                // 부모 클래스
    private String name;                               // 필드는 private (은닉)
    private int price;
    public Product(String name, int price) { // 생성자
        this.name = name; this.price = price;
    }
    public int getPrice() { return price; } // getter
    public void setPrice(int price) {           // setter(검증 포함)
        if (price < 0) throw new IllegalArgumentException();
        this.price = price;
    }
}

public class DiscountProduct extends Product { // 상속
    private int rate;
    public DiscountProduct(String n, int p, int r) {
        super(n, p);                               // 부모 생성자 호출
        this.rate = r;
    }
    @Override                                       // 다형성(오버라이딩)
    public int getPrice() { return super.getPrice()*(100-rate)/100; }
}
```

[핵심 요약] 객체지향 프로그래밍(OOP) 복습

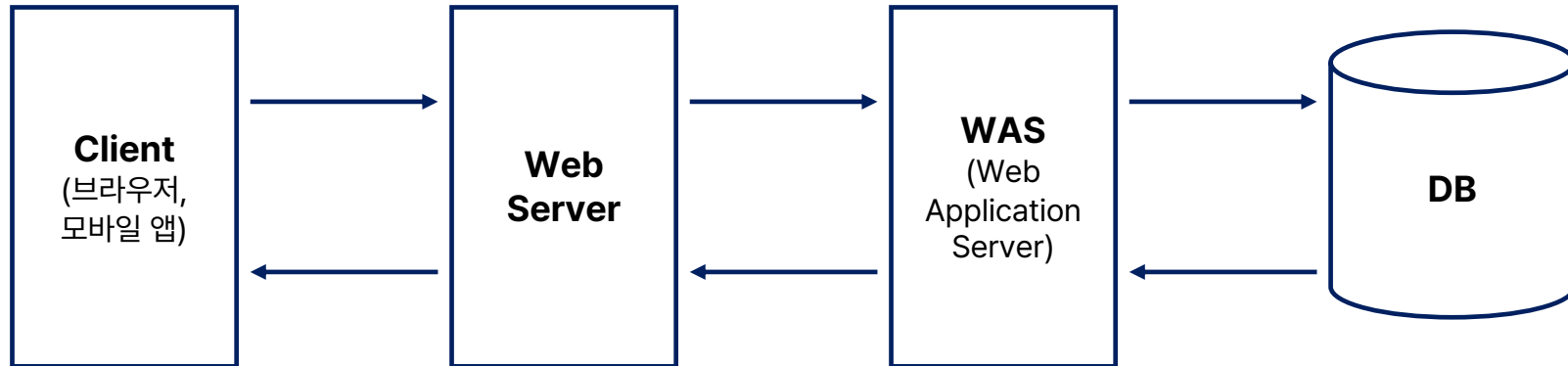
핵심 포인트	반드시 숙지할 내용
OOP 4대 특성	캡슐화 (은닉·getter/setter) · 상속 · 다형성 · 추상화 — OOP를 지탱하는 핵심 개념
핵심 용어	클래스 =설계도, 객체(인스턴스) =실체, 필드 =상태, 메서드 =동작
절차적 vs OOP	OOP는 객체 단위로 재사용·유지보수·확장 에 유리 (중대형 시스템 적합)
SOLID 5원칙	SRP·OCP·LSP·ISP· DIP — 유지보수성과 확장성을 높이는 설계 원칙
DIP → DI 근거	고수준 모듈은 구현이 아닌 추상(인터페이스)에 의존 → 이후 배우는 DI의 이론적 토대

RESTful API

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

웹 애플리케이션

- 클라이언트 (사용자)와 서버 사이의 통신을 HTTP 프로토콜 등으로 데이터를 주고 받으며 동작하는 프로그램
- 웹 어플리케이션은 웹 브라우저를 통해 접속할 수 있음
- 클라이언트: HTML, CSS, JavaScript 와 같은 웹 기술로 구현 (Angular, React, Vue.js 등의 프레임워크 사용)
- 서버 : Java, Node.js, Python 등의 프로그래밍 언어로 구현 (Spring Boot, Express, Django 등의 프레임워크)



[참고] Web Server와 WAS

구분	Web Server	WAS(Web Application Server)
주요 기능	정적 콘텐츠 제공	동적 콘텐츠 제공
역할	HTML, CSS, JS, 이미지 등 정적 파일 반환	비즈니스 로직 처리, 동적 페이지 생성
예시	Apache, Nginx	Tomcat, JBoss, WebLogic
처리 방식	클라이언트 요청 → 파일 반환	클라이언트 요청 → 로직 처리 → 결과 반환
DB 연동	불가	가능
속도	빠름	상대적으로 느림
실행 환경	OS 레벨	JVM 등 어플리케이션 레벨

RESTful API

- REST(Representational State Transfer) 원칙을 따르는 웹 기반 API
- 자원을 '명사'로 표현하고, HTTP 메서드로 자원을 다루는 아키텍처 스타일
- 웹의 기본 구조(HTTP, URL, Stateless 등)를 최대한 활용하여 설계
- HTTP 메서드를 통한 자원 조작
 - GET: 조회
 - POST: 생성
 - PUT/PATCH: 수정
 - DELETE: 삭제

[참고] URI vs. URL

- **URI**는 리소스를 식별하는 모든 방식(이름, 위치, 둘 다 포함)의 총칭
- **URL**은 그 중에서 "위치(주소)를 알려주는" 식별자
- 모든 URL은 URI이지만, 모든 URI가 URL은 아니다

구분	URI (Uniform Resource Identifier)	URL (Uniform Resource Locator)
정의	리소스(자원)를 식별하는 모든 방법의 총칭	리소스의 위치(주소)를 명시하는 식별자
포함 관계	URL은 URI의 하위 개념	URI의 한 종류
형식	scheme:[//authority]path[?query][#fragment]	scheme://host[:port]/path[?query][#fragment]
예시	mailto:someone@example.com https://example.com/index.html	https://example.com/index.html ftp://ftp.example.com/file.txt
기능	리소스의 식별(이름, 위치, 둘 다 가능)	리소스의 위치(주소) 지정, 접근 방법 명시
설명	이름(URN), 위치(URL) 모두 포함	리소스가 어디 있는지, 어떻게 접근하는지 명확히 나타냄

❖ **URN (Uniform Resource Name):** 리소스의 "이름"에 초점을 맞춘 URI의 하위 개념

REST API 설계 규칙

규칙	설명 및 예시
자원(Resource) 명사 사용	URI는 동작이 아닌 자원(명사)로 작성 예) <code>/users</code> , <code>/products/123/comments</code>
동사 대신 HTTP 메서드 사용	URI에 동사를 쓰지 않고, GET/POST/PUT/DELETE 등 HTTP 메서드로 동작을 표현 예) <code>GET /users</code>
계층적 구조	자원 간 포함 관계는 URI로 계층 구조 표현 예) <code>/users/123/orders/5</code>
단수/복수 일관성	보통 전체 집합은 복수형으로 표현 (단수형 가능) → 일관성 유지 예) <code>/users</code> , <code>/products</code>
쿼리 파라미터	필터, 정렬, 검색 등은 쿼리 파라미터 사용 예) <code>/users?age=20&sort=name</code>
상태코드 활용	처리 결과에 따라 적절한 HTTP 상태코드 반환 (200 OK에 일관성 있는 메시지 형식도 가능) 예) <code>200(성공)</code> , <code>201(생성됨)</code> , <code>404(없음)</code> , <code>400(잘못된 요청)</code>
응답 포맷	보통 JSON 형식 사용 예) <code>{ "id": 1, "name": "Alice" }</code>
Stateless	각 요청은 독립적으로 처리, 서버는 클라이언트의 상태 저장 X

REST API 예시

■ 사용자 관리

기능	HTTP 메서드	URI	설명
목록 조회	GET	/users	모든 사용자 목록
생성	POST	/users	새 사용자 생성
단일 조회	GET	/users/3	id=3인 사용자 조회
전체 수정	PUT	/users/3	id=3인 사용자 정보 전체 수정
일부 수정	PATCH	/users/3	id=3인 사용자 정보 일부 수정
삭제	DELETE	/users/3	id=3인 사용자 삭제

■ 게시판 관리

기능	HTTP 메서드	URI	설명
게시글 목록	GET	/posts	게시글 전체 목록
게시글 생성	POST	/posts	게시글 등록
게시글 조회	GET	/posts/10	id=10 게시글 조회
댓글 목록	GET	/posts/10/comments	게시글 10번의 댓글들

OAS (OpenAPI Specification)

개념

**“RESTful API를 기술하고
문서화하기 위한 표준 포맷”**

- 예전 이름: Swagger Specification
- 주요 형식: YAML 또는 JSON
- 현재 버전: OpenAPI 3.x (3.0.0, 3.0.3, 3.1.0 등)

목적

- API 명세를 명확히 하여 팀 간 협업 효율화
- API 문서 자동 생성 및 테스트 도구 활용 가능
- 서버/클라이언트 코드 자동 생성
- Mock 서버 및 검증 자동화 기반 구축

OAS 기반 API 명세서

OAS (OpenAPI Specification)

RESTful API를 정의 하는 표준 형식

- OAS History
 - Swagger (2010, Tony Tam)
 - SmartBear 가 인수
 - OpenAPI Initiative (2015)
 - : Google, Microsoft, IBM, Adobe, Red Hat 등에 의해 추진된 OAS 표준화를 위한 프로젝트
 - 2017년 OpenAPI Specification 3.0 발표

【 Open API vs OpenAPI 】

Open API	OpenAPI
개방형API (ex. 공공데이터조화)	OpenAPI Specification RESTful API에 대한 명세서를 작성하는 표준 규격

【 OAS vs Swagger 】

OAS	Swagger
 OAI에 의해 추진된 RESTful API 표준규격	 SmartBear 에서 제공 하는 OAS 작성 도구 (Swagger Editor, Swagger UI, Swagger Codegen 등)

OAS (OpenAPI Specification) 필요 이유

【 전통적인 API 문제점 】

- 문서가 항상 최신이 아님
- API 응답 구조가 다르게 구현
- 프론트엔드-백엔드 협업 지연



【 OAS 기반의 장점 】

- 문서화와 코드 일관성 유지
- 명세 기반 자동화 (코드 생성, 문서, Mock 서버 등)
- API 테스트 자동화 및 오류 감소
- 다양한 도구와 생태계 연계

[참고] OAS (OpenAPI Specification)

【 Swagger 생태계 도구 】

도구	설명
Swagger Editor	YAML 기반 API 문서 작성 및 미리보기
Swagger UI	API 스펙 기반 인터랙티브 문서
Swagger Codegen	API 명세서로 서버/클라이언트 코드 자동 생성
OpenAPI Generator	Swagger Codegen 확장판, 언어/프레임워크 다양
SwaggerHub	API 명세 협업 플랫폼 (SaaS)

[참고] OAS 기반 API 명세서 구조

- OAS API 명세서는 YML 또는 JSON 파일로 작성 :

Text Editor(IDE에 플러그인)로 작성할 수 있으며 Swagger UI로 확인 가능

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "Swagger Petstore - OpenAPI 3.0",
    "description": "This is a sample Pet Store Server based on the OpenAPI 3.0 specification.",
    "termsOfService": "http://swagger.io/terms/",
    "contact": {
      "email": "apiteam@swagger.io"
    },
    "license": {
      "name": "Apache 2.0",
      "url": "http://www.apache.org/licenses/LICENSE-2.0.html"
    },
    "version": "1.0.11"
  },
  "externalDocs": {
    "description": "Find out more about Swagger",
    "url": "http://swagger.io"
  },
  "servers": [
    {
      "url": "https://petstore3.swagger.io/api/v3"
    }
  ],
  ...
}
```

Swagger Petstore - OpenAPI 3.0

1.0.11

OAS 3.0

<https://raw.githubusercontent.com/asyncapi/spec/v2.6.0/examples/streetlights-kafka.yml>

This is a sample Pet Store Server based on the OpenAPI 3.0 specification.

[Terms of service](#)[Contact the developer](#)[Apache 2.0](#)[Find out more about Swagger](#)

Servers

- 작성된 JSON 파일이 Swagger UI로 표현된 화면
- OAS 버전 정보 명시(3.0)
- API 문서의 제목과 서버 정보 등을 작성

Authorize



[핵심 요약] RESTful API

핵심 포인트	반드시 숙지할 내용
REST 원칙	자원(URI) + HTTP 메서드(GET·POST·PUT·DELETE) 로 상태 없이(Stateless) 통신
설계 규칙	URI는 명사·복수형 으로 자원 표현, 행위는 HTTP 메서드로 — 계층 구조 표현
Web Server vs WAS	정적 자원은 Web Server, 동적 처리(비즈니스 로직)는 WAS 가 담당
OAS(OpenAPI)	API 명세 표준 규격 — 문서화·협업·코드 생성의 기준

Spring과 Spring Boot

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Spring 개요

- Java 기반의 오픈소스 애플리케이션 프레임워크
- 엔터프라이즈급 애플리케이션 개발을 위한 표준 프레임워크
- 복잡한 자바 개발을 단순화하는 컨테이너 역할

특징	설명
경량 프레임워크	필요 모듈만 사용, 메모리 부담 적음
제어의 역전(IOC)	객체 생성/관리를 개발자가 아닌 프레임워크가 담당
의존성 주입(DI)	객체 간 결합도 최소화, 유지보수성 향상
관점 지향 프로그래밍(AOP)	로깅, 보안, 트랜잭션 등 공통 기능을 분리, 코드 중복 감소
모듈화	Web, Data, Security 등 다양한 기능을 모듈화해서 관리 가능
확장성	오픈소스 커뮤니티 기반 다양한 확장 프로젝트와 연동
테스트 용이성	DI를 통한 Mock 객체 활용 등 유닛 테스트 용이

Spring의 역사

- Spring: '복잡한 Java EE의 대안' → 유연하고 강력한 자바 엔터프라이즈 개발 프레임워크
- Spring Boot: '쉽게, 빠르게 Spring을!' → 설정 최소화, 스타트업/대기업 모두 사랑하는 표준 플랫폼으로 발전

구분	Spring	Spring Boot
등장	2003년(1.0)	2014년(1.0)
목적	유연하고 강력한 엔터프라이즈 개발	빠르고 쉽게 Spring 개발, 복잡한 설정 최소화
주요 특징	IoC/DI, AOP, 다양한 모듈	자동설정, 스타터, 내장 서버, 쉬운 실행

[참고] 전자정부 프레임워크

- 대한민국 행정안전부가 공공기관 정보시스템의 개발 효율화와 품질 제고 및 유지보수의 표준화·효율화를 위해 개발한 국가 표준 **오픈소스(스프링) 기반** 소프트웨어 개발 프레임워크

구분	주요 내용
기반 프레임워크	Spring Framework, MyBatis, Maven/Gradle, log4j/slf4j 등
공통 컴포넌트	사용자, 권한, 메뉴, 파일, 암호화, 메시지, 코드 관리, 로깅 등
표준 템플릿	표준 프로젝트 구조, 기본 JSP/Controller/Service/DAO 샘플 제공
개발 도구	이클립스 기반 개발 환경(eGovFrame Dev Tool), 프로젝트 생성 지원
문서/가이드	개발 가이드, API 문서, 표준 코드 샘플, 품질 체크리스트 등

사이트 : <https://www.egovframe.go.kr/home/main.do>

Spring 생태계

프로젝트	주요 역할/설명
Spring Boot	복잡한 환경 설정을 자동화해주는 스타터(시작 도우미)
Spring Data JPA	DB 연동을 쉽고 표준화하여 개발 생산성/유지보수성 향상
Spring Security	인증/인가 등 보안 기능을 손쉽게 구현
Spring Batch	대용량 데이터 일괄 처리
Spring Cloud	마이크로서비스, 클라우드 기반 개발 지원
Spring WebFlux	비동기/논블로킹 방식 웹 애플리케이션 개발

❖ 실제 현업에서는 **Spring Boot**, **Spring Data JPA** 및 **Spring Security** 조합이 많이 쓰임

[참고] 스프링 프로젝트 사이트

<https://spring.io/projects>

 Spring Boot Takes an opinionated view of building Spring applications and gets you up and running as quickly as possible.	 Spring Framework Provides core support for dependency injection, transaction management, web apps, data access, messaging, and more.	 Spring Integration Supports the well-known Enterprise Integration Patterns through lightweight messaging and declarative adapters.	 Spring HATEOAS Simplifies creating REST representations that follow the HATEOAS principle.
 Spring Data Provides a consistent approach to data access – relational, non-relational, map-reduce, and beyond.	 Spring Cloud Provides a set of tools for common patterns in distributed systems. Useful for building and deploying microservices.	 Spring REST Docs Lets you document RESTful services by combining hand-written documentation with auto-generated snippets produced with Spring MVC Test or REST Assured.	 Spring Batch Simplifies and optimizes the work of processing high-volume batch operations.
 Spring Cloud Data Flow Provides an orchestration service for composable data microservice applications on modern runtimes.	 Spring Security Protects your application with comprehensive and extensible authentication and authorization support.	 Spring AMQP Applies core Spring concepts to the development of AMQP-based messaging solutions.	 Spring CredHub Provides client-side support for storing, retrieving, and deleting credentials from a CredHub server running in a Cloud Foundry platform.
 Spring for GraphQL Spring for GraphQL provides support for Spring applications built on GraphQL Java.	 Spring Session Provides an API and implementations for managing a user's session information.	 Spring Flo Provides a JavaScript library that offers a basic embeddable HTML5 visual builder for pipelines and simple graphs.	 Spring for Apache Kafka Provides Familiar Spring Abstractions for Apache Kafka.

Spring Boot의 등장 배경

- 전통적인 Spring 설정은 복잡한 XML/설정 파일, 환경 구축에 오랜 시간 소요
- 신규 프로젝트를 빠르게 만들고 배포하려면, 반복적/필수적인 설정을 자동화할 필요
- "Spring을 더 쉽고 빠르게!" → Spring Boot 등장

2014	Spring Boot 1.0 발표 - 자동설정(Auto Configuration), Starter 기반 의존성, 내장 서버(톰캣), JAR 실행 등
2016	Spring Boot 1.4~1.5 - 테스트 지원 강화, 보안, Actuator 등 부가 기능 확장
2018	Spring Boot 2.0 - Spring 5 기반, 리액티브 프로그래밍 정식 지원, 더 개선된 개발 경험 제공
2021~	Spring Boot 2.5~3.x - Java 17, Jakarta EE, GraalVM 네이티브 이미지 등 최신 기술 적용

Spring Boot의 주요 특징

특징	설명
자동 설정(Auto-configuration)	환경/라이브러리 감지 후 적절한 설정 자동 적용
내장 서버 지원	톰캣, 제티 등 별도 설치 없이 JAR로 바로 실행 가능
Starter에 의한 의존성 관리	프로젝트에 맞는 기본 라이브러리 세트를 starter로 한 번에 추가
복잡한 XML 미사용	어노테이션 기반의 간단한 설정, 프로퍼티(YAML/Properties) 기반
운영/모니터링 용이	헬스체크, 메트릭스 등 운영 관리 기능 기본 탑재
쉬운 배포	빌드 후 JAR 실행, Docker 등 다양한 배포 환경과 연동

[참고] 전통적인 Spring vs Spring Boot

비교 항목	전통적 Spring	Spring Boot
환경 설정	XML 및 Java Config 필요, 복잡함	자동설정, Starter, 어노테이션으로 간단함
서버 실행	WAR 패키지 + 별도 WAS 필요	JAR 패키지로 내장 서버와 함께 실행
의존성 추가	개별 라이브러리 직접 추가/관리	Starter 패키지 한 번에 의존성 관리
실습/테스트 환경	별도 설정 필요	spring-boot-devtools 등 쉽게 Hot Reload 지원
배포/운영	배포 환경에 맞게 추가 설정 필요	"java -jar" 한 줄로 실행, Docker 용이

[핵심 요약] Spring과 Spring Boot

핵심 포인트	반드시 숙지할 내용
Spring	IoC/DI를 중심으로 한 엔터프라이즈 Java 프레임워크 생태계
Spring Boot 등장	복잡한 XML 설정을 제거하기 위해 등장
Boot 핵심 특징	자동 설정(Auto-Configuration) · 내장 서버(Tomcat) · Starter 의존성 · 단독 실행(jar)
설계 철학	"설정보다 관례(Convention over Configuration)" — Boot가 대부분을 자동 구성

개발 환경 구성

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

JDK 설치

- 사이트: <https://www.oracle.com/kr/java/technologies/downloads/>
- JDK 21 설치

[Tools and resources](#) [Java downloads](#) [Java archive](#)

[JDK 24](#) [JDK 21](#) [GraalVM for JDK 24](#) [GraalVM for JDK 21](#)

Java SE Development Kit 21.0.8 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions](#) (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the [Java SE OTN](#) beyond the [limited free grants](#) of the OTN license will [require a fee](#).

[Linux](#) [macOS](#) [Windows](#)

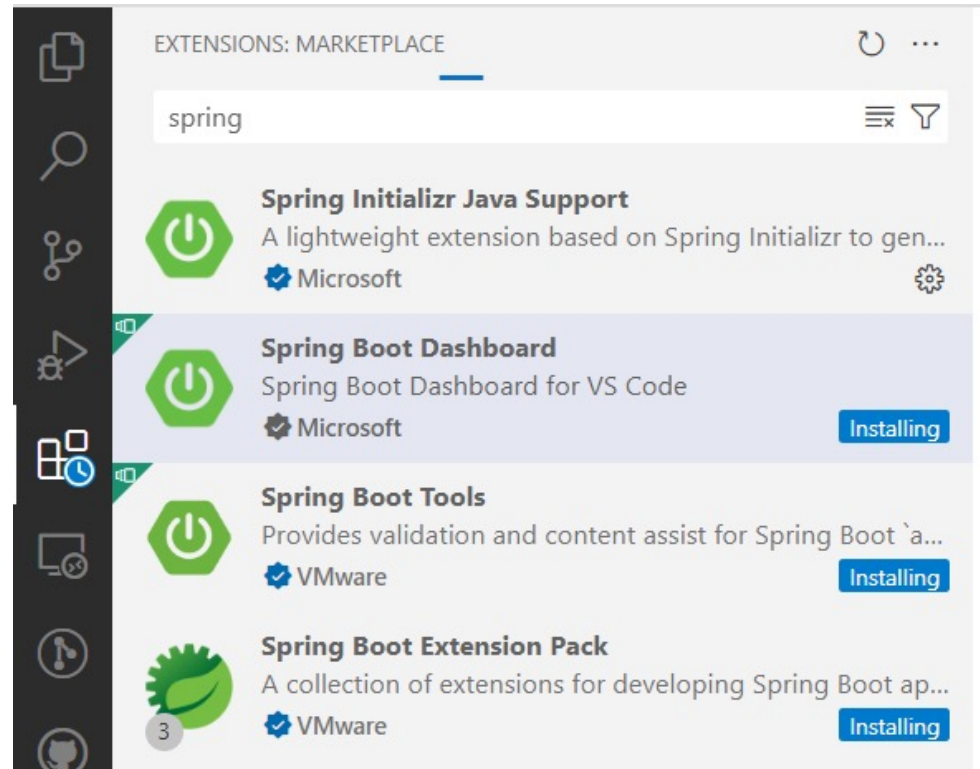
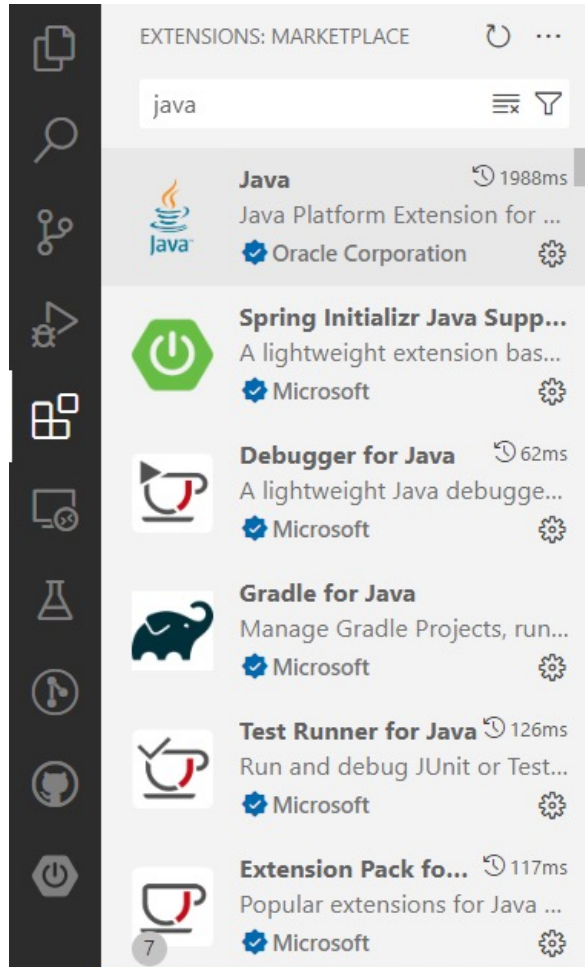
Product/file description	File size	Download
x64 Compressed Archive	186.05 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	164.42 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	163.16 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

[참고] Java, Groovy, Kotlin

- **Java : 가장 오래된, 표준적인 JVM 언어.**
 - 1995년 Sun Microsystems에서 James Gosling 등이 개발
- **Groovy : Java 친화적 스크립트/빌드 언어**
 - 2003년 Java의 복잡성을 줄이고, 스크립트 언어처럼 간결하게 JVM 기반 프로그래밍을 하자는 취지로 개발
- **Kotlin : 현대적이고 안전하며 간결한 차세대 JVM 언어.**
 - 2011년 JetBrains(인텔리제이 개발사)에서 발표, 2017년 Google이 공식 Android 개발 언어로 채택하며 성장

VS Code Extensions

- Oracle, Microsoft와 VMware 제공 Extensions 설치



Spring Initializr (실습)

- 사이트: <https://start.spring.io/>
- Project: Maven, Language: Java



Project

☐ Gradle - Groovy
 ☐ Gradle - Kotlin
 ☒ Java
 ☐ Kotlin
 ☐ Groovy

Spring Boot

☐ 4.0.0 (SNAPSHOT)
 ☐ 4.0.0 (M1)
 ☐ 3.5.5 (SNAPSHOT)
 ☒ 3.5.4
 ☐ 3.4.9 (SNAPSHOT)
 ☐ 3.4.8

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Java ☐ 24 ☒ 21 ☐ 17

Dependencies

No dependency selected

Project Metadata

- Group: com.skala
- Artifact: basic
- Packaging: Jar
- Java: 21

Dependencies

- Lombok
- Spring Configuration Processor
- Spring Web
- Spring Data JPA
- H2 Database

Maven 프로젝트 구조

경로 또는 파일	설명
/src/main/java	실제 애플리케이션의 자바 소스 코드가 위치
/src/main/java/com/skala/basic	(예시) 실제 패키지: 비즈니스 로직, 컨트롤러, 서비스 등
/src/main/resources	설정 파일, 정적 파일, 템플릿 등 리소스 파일 저장
/src/main/resources/application.yml	Spring Boot 설정 파일(yml 또는 properties)
/src/main/resources/static	정적 파일(css, js, images 등)
/src/main/resources/templates	템플릿 엔진 파일(Thymeleaf, FreeMarker 등)
/src/test/java	테스트용 자바 코드
/src/test/resources	테스트에 필요한 리소스 파일
/target	컴파일 된 클래스 파일
pom.xml	Maven 프로젝트의 핵심 설정 파일(의존성, 플러그인 등 관리)
.gitignore	git 관리에서 제외할 파일/폴더 지정
README.md	프로젝트 설명 파일

❖ 과정 완성 코드 : <https://github.com/lsmin625/skala-spring-basic.git>

Maven

- Java 프로젝트의 '빌드 자동화'와 '의존성 관리' 도구
- 프로젝트의 라이브러리, 빌드, 테스트, 배포 과정을 표준화
- pom.xml 파일에 모든 설정과 의존성을 선언 → Maven이 알아서 다운로드 및 관리

명령어	역할/설명
mvn clean	이전 빌드 결과(타겟 폴더) 삭제
mvn compile	소스 코드 컴파일
mvn test	테스트 코드 실행
mvn package	JAR/WAR 패키지 생성 (실행 파일 생성)
mvn spring-boot:run	Spring Boot 내장 서버로 프로젝트 실행
mvn dependency:tree	프로젝트 의존성(라이브러리) 구조 트리 출력

- ❖ 스프링 부트 등 최신 프로젝트로 개발하는 경우 별도 설치 필요 없음
→ VS Code와 'Extension Pack for Java'만 설치하면 프로젝트에 포함된 Maven 래퍼(mvnw)를 사용

pom.xml의 기본 구조

태그	역할/설명
<groupId>	회사/조직 도메인명(반대로, 예: com.skala)
<artifactId>	프로젝트 이름(폴더명과 유사)
<version>	프로젝트 버전
<parent>	상위(부모) pom, Spring Boot 프로젝트에서 표준 설정 상속
<dependencies>	사용할 라이브러리 선언(예: spring-boot-starter-web, lombok 등)
<build><plugins>	빌드 및 실행 플러그인(예: spring-boot-maven-plugin)
<properties>	Java 버전 등 공통 속성 지정

❖ pom.xml에서 Starter의 의미

- spring-boot-starter-web, spring-boot-starter-data-jpa 등은 Spring Boot에서 "핵심 라이브러리+필수 라이브러리" 세트를 한 번에 묶어주는 패키지
- Starter만 추가하면 세부 라이브러리를 일일이 쓸 필요 없이 자동으로 모두 포함됨

[참고] Gradle 상세 이해

구분	상세 설명
개요	Groovy/Kotlin DSL 기반 빌드 자동화 도구 — Maven의 XML 방식보다 유연하고 빠름
빌드 스크립트	build.gradle (Groovy) 또는 build.gradle.kts (Kotlin DSL) 에 빌드를 코드로 선언
핵심 블록	plugins · repositories · dependencies · tasks 로 구성
의존성 구성(scope)	implementation (컴파일+런타임) · api · testImplementation · runtimeOnly · compileOnly
의존성 표기	group:artifact:version 형식 (예: org.springframework.boot:spring-boot-starter-web)
Gradle Wrapper	gradlew / gradlew.bat — 별도 설치 없이 프로젝트 지정 버전으로 빌드(팀 버전 통일)
빠른 빌드 원리	중분 빌드 · 빌드 캐시 · 병렬 실행 · Gradle 데몬(Daemon) 으로 재빌드 시간 최소화
주요 명령어	./gradlew build · bootRun · test · clean (Maven의 mvn 명령을 대체)
build.gradle 예시 <pre> plugins { id 'org.springframework.boot' version '3.3.0' id 'io.spring.dependency-management' version '1.1.5' id 'java' } repositories { mavenCentral() } dependencies { implementation 'org.springframework.boot:spring-boot-starter-web' testImplementation 'org.springframework.boot:spring-boot-starter-test' } </pre>	

[참고] Maven vs Gradle

항목	Maven	Gradle
빌드 스크립트	XML 기반 (pom.xml)	Groovy, Kotlin DSL 기반 (build.gradle, build.gradle.kts)
의존성 관리	간단, 명확	강력, 유연, 커스텀하기 쉬움
성능	느린 편 (캐시/병렬처리 약함)	빠름 (빌드 캐시, 병렬 빌드 지원)
사용성	표준화, 초보자에게 익숙함	초보자에게엔 어렵지만, 커스터마이징과 자동화가 강력
문서화	방대하고 오래됨	계속 개선 중, 최근 문서 많아짐
빌드 출력	표준화된 폴더 구조	자유롭게 구조화 가능
학습 곡선	완만함	초반 진입장벽 있음
주요 사용 예	전통적 Java/Spring 프로젝트, 기업 환경	Android, 최신 Spring, 마이크로서비스 등 다양
대표 명령어	mvn clean install	gradle build (또는 ./gradlew build)
Spring 공식 지원	공식 지원, 샘플 코드 多	공식 지원, 점점 확대

- ❖ **Maven:** 구조가 명확하고 보수적인 기업 환경에 많이 쓰임. 표준화된 작업, 문서, 초보자 학습에 유리.
- ❖ **Gradle:** 속도 빠르고 빌드 자동화에 강함. 현대적 Spring 개발, 대규모 멀티 프로젝트에 인기.

[참고] JAR 구조 이해하기

example-0.0.1-SNAPSHOT.jar

```
|— META-INF/
|   |— MANIFEST.MF           # Main-Class 정보 포함
|— BOOT-INF/
|   |— classes/              # 애플리케이션 클래스 (com.example.Application 등)
|   |— lib/                  # 모든 의존 라이브러리 JAR 포함 (jarLauncher에 로딩 예정)
|— org.springframework.boot.loader/ # spring framework 라이브러리이며, JarLauncher 포함
```


Postman 설치

- 사이트: <https://www.postman.com/downloads/>
- OS 플랫폼에 맞게 설치

Download Postman

Download the app to get started using the Postman API Platform today. Or, if you prefer a browser experience, you can try the web version of Postman.

The Postman app

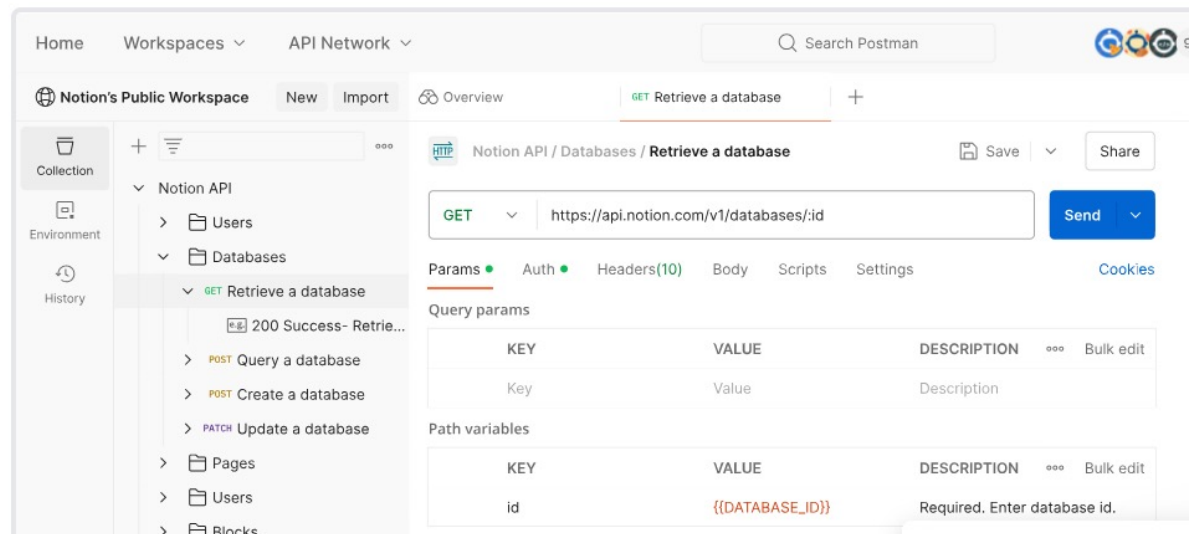
Download the app to get started with the Postman API Platform.

 Windows 64-bit

By downloading and using Postman, I agree to the [Privacy Policy](#) and [Terms](#).

[Release Notes](#) →

Not your OS? Download for Mac ([Intel Chip](#), [Apple Chip](#)) or Linux (x64, arm64)



[코드] BasicApplication.java

```
package com.skala.basic;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BasicApplication {

    public static void main(String[] args) {
        SpringApplication.run(BasicApplication.class, args);
    }

}
```

- **@SpringBootApplication** : 여러개의 스프링 부트 설정 어노테이션이 합쳐진 메타 어노테이션으로 **컴포넌트 스캔**을 진행
- **SpringApplication.run(BasicApplication.class, args)** : 스프링 부트 애플리케이션을 실행하는 명령어

[샘플코드] 개발 환경 구성 (Spring Boot 시작)

```
// build.gradle
// implementation 'org.springframework.boot:spring-boot-starter-web'

package com.sk.skala.shopapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication          // 자동설정 + 컴포넌트 스캔 + 설정 등록
public class ShopApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(ShopApiApplication.class, args);
        // 내장 Tomcat 기동 -> http://localhost:8080
    }
}
```

[핵심 요약] 개발 환경 구성

핵심 포인트	반드시 숙지할 내용
필수 도구	JDK 설치 + VS Code Extensions(Spring 지원)
프로젝트 생성	Spring Initializr (start.spring.io)로 스켈레톤 생성
Maven	Java 빌드 자동화 + 의존성 관리 도구, 설정은 pom.xml
Gradle	Groovy/Kotlin DSL 기반 빌드 도구, 설정은 build.gradle — 증분 빌드·캐시로 빠름
Starter	특정 기능에 필요한 의존성 묶음 — 추가만 하면 관련 라이브러리 자동 구성
Postman	REST API 요청·응답 테스트 도구

Springboot 핵심기능

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

의존성 주입과 IoC 컨테이너

IoC (Inversion of Control)

■ 개념

- 객체의 생성 및 의존성 관리를 개발자가 아닌 Spring 컨테이너가 대신 수행하는 방식
- 개발자가 `new`로 직접 생성하지 않고 Spring이 대신 객체를 생성 및 관리
- 이를 통해 코드의 결합도를 낮추고 유지보수와 테스트가 쉬워짐

■ 기존 방식의 문제점

- 클래스들간 강결합 (구현체 클래스에 직접 의존)
- 코드 수정 및 유지보수 어려움

ex.

```
public class OrderService {  
    private OrderRepository orderRepository = new OrderRepository(); // 직접 생성 (강결합)  
}
```

의존성 주입과 IoC 컨테이너

IoC (Inversion of Control)

■ 기존 방식의 문제점 해결

- 객체생성과 주입은 Spring IoC 컨테이너가 담당
- 개발자는 단지 연결 관계(interface)만 정의
- 코드 변경없이 다른 구현체로 쉽게 교체 가능 (유연하고 테스트 용이)

ex.

```
public class JpaOrderRepository implements OrderRepository{}  
public class JdbcOrderRepository implements OrderRepository{}
```

```
public class OrderService {  
    private final OrderRepository orderRepository;  
    // 외부에서 주입 받음 ( 약결합 )  
    public OrderService(OrderRepository orderRepository) {  
        this.orderRepository = orderRepository;  
    }  
}
```

의존성 주입과 IoC 컨테이너

기존 방식 vs IoC 방식

항목	기존방식 (직접 제어)	IoC 방식 (Spring 컨테이너 제어)
객체 생성	개발자가 직접 new 로 생성	컨테이너가 자동 생성 @Component,@Service, @Controller,@Repository 등
의존성 주입	직접 코드내에서 주입	생성자 또는 @Autowired 이용
관리 주체	개발자	Spring IoC 컨테이너
테스트 용이성	낮음 (강결합)	높음(느슨한 결합)
유지보수성	낮음	높음

의존성 주입과 IoC 컨테이너

IoC 컨테이너

■ 개념

- Spring 에서 IoC는 컨테이너(Container)를 통해 구현
- IoC 방법으로 Bean을 관리한다는 의미에서 IoC 컨테이너라고 부름
- 컨테이너 자체도 Bean으로서 BeanFactory와 ApplicationContext가 컨테이너가 구현해야 될 기본 인터페이스임

■ 주요 역할

기능	설명
Bean 생성	설정정보(@Configuration, @ComponentScan) 기반으로 Bean 자동생성
의존성 주입	필요한 객체를 자동으로 연결 (생성자 또는 @Autowired 이용)
생명주기 관리	Bean 의 생성 -> 초기화 -> 사용 -> 소멸까지의 전 과정을 관리
설정 및 환경 관리	외부 설정 파일 (application.yml)과 연동

의존성 주입과 IoC 컨테이너

IoC 컨테이너 주요 인터페이스

인터페이스	설명
BeanFactory	가장 기본적인 IoC 컨테이너
ApplicationContext	BeanFactory를 확장한 고급 컨테이너 메시지소스, 이벤트 발행, 환경설정 관리 등 지원
WebApplicationContext	웹 어플리케이션용 IoC 컨테이너 Spring MVC와 연동됨

의존성 주입과 IoC 컨테이너

IoC 관련 주요 특징

특징	설명
제어의 역전	객체의 생성 및 관리를 IoC 컨테이너가 담당
의존성 주입(DI)	객체 간의 관계를 자동으로 연결
느슨한 결합	코드 간 의존성이 낮아 유지보수 용이
확장성 향상	설정 변경만으로 쉽게 구현체 교체 가능
테스트 용이성	Mock 객체 활용하여 단위 테스트 가능
재사용성 향상	Bean 재활용 가능 (Singleton 기본)

의존성 주입과 IoC 컨테이너

POJO (Plain Old Java Object)

- 개념
 - 특정 프레임워크 규약/환경에 종속되지 않는 순수한 객체 의미
 - Spring 은 POJO 중심으로 Bean들을 결합해 확장성/테스트 용이성 보장
- 특징
 - Spring 프레임워크 핵심
 - 특정 규약에 종속되지 않음
 - 특정 환경에 종속되지 않음
 - 객체 지향적인 원리에 충실

의존성 주입과 IoC 컨테이너

의존성 주입 (Dependency Injection, DI)

■ 개념

- 객체가 의존하는 다른 객체를 직접 생성하지 않고 외부에서 주입 받는 방식
- 핵심원리는 IoC 컨테이너가 객체 간의 관계를 자동으로 연결해줌

■ 특징

- IoC의 구체적인 구현 방법
- 객체 간 결합도(Coupling)를 낮춤
- 유연성과 재사용성 향상

의존성 주입과 IoC 컨테이너

의존성 주입 3가지 방식

주입방식	설명	장점	단점
생성자 주입 (권장) (Constructor Injection)	생성자를 통해서 의존 객체를 전달	불변성 보장, 필수 의존성 명확	순환 참조 시 이슈 가능
Setter 주입 (Setter Injection)	Setter 메서드를 통해서 주입	선택적 의존성 처리 가능	객체가 완전하지 않은 상태로 사용될 수 있음.
필드 주입 (Field Injection)	@Autowired로 필드 직접 주입	코드 간결	테스트/DI 프레임워크 외부 사용 어려움

[참고] 생성자 주입이 필드 주입보다 권장되는 이유

- 생성자 주입이 필드 주입보다 더 안전하고, 견고한 코드를 작성할 수 있게 해주기 때문에 생성자 주입을 권장

구분	상세 설명
불변성(Immutable) 보장	final 키워드 사용 가능. 객체 생성 후 의존성 변경 불가 → 안정성 증가
필수 의존성 보장	필수 의존성을 생성자 파라미터로 강제하여 누락 시 컴파일/런타임 에러 → 오류 조기 발견
테스트 용이성	단위 테스트 시 생성자를 통해 mock/fake 객체 직접 주입 가능 → 테스트 코드 작성 편리
순환 참조 조기 감지	순환 참조 발생 시 애플리케이션 구동 시점에 즉시 에러 발생 → 빠른 버그 확인
의존성 명확성 및 가독성	생성자 시그니처만 봐도 어떤 의존성이 필요한지 한눈에 파악 → 코드 가독성 및 유지보수 용이

의존성 주입과 IoC 컨테이너

생성자 주입 (Constructor Injection) 예시

```
@Service
public class OrderService {
    private final OrderRepository orderRepository;

    // 생성자 기반 주입
    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    public void processOrder() {
        orderRepository.saveOrder();
    }
}
```

❖ 생성자 주입에서 **private final** 선언의 필요성

- final로 선언하면 해당 필드는 초기화 이후 값이 변하지 않는 불변(immutable) 객체가 됨.
- 생성자를 통해 반드시 1번만 값이 할당되고, 이후 변경 불가 → 안정성과 신뢰성 확보
- Spring이 주입할 때도 반드시 생성자를 통해 값을 넣기 때문에, 필수 의존성이 반드시 주입 되었음을 컴파일 타임에 확인할 수 있음

의존성 주입과 IoC 컨테이너

Setter 주입 (Setter Injection) 예시

```
@Service
public class PaymentService {
    private PaymentRepository paymentRepository;

    // Setter 기반 주입
    @Autowired
    public void setPaymentRepository(PaymentRepository paymentRepository) {
        this.paymentRepository = paymentRepository;
    }

    public void pay() {
        paymentRepository.savePayment();
    }
}
```

의존성 주입과 IoC 컨테이너

필드 주입 (Field Injection) 예시

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public void printUser() {
        System.out.println(userRepository.findUser());
    }
}
```

의존성 주입과 IoC 컨테이너

DI 적용 흐름



의존성 주입과 IoC 컨테이너

실습 프로젝트 구조 (DI)

```
src/main/java/  
  com/  
    example/  
      demo/  
        DemoApplication.java          # @SpringBootApplication  
        repository/  
          UserRepository.java          # @Repository  
        service/  
          UserService.java             # @Service
```

의존성 주입과 IoC 컨테이너

실습(DI)

```
// UserRepository.java
package com.example.demo.repository;

import org.springframework.stereotype.Repository;

@Repository
public class UserRepository {

    public String findUser() {
        return "홍길동";
    }
}
```

```
// UserService.java
package com.example.demo.service;

import org.springframework.stereotype.Service;
import com.example.demo.repository.UserRepository;

@Service("userService")
public class UserService {

    private UserRepository userRepository;

    // 생성자 주입
    public UserService(UserRepository userRepository){
        this.userRepository = userRepository;
    }

    public void printUser() {
        System.out.println("사용자: " + userRepository.findUser());
    }
}
```

의존성 주입과 IoC 컨테이너

실습(DI)

```
// DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.example.demo.service.UserService;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {

        ApplicationContext ctx = SpringApplication.run(DemoApplication.class, args);
        UserService service = ctx.getBean("userService", UserService.class);
        service.printUser();

    }
}
```

의존성 주입과 IoC 컨테이너

@Autowired 동작 원리

■ 개념

- IoC 컨테이너에 등록된 Bean 중에서 **타입(Type)**이 일치하는 객체를 자동으로 주입함
- 만약 타입(Type)이 일치하는 객체가 여러 개인 경우 필드명이 일치하는 객체가 주입됨.

■ 예외 사항

- 타입(Type)이 일치하는 Bean이 둘 이상인 경우 예외 발생 가능성 있음
ex. 필드명이 일치하는 객체가 없는 경우

■ 예외 사항 해결

- @Qualifier 또는 @Primary 이용해서 명시적으로 주입할 객체 지정

의존성 주입과 IoC 컨테이너

@Qualifier 예시 (특정 Bean 지정)

```
@Repository("mysqlRepository")  
public class MySqlOrderRepository implements OrderRepository {}
```

```
@Repository("mongoRepository")  
public class MongoOrderRepository implements OrderRepository {}
```

```
@Service  
public class OrderService {  
    @Autowired  
    @Qualifier("mongoRepository")  
    private OrderRepository repository;  
}
```


의존성 주입과 IoC 컨테이너

@Primary 예시 (기본 Bean 우선 지정)

@Repository

@Primary

```
public class DefaultOrderRepository implements OrderRepository {}
```

@Repository("mysqlRepository")

```
public class MySqlOrderRepository implements OrderRepository {}
```

@Repository("mongoRepository")

```
public class MongoOrderRepository implements OrderRepository {}
```

@Service

```
public class OrderService {
```

@Autowired

```
    private OrderRepository repository;
```

```
}
```

의존성 주입과 IoC 컨테이너

실습 프로젝트 구조 (@Qualifier)

```
src/main/java/  
  com/  
    example/  
      demo/  
        DemoApplication.java           # @SpringBootApplication  
        repository/  
          BaseRepository.java          # 인터페이스  
          UserRepository.java          # @Repository  
          ProductRepository.java       # @Repository  
        service/  
          BaseService.java             # @Service
```

의존성 주입과 IoC 컨테이너

실습 (@Qualifier)

```
// BaseRepository.java
package com.example.demo.repository;

import java.util.List;

public interface BaseRepository {
    public List<String> findAll();
}

// UserRepository.java
package com.example.demo.repository;

import java.util.Arrays;
import java.util.List;
import org.springframework.stereotype.Repository;

@Repository("productRepository")
public class ProductRepository implements BaseRepository{

    @Override
    public List<String> findAll() {
        return Arrays.asList("노트북", "냉장고", "핸드폰");
    }
}
```

```
// UserRepository.java
package com.example.demo.repository;

import java.util.Arrays;
import java.util.List;

import org.springframework.stereotype.Repository;

@Repository("userRepository")
public class UserRepository implements BaseRepository{

    @Override
    public List<String> findAll() {
        return Arrays.asList("홍길동", "이순신", "유관순");
    }
}
```

의존성 주입과 IoC 컨테이너

실습 (@Qualifier)

```
// BaseService.java
package com.example.demo.service;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;
import com.example.demo.repository.BaseRepository;

@Service("baseService")
public class BaseService {

    private BaseRepository baseRepository;

    // 주입하고자 하는 Bean 이름 지정, 생략 시 예외 발생
    public BaseService(@Qualifier("productRepository")
                      BaseRepository baseRepository){
        this.baseRepository = baseRepository;
    }

    public void printAll() {
        System.out.println("데이터: " + baseRepository.findAll());
    }
}
```

```
// DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.example.demo.service.BaseService;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        ApplicationContext ctx = SpringApplication.run(DemoApplication.class, args);
        BaseService service = ctx.getBean("baseService", BaseService.class);
        service.printAll();
    }
}
```

의존성 주입과 IoC 컨테이너

Bean 생명주기 콜백 메서드

■ 개념

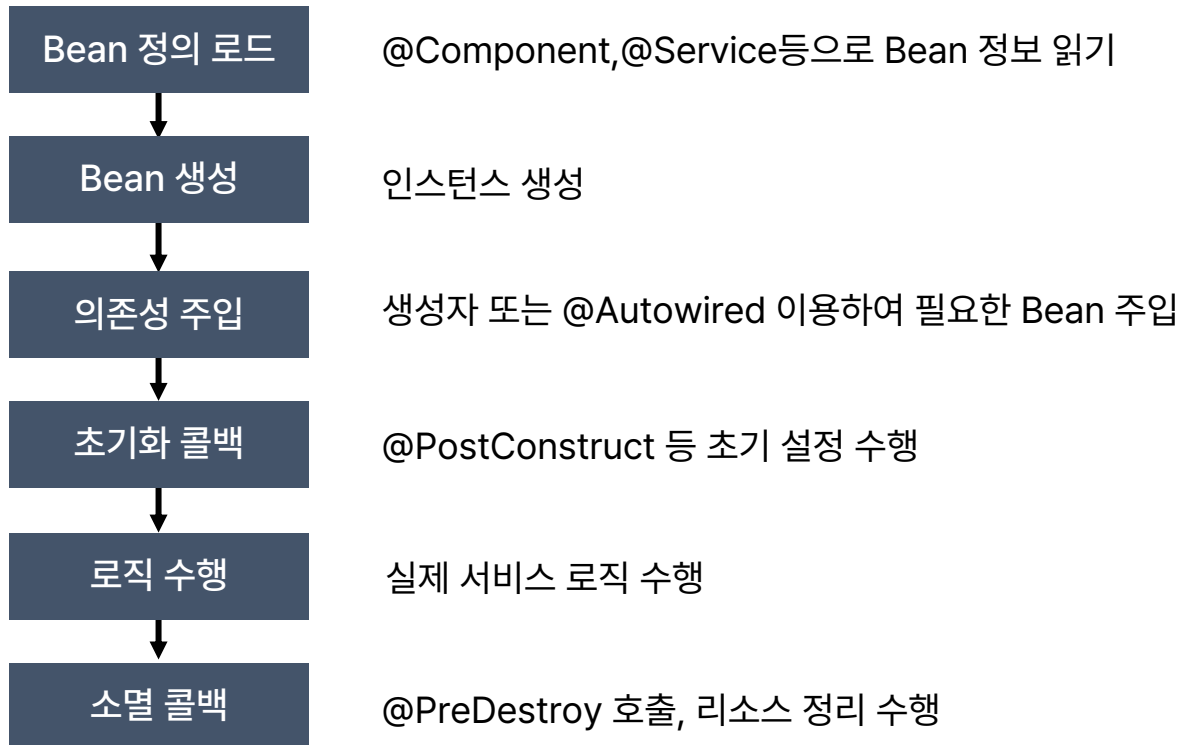
IoC 컨테이너에 저장되는 Bean 객체는 생성/초기화/소멸의 생명주기를 가짐
이때 특정 상태에서 자동으로 호출되는 콜백 메서드를 사용하여 빈 객체의 상태를
정교하게 제어 가능

■ 방법

어노테이션	설명
@PostConstruct	Bean 생성 및 의존성 주입이 끝난 직후 1회 호출 컨테이너가 해당 메서드를 실행한 후 Bean이 준비상태가 됨
@PreDestroy	어플리케이션 컨텍스트가 종료 될 때 cleanup 용으로 1회 호출 연결 자원 정리, 스레드 풀 종료, 버퍼 flush 등 수행

의존성 주입과 IoC 컨테이너

Bean 생명주기 흐름



의존성 주입과 IoC 컨테이너

실습 (생명주기)

```
// UserService.java
package com.example.demo.service;

import org.springframework.stereotype.Service;
import com.example.demo.repository.UserRepository;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;

@Service("userService")
public class UserService {

    private UserRepository userRepository;

    public UserService(UserRepository userRepository){
        System.out.println("1. UserService 생성 및 UserRepository 의존성 주입");
        this.userRepository = userRepository;
    }

    public void printUser() {
        System.out.println("3. 서비스 로직 호출: " + userRepository.findUser());
    }
}
```

```
@PostConstruct
public void init() {
    System.out.println("2. Bean 초기화, PostConstruct 호출");
}

@PreDestroy
public void destroy() {
    System.out.println("4. Bean 소멸초기화, PreDestroy 호출");
}
}
```

[샘플코드] 의존성 주입 (DI / IoC)

@Service

```
public class ProductService {  
  
    private final ProductRepository repository;    // final (불변)  
  
    // 생성자 주입(권장): 컨테이너가 Bean 을 자동 주입  
    public ProductService(ProductRepository repository) {  
        this.repository = repository;  
    }  
  
    public Product findById(Long id) {  
        return repository.findById(id)  
            .orElseThrow(() -> new NoSuchElementException("not found"));  
    }  
}  
// @RequiredArgsConstructor 를 쓰면 final 필드 생성자를 자동 생성
```


[핵심 요약] 의존성 주입과 IoC 컨테이너

핵심 포인트	반드시 숙지할 내용
IoC(제어의 역전)	객체 생성·생명주기를 개발자가 아닌 컨테이너가 관리 → 결합도 ↓ · 테스트 용이
DI 3가지 방식	생성자 · Setter · 필드 주입
생성자 주입 권장	불변·필수 의존성 보장 , 테스트 쉬움, 순환 참조를 기동 시 조기 발견
ApplicationContext	가장 널리 쓰는 IoC 컨테이너 (BeanFactory 확장)
빈 선택 제어	동일 타입 빈이 여러이면 @Qualifier / @Primary 로 지정

Configuration과 Profile 관리

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Configuration과 Profile 관리

Configuration

■ 개념

- 어플리케이션 실행 시 필요한 각종 환경 정보 (서버 포트, DB, API Key, 로그레벨 등)를 코드가 아닌 외부 파일 또는 환경변수로 분리해 관리하는 것을 의미
- 즉, 코드를 수정하지 않고 환경에 따라 설정만 바꿔 어플리케이션 실행이 가능하도록 지원

■ 구성요소 예시

설정 항목	예시
서버 포트	server.port=8080
DB 연결 정보	spring.datasource.url=jdbc:mariadb://localhost:3306/test
로그 레벨	logging.level.org.springframework=DEBUG
외부 API Key	weather.api.key=1234567890abcdef

Configuration과 Profile 관리

Configuration 문제점 및 해결

■ 기존 방식 문제점

- 설정이 코드에 하드코딩 되어 환경이 바뀌면 재컴파일 필요
- 테스트 환경 및 운영 환경이 달라 오류 발생 가능성 증가
- 민감한 정보(DB 비밀번호, API Key 등)가 코드에 노출

■ 문제점 해결

- 설정 정보를 코드 밖 (application.yml, 환경 변수 등)으로 추출
- 실행 시점에 필요한 설정을 주입 받아서 실행
- 재배포 없이 실행 환경 전환 가능

Configuration과 Profile 관리

Configuration 특징

- 유지보수성 향상
 - 설정 변경 시 코드 수정 없이 환경 전환 가능
- 보안성 확보
 - 비밀번호, API Key 등 민감정보를 코드 외부로 분리
- CI/CD 자동화
 - 환경 변수 기반 자동 빌드 및 배포 지원
- 가시성 향상
 - 설정 파일만 봐도 시스템 구조 파악 가능

Configuration과 Profile 관리

Configuration 구조

SpringApplication

Environment

PropertySources (리스트형)

CommandLinePropertySource

SystemProperties

SystemEnvironment

ConfigData (application.yml)

DefaultProperties

Profiles

ActiveProfiles (spring.profiles.active)

DefaultProfiles (spring.profiles.default)

Configuration과 Profile 관리

Environment

■ 개념

Spring Boot 어플리케이션이 외부 설정값 (application.yml, 환경변수 등)을 읽어와 현재 실행 환경(dev/test/prod)을 구분하며 관리하는 중앙 제어 객체
어플리케이션이 시작될 때 자동 생성되어 모든 Bean, 설정파일, AutoConfiguration의 값 바인딩 근거가 됨.

■ 핵심 기능

기능	설명
Property 관리	다양한 설정 소스 (yml, 명령줄)의 값을 통합 저장
Profile 관리	실행 환경 (개발, 테스트, 운영 등) 구분 및 활성화
값 조회 제공	@Value, @ConfigurationProperties 등에서 사용

Configuration과 Profile 관리

Environment 필요성

- 외부 환경 별 유연한 설정관리

개발(dev), 운영(prod) 환경별로 DB, 서버포트, 로깅 등 설정을 분리

- 코드와 설정 분리 (Configuration Separation)

설정값이 코드에 고정되지 않도록 분리

유지보수 용이

- 다양한 설정 소스 통합 관리

CLI 옵션, 시스템 변수, yml 파일 등 여러 접근을 하나로 관리

- 자동 구성(AutoConfiguration) 기반

Spring Boot의 자동 설정은 Environment에서 값을 가져와 동작

Configuration과 Profile 관리

PropertySource

■ 개념

- 설정값(key/value)을 보관하는 하나의 저장소 단위
- Environment 내에 리스트로 순서대로 쌓고 Top-Down 방식으로 조회

■ 기존 방식 문제점

- 개발/운영 환경마다 DB, 포트, 로그 레벨이 다름
- 설정 방식이 다양 ex. 명령줄, OS 환경변수, yml, 코드 기본값
- 같은 key 존재 시 어느 값이 적용되는지 혼란

■ 문제점 해결

- PropertySource를 리스트로 쌓고 위에서부터 값을 조회 (우선순위 규칙)
- Environment 가 최종 1개 값으로 정리해 반환

Configuration과 Profile 관리

PropertySource 우선 순위

우선순위	이름	출처	예시	비고
1	commandLineArgs	명령줄 인자	--server.port=9090	최상위, java -jar 옵션
2	systemProperties	JVM 인자	-Dserver.port=8085	JVM 옵션
3	systemEnvironment	OS 환경변수	SERVER_PORT=8083	CI/CD, Docker 사용
4	applicationConfig	yml/properties	server.port=8080	파일 기반 구성
5	defaultProperties	코드 주입	setDefaultProperties	최하위

Configuration과 Profile 관리

실습 (env)

```
# application.yml
server:
  port: 8080

app:
  name: My Demo App
  version: 1.0.0
  mode: local

// DemoApplication.java
package com.example.demo;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

```
/**
 * CommandLineRunner:
 * 애플리케이션이 실행된 직후 실행되는 콜백 인터페이스
 * run(String... args) 재정의 필요
 */
@Component
class EnvPrinter implements CommandLineRunner {

    private final Environment env;

    // @Value로 직접 주입받기
    @Value("${app.name}")
    private String appName;

    @Value("${app.version}")
    private String appVersion;

    // Environment 객체 자동 주입
    public EnvPrinter(Environment env) {
        this.env = env;
    }
}
```

Configuration과 Profile 관리

실습 (env)

```
@Override
public void run(String... args) throws Exception {
    System.out.println(" Spring Boot Environment 실습");

    // 1. Environment 객체를 통한 값 조회
    System.out.println("server.port = " + env.getProperty("server.port"));
    System.out.println("app.mode = " + env.getProperty("app.mode"));

    // 2. @Value로 주입받은 값
    System.out.println("app.name = " + appName);
    System.out.println("app.version = " + appVersion);

    // 3. 활성 프로파일 출력
    String[] activeProfiles = env.getActiveProfiles();
    if (activeProfiles.length == 0) {
        System.out.println("active.profiles = (none)");
    } else {
        System.out.println("active.profiles = " + String.join(", ", activeProfiles));
    }
    System.out.println(" Environment 테스트 완료");
}
```

Configuration과 Profile 관리

값 주입(Value Injection)

■ 개념

- application.yml 파일에 정의된 설정 값을 자바 코드(Bean)내부로 전달하여 사용하는 방법

■ 구현

구분	주용도	어노테이션	특징
단일값	간단한 값 ex. 문자열,숫자	@Value	빠르고 간단
여러값	구조적/복합 설정	@ConfigurationProperties	타입 안정, 유지보수 탁월

Configuration과 Profile 관리

@Value

- 개념
 - 한두 개의 단일 값을 빠르게 주입할 때 사용
 - application.yml의 특정 키를 읽어서 변수에 직접 할당
- 문법
 - `@Value("${특정 key값:기본값}")`

Configuration과 Profile 관리

@Value 장단점

- 장점
 - 간단하고 빠름
 - 별도의 클래스 정의 필요 없음
- 단점
 - 값이 여러 클래스에 흩어짐 (스파게티 구조 위험)
 - 타입 안정성 및 유효성 검증 불가
 - 대규모 프로젝트에서는 유지보수 어려움

Configuration과 Profile 관리

실습 (@Value)

```
# application.yml
spring:
  application:
    name: demo
```

```
server:
  port: 8080
```

```
// DemoApplication.java
package com.example.demo;
```

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class DemoApplication implements CommandLineRunner{
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
        System.out.println("DemoApplication");
    }
}
```

```
@Value("${spring.application.name}")
private String appName; // demo
```

```
@Value("${server.port}")
private int port; // 8080
```

```
@Value("${spring.application.name:app}-${server.port:8081}")
private String composed; // demo-8080
```

```
@Override
public void run(String... args) throws Exception {
    System.out.println("appName:" + appName);
    System.out.println("port:" + port);
    System.out.println("composed:" + composed);
}
}
```


Configuration과 Profile 관리

@ConfigurationProperties

■ 개념

- 관련된 여러 설정 값을 하나의 객체(Bean)로 묶어 주입
- 계층형 구조인 YAML과 자연스럽게 매핑됨
- 타입 검증과 자동완성 지원
- Spring Boot에서 권장하는 표준적인 설정 주입 방식

■ 특징

- 구조적 바인딩
- 타입 안정성(Type-safe)
- 자동 변환 지원
- 검증 (Validation) 지원
- IDE 자동완성 지원
- 스캔 가능

Configuration과 Profile 관리

@ConfigurationProperties

- 구현 방법

- A. 스캔 활성화

- ex. **@ConfigurationPropertiesScan**
@SpringBootApplication
public class DemoApplication{}

- B. 프로퍼티 클래스 정의

- ex. **@ConfigurationProperties (prefix="app")**
public record AppProps {}

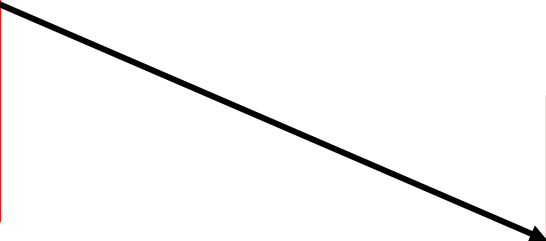
- C. 주입 및 사용

- ex. **@Autowired**
AppProps appProps;

실습 (@ConfigurationProperties)

application.yml

```
app:
  name: demo
  cors:
    allowed-origins:
      - https://a.com
      - https://b.com
  metadata:
    owner: ops
    region: ap-northeast-2
  admins: ["alice", "bob"]
  timeout: 30s
  max-upload: 25MB
  mode: PRO
```



// AppProps.java

```
package com.example.demo.config;
```

```
import java.time.Duration;
```

```
import java.util.List;
```

```
import java.util.Map;
```

```
import org.springframework.boot.context.properties.ConfigurationProperties;
```

```
import org.springframework.util.unit.DataSize;
```

```
@ConfigurationProperties(prefix = "app")
```

```
public record AppProps(
```

```
    String name,
```

```
    Cors cors,
```

```
    Map<String, String> metadata,
```

```
    List<String> admins,
```

```
    Duration timeout,
```

```
    DataSize maxUpload,
```

```
    Mode mode
```

```
) {
```

```
    public record Cors(List<String> allowedOrigins) {}
```

```
    public enum Mode { BASIC, PRO, ENTERPRISE }
```

```
}
```

Configuration과 Profile 관리

실습 (@ConfigurationProperties)

```
// DemoApplication.java
package com.example.demo;

import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
import com.example.demo.config.AppProps;

@SpringBootApplication
@ConfigurationPropertiesScan
public class DemoApplication implements CommandLineRunner{
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
        System.out.println("DemoApplication");
    }
}
```

```
@Autowired
AppProps props;

@Override
public void run(String... args) throws Exception {

    Map<String, Object> appProps = Map.of(
        "name", props.name(),
        "origins", props.cors().allowedOrigins(),
        "admins", props.admins(),
        "timeout", props.timeout().toSeconds(),
        "mode", props.mode().name()
    );
    System.out.println("appProps:" + appProps);
}
```

Configuration과 Profile 관리

비교 요약

비교 항목	@Value	@ConfigurationProperties
주입 대상	단일 값	여러 값(객체 단위)
타입 변환	문자열 기반 수동 변환	자동 변환 ex. Duration, DataSize, Enum 등
검증 기능	불가	@Validated 로 검증 가능
IDE 자동완성	불가	가능 (prefix 기반)
유지 보수	낮음	높음
사용 예	간단한 값	구조적/프로젝트 전역 설정

Configuration과 Profile 관리

검증(Validation)

- 개념
 - 입력값이나 설정값이 올바른지 확인하는 과정
 - @ConfigurationProperties 등의 데이터가 규칙에 맞는지 자동으로 검사하는 기능 제공
- 의존성 추가
 - implementation 'org.springframework.boot:spring-boot-starter-validation'
- 포함 라이브러리
 - Jakarta.validation-api
검증 표준 인터페이스 제공 ex. @NotBlank, @Max, @Min 등
 - hibernate-validator
실제 검증 로직 구현체

Configuration과 Profile 관리

제약(Constraint) 어노테이션

어노테이션	설명	예시
@NotNull	null 금지	@NotNull String host
@NotBlank	공백 문자열 금지	@NotBlank String password
@Min, @Max	숫자 범위 제한	@Min(1) @Max(10)
@Size	문자열/리스트 크기 제한	@Size (min=3, max=10)
@Email	이메일 형식	@Email String email
@Pattern	정규식 매칭	@Pattern(regexp="^[a-z]"

Validation 주요 특징

항목	설명
타입 안정성	잘못된 타입 자동 감지
유효성 제약 설정 가능	@NotBlank, @Min, @Max 등 조건 지정
부팅 시점 검증	설정값 오류는 앱 시작 전에 실패
자동 오류 메시지 출력	어떤 값이 잘못됐는지 명확하게 표시
@Validated 필수	검증 활성화 방법으로 클래스에 지정
검증 실패 시 부팅 중단	어플리케이션이 실행되지 않음

Configuration과 Profile 관리

실습 (Validation)

```
# build.gradle

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

```
# application.yml
db:
    url: jdbc:mysql://localhost:3306/demo
    username: root
    password: "password"
    pool-size: 10
```

Configuration과 Profile 관리

실습 (Validation)

```
// DbProps.java
package com.example.demo.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.validation.annotation.Validated;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

@Validated
@ConfigurationProperties(prefix = "db")
public record DbProps(
    @NotBlank String url,
    @NotBlank String username,
    @NotBlank String password,
    @Min(1) @Max(10) int poolSize
) {}
```

Configuration과 Profile 관리

실습 (Validation)

```
// DemoApplication.java
package com.example.demo;

import java.util.Map;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
import com.example.demo.config.DbProps;

@SpringBootApplication
@ConfigurationPropertiesScan
public class DemoApplication implements CommandLineRunner{
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
        System.out.println("DemoApplication");
    }
}
```

```
@Autowired
DbProps props;

@Override
public void run(String... args) throws Exception {

    Map<String, Object> appProps = Map.of(
        "url", props.url(),
        "username", props.username(),
        "password", props.password(),
        "pool-size", props.poolSize()
    );
    System.out.println("appProps:" + appProps);
}
```

Configuration과 Profile 관리

실행 결과 (위배된 경우)

Error starting ApplicationContext. To display the condition evaluation report re-run your application with 'debug' enabled.
2025-10-08T11:57:21.862+09:00 ERROR 6076 --- [main] o.s.b.d.LoggingFailureAnalysisReporter :

```
*****  
APPLICATION FAILED TO START  
*****
```

Description:

Binding to target com.example.demo.config.DbProps failed:

Property: db.password
Value: ""
Origin: class path resource [application.yml] - 5:13
Reason: 공백일 수 없습니다

Property: db.poolSize
Value: "15"
Origin: class path resource [application.yml] - 6:14
Reason: 10 이하여야 합니다

Configuration과 Profile 관리

Profile

■ 개념

- dev, test, prod 등 환경별로 다른 설정을 자동으로 적용하는 기능
- 즉 현재 실행중인 프로파일이 무엇인지에 따라서 다른 DB 설정, 다른 로깅 설정,
- 다른 서버 포트 등 서로 다른 설정이 가능

■ 주요 특징

구분	설명
환경 분리	개발(dev), 테스트(test), 운영(prod) 환경에 따라 서로 다른 설정 파일을 분리 가능
유연한 실행	실행시점에 <code>--spring.profiles.active</code> 값만 변경해서 손쉽게 환경 전환이 가능
자동 병합	공통 설정(application.yml)과 환경별 설정(application-{프로파일}.yml)을 자동 병합
코드 수준 분리	@Profile 어노테이션을 사용하여 특정 환경에서만 Bean 등록 가능

Configuration과 Profile 관리

Profile

■ 구성 파일

문법: application-{프로파일}.yml

ex.

공통: application.yml

개발: application-**dev**.yml

테스트: application-**test**.yml

운영: application-**prod**.yml

■ 활성화

java -jar myapp.jar --spring.profiles.active=prod

gradlew bootRun --args='--spring.profiles.active=prod'

Configuration과 Profile 관리

실습 프로젝트 구조 (Profile 기본)

```
src/main/  
  java/  
    com/  
      example/  
        demo/  
          DemoApplication.java          # @SpringBootApplication  
  
  resources/  
    application.yml                    # 공통  
    application-dev.yml                # 개발 프로파일  
    application-prod.yml               # 운영 프로파일
```

Configuration과 Profile 관리

실습 (Profile 기본)

application.yml

```
server:  
  port: 8080
```

application-dev.yml

```
spring:  
  config:  
    activate:  
      on-profile: dev
```

```
server:  
  port: 8081
```

application-prod.yml

```
spring:  
  config:  
    activate:  
      on-profile: prod
```

```
server:  
  port: 9001
```


Configuration과 Profile 관리

실습 (Profile 기본)

```
// DemoApplication.java
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.core.env.Environment;

@SpringBootApplication
public class DemoApplication implements CommandLineRunner{

    @Autowired
    private Environment env;

    @Value("${server.port}")
    String port;
```

```
public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
}

@Override
public void run(String... args) throws Exception {

    System.out.println("server.port: " + port);

    // 활성화 프로파일 출력
    String[] activeProfiles = env.getActiveProfiles();
    if (activeProfiles.length == 0) {
        System.out.println("active.profiles = (none)");
    } else {
        System.out.println("active.profiles = " + String.join(", ", activeProfiles));
    }
}
```

Configuration과 Profile 관리

실습 프로젝트 구조 (@Profile 어노테이션)

```
src/main/
  java/
    com/
      example/
        demo/
          DemoApplication.java          # @SpringBootApplication
          config
            DevConfig.java              # @Profile("dev") 클래스 레벨
            ProdConfig.java            # @Profile("prod") 클래스 레벨
            ProfileExampleConfig.java   # @Profile("dev"), @Profile("prod") 메서드 레벨
        resources/
          application.yml               # 공통
          application-dev.yml           # 개발 프로파일
          application-prod.yml          # 운영 프로파일
```

Configuration과 Profile 관리

실습 (@Profile 어노테이션)

```
// DevConfig.java
package com.example.demo.config;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

// 프로파일에 따라서 생성되는 Bean이 달라짐
@Configuration
@Profile("dev")
public class DevConfig {

    public DevConfig(){
        System.out.println("DevConfig");
    }
}
```

```
// ProdConfig.java
package com.example.demo.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

// 프로파일에 따라서 생성되는 Bean이 달라짐
@Configuration
@Profile("prod")
public class ProdConfig {

    public ProdConfig(){
        System.out.println("ProdConfig");
    }
}
```

Configuration과 Profile 관리

실습 (@Profile 어노테이션)

```
// ProfileExampleConfig.java
package com.example.demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
public class ProfileExampleConfig {

    // 프로파일에 따라서 생성되는 Bean이 달라짐
    @Bean
    @Profile("dev")
    public String devBean() {
        return "dev Bean";
    }

    @Bean
    @Profile("prod")
    public String prodBean() {
        return "prod Bean";
    }
}
```

Configuration과 Profile 관리

실습 (@Profile 어노테이션)

```
// DemoApplication.java
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.core.env.Environment;

@SpringBootApplication
public class DemoApplication implements CommandLineRunner{

    @Autowired
    private Environment env;

    @Value("${server.port}")
    String port;

    @Autowired
    private String bean;
```

```
public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
}

@Override
public void run(String... args) throws Exception {

    System.out.println("server.port: " + port);
    System.out.println("bean: " + bean);

    String[] activeProfiles = env.getActiveProfiles();
    if (activeProfiles.length == 0) {
        System.out.println("active.profiles = (none)");
    } else {
        System.out.println("active.profiles = " + String.join(", ", activeProfiles));
    }
}
}
```

[샘플코드] Configuration & Profile

```
# application.yml
shop:
  api:
    base-url: https://api.shop.com
    timeout: 5000

// ---- Java ----
@Component
@ConfigurationProperties(prefix = "shop.api") // 계층 설정 -> 객체 매핑
public class ShopApiProperties {
    private String baseUrl; // shop.api.base-url
    private int timeout;    // shop.api.timeout
    // getter / setter
}
// 환경별: application-dev.yml / application-prod.yml
// 실행: --spring.profiles.active=dev
```

[핵심 요약] Configuration과 Profile 관리

핵심 포인트	반드시 숙지할 내용
Configuration	환경 정보(포트·DB·Key 등)를 코드가 아닌 외부 파일(application.yml) 로 분리
PropertySource 우선순위	명령줄 인자 > JVM 옵션 > 환경변수 > yml 순으로 설정값 결정
값 주입	@Value (단일 값) vs @ConfigurationProperties (구조적 묶음·타입 안전)
Profile	dev·test·prod 환경별 설정 자동 적용 — application-{profile}.yml, @Profile
설정값 검증	@Validated + 제약 어노테이션으로 기동 시 설정 오류 차단

Springboot MVC & Actuator

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SpringBoot MVC & Actuator

Spring MVC

■ 개념

- 웹 어플리케이션의 구조적 설계 패턴 중 하나로서 역할별로 구분하는 아키텍처
- 데이터처리, 화면처리, 제어 로직을 분리하는 것이 목적

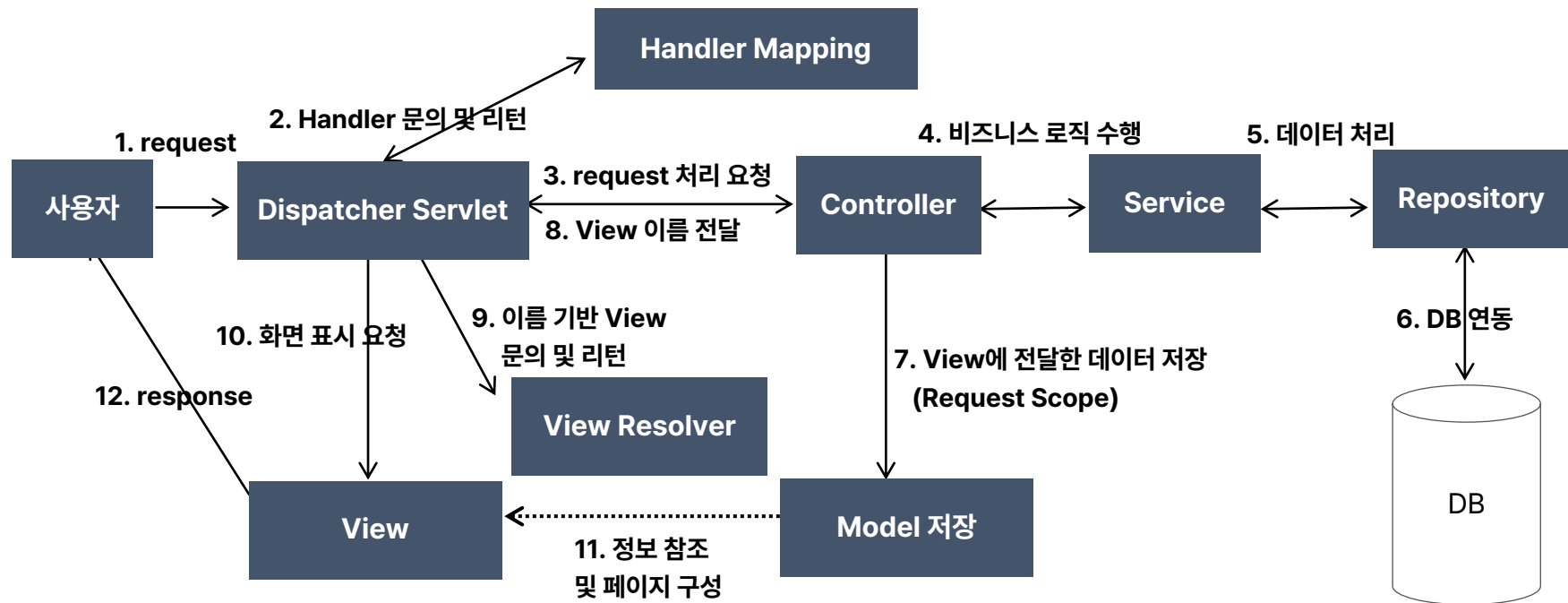
■ 구성요소별 역할

구성요소	설명	Spring 역할
Model	비즈니스 로직 및 데이터 처리	Service 및 Repository 계층과 연결 @Service, @Repository 사용
View	사용자에게 보여지는 화면 처리	Thymeleaf, JSP, XML, JSON 응답
Controller	요청(request)와 응답(response)제어	@Controller, @RestController

Spring MVC 특징

구분	설명
Annotation 기반 개발	XML 설정 없이 어노테이션(ex. @Controller, @GetMapping 등)으로 구성
Front Controller 패턴	모든 요청은 DispatcherServlet 이 단일 진입점 역할
유연한 View 처리	Thymeleaf, JSP, XML, JSON 등 다양한 View 형식 지원 및 교체 용이
POJO 기반 구조	특별한 인터페이스 상속 없이 순수 Java 객체로 개발 가능
Spring IoC 통합	MVC 구조 내에서도 IoC/DI 로 객체 간 의존성 자동 관리
Exception 처리 내장	Controller 단위 또는 전역 예외 처리 지원 ex. @ExceptionHandler, @ControllerAdvice
RESTful 아키텍처 지원	REST API 방식으로 데이터를 JSON 형태로 응답
Spring Boot 자동 설정	spring-boot-starter-web 의존성으로 MVC 환경 자동 구성

Spring MVC 요청 흐름



주요 컴포넌트

구성 요소	설명
DispatcherServlet	요청을 받아서 Controller로 전달하고 결과를 반환하는 중앙 처리 기능
HandlerMapping	요청 URL과 매핑된 Controller 탐색
Controller	실제 요청을 처리하는 핵심 비즈니스 로직 호출 @Controller 또는 @RestController 사용
Service	비즈니스 로직 및 트랜잭션 처리 @Service , @Transactional 사용
Repository	DB 연동 및 데이터 처리 @Repository 사용
Model	Controller와 View 사이의 데이터 전달 객체
ViewResolver	논리적 View 이름을 실제 View 객체로 변환
View	사용자에게 출력되는 화면 (View Template)

spring-boot-starter-web 의존성

■ 개념

- 웹 어플리케이션 개발에 필요한 모든 라이브러리를 한 번에 포함하는 Starter
- implementation 'org.springframework.boot:spring-boot-starter-web'

■ 주요 특징

특징	설명
생산성 향상	개발자가 웹 서버 설정 없이 바로 개발 가능
표준화	Spring MVC 구조를 자동 구성
단순화	Servlet 설정, DispatcherServlet 등록 자동 처리
내장 서버	Tomcat 자동 제공으로 별도 설치 불필요
확장성	REST API, 웹 MVC, JSON, 예외처리까지 모두 지원

구성 라이브러리

포함 라이브러리	주요 기능	설명
spring-boot-starter	공통 설정	로깅, 자동 설정 등 공통 구성 포함
spring-boot-starter-json	JSON 처리	Jackson 기반 JSON 직렬화/역직렬화 직렬화: Java객체 -> JSON 변환 역직렬화: JSON -> Java 객체 변환
spring-web	서블릿 기반 웹 기능	DispatcherServlet, Controller, Filter 등
spring-webmvc	MVC 아키텍처	Model, View, Controller 패턴 지원
spring-boot-starter-tomcat	내장 서버	Tomcat 내장 웹 서버 자동 실행
spring-boot-starter-logging	로깅	SLF4J + Logback 로깅 지원

DispatcherServlet

■ 개념

- 모든 요청을 중앙에서 처리하는 프론트 컨트롤러 역할
- 요청을 적절한 Controller로 위임하고 응답을 View로 전달

■ 특징

항목	설명
단일 진입점	모든 요청이 DispatcherServlet을 거침
흐름 제어	요청 -> 처리 -> 응답 전 과정을 관리
확장성	다양한 View 기술과 연동 가능
자동 등록	Spring Boot 에서는 자동으로 DispatcherServlet 등록됨

Controller

■ 개념

- 사용자의 요청을 받아 처리하는 핵심 컴포넌트
- 요청 파라미터를 받고 비즈니스 로직(Service)을 호출한 뒤 결과를 Model에 담아 반환

■ 주요 어노테이션

어노테이션	설명
@Controller, @RestController	Spring MVC의 Controller 클래스로 등록
@RequestMapping	URL 패턴과 Controller 메서드 매핑 정의
@GetMapping, @PostMapping	HTTP 메서드 별 매핑 단축 어노테이션
@RequestParam, @ModelAttribute	파라미터 매핑 및 모델 바인딩
@ResponseBody, @RequestBody	View가 아닌 데이터(ex. JSON) 직접 반환 및 객체 매핑

SpringBoot MVC & Actuator

Controller 특징

- 요청 기반 매핑 구조
 - URL, HTTP Method(GET,POST)에 따라 처리 메서드가 구분됨
ex. @GetMapping, @PostMapping
- 어노테이션 기반 설정
 - @Controller 또는 @RestController를 사용하여 Bean으로 등록
- POJO 기반
 - 일반 Java 클래스(Plain Old Java Object)로 작성 가능하며 Spring이 자동으로 관리
- 의존성 주입 지원
 - @Autowired 또는 생성자를 이용해 Service, Repository 등의 Bean을 주입 받아 사용

SpringBoot MVC & Actuator

@RequestMapping

- 개념
 - 사용자가 요청한 URL 정보를 이용하여 Controller의 실제 처리 메서드와 매핑하는 역할
- 특징
 - 가장 범용적으로 사용되는 방법
 - @Controller / @RestController의 클래스 수준과 메서드 수준에 경로를 선언
 - HTTP 메서드 특화된 어노테이션 사용 권장
 - ex. @GetMapping, @PostMapping, @PutMapping, @DeleteMapping

SpringBoot MVC & Actuator

기본 사용 예

요청 경로	설명
<code>@RequestMapping(value="/hello")</code>	/hello 경로로 들어오는 요청을 처리. value 속성은 생략가능 ex. http://localhost:8080/hello
<code>@RequestMapping(value="/main*")</code>	* 는 한 단계의 와일드 카드로서 /main으로 시작하는 모든 경로 처리 가능 ex. /main, /mainA, /mainTest
<code>@RequestMapping("/admin/**/user")</code>	** 는 여러 경로 계층 와일드 카드로서 /admin으로 시작해서 /user로 끝나는 모든 경로 처리 가능 ex. /amin/user, /admin/a/user, /admin/a/b/user
<code>@RequestMapping(value="/hello", method=RequestMethod.GET)</code>	GET 방식의 요청 <code>@GetMapping("/hello")</code> 와 동일
<code>@RequestMapping(value="/hello", method=RequestMethod.POST)</code>	POST 방식의 요청 <code>@PostMapping("/hello")</code> 와 동일

@RequestParam

- 개념

HTTP 요청의 파라미터 값을 Controller 메서드의 매개변수로 매핑
필수 옵션이기 때문에 파라미터가 누락되면 400 Bad Request 에러 발생

속성	타입	기본값	설명
value	String	파라미터 명과 동일	요청 파라미터 이름 지정 ex. @RequestParam(value="id")
required	Boolean	true	필수 여부 지정 ex. @RequestParam(required=false)
defaultValue	String	없음	파라미터가 없을 때 기본값 지정 ex. @RequestParam(required=false, defaultValue="기본값")

@RequestParam 예시

@Controller

```
public class HelloController {  
  
    // GET /hello 요청  
    @GetMapping ("/hello")  
    public String showHelloPage(Model model) {  
        model.addAttribute("message", "Spring MVC 구조 실습");  
        return "hello"; // 논리적인 View 이름 반환  
    }  
  
    // GET /user?id=abc123 요청  
    @GetMapping("/user")  
    @ResponseBody  
    public String getUser(@RequestParam("id") String userId) {  
        return "User ID: " + userId; // 본문으로 반환  
    }  
}
```

```
@GetMapping("/page")
```

@ResponseBody

```
public String page(@RequestParam(required = false) Integer size) {  
    return "페이지 크기: " + size;  
}
```

```
@GetMapping("/search")
```

@ResponseBody

```
public String search(  
    @RequestParam String keyword,  
    @RequestParam(defaultValue = "1") int page) {  
    return "검색어: " + keyword + ", 페이지: " + page;  
}
```

```
@GetMapping("/params")
```

@ResponseBody

```
public String params(@RequestParam Map<String, String> paramMap) {  
    return "요청 파라미터: " + paramMap;  
}
```

SpringBoot MVC & Actuator

@PathVariable

■ 개념

- 요청 URL 경로 자체에 포함된 값(path variable)을 Controller 메서드의 매개변수로 매핑
RESTful 한 API 설계에 자주 사용됨

속성	타입	기본값	설명
value	String	파라미터 명과 동일	요청 파라미터 이름 지정 ex. @PathVariable(value="id")
required	Boolean	true	필수 여부 지정 ex. @PathVariable (required=false)
name	String	파라미터 명과 동일	value 와 동일 (가독성 용도)

SpringBoot MVC & Actuator

@PathVariable 예시

@Controller

```
public class HelloController {
```

```
    // GET /hello 요청
```

```
    @GetMapping("/hello")
```

```
    public String showHelloPage(Model model) {
```

```
        model.addAttribute("message", "Spring MVC 구조 실습");
```

```
        return "hello"; // 논리적인 View 이름 반환
```

```
    }
```

```
    // GET /item/10 요청
```

```
    @GetMapping("/item/{id}")
```

```
    @ResponseBody
```

```
    public String getItem(@PathVariable("id") int id, Model model) {
```

```
        model.addAttribute("message", "상품번호: " + id);
```

```
        return "itemDetail";
```

```
    }
```

```
@GetMapping("/{user/{id}", "/user")
```

```
@ResponseBody
```

```
public String getUser(@PathVariable(required = false) Integer id) {
```

```
    if (id == null) {
```

```
        return "전체 사용자 목록";
```

```
    }
```

```
    return "사용자 ID: " + id;
```

```
}
```

@RequestParam vs @PathVariable

항목	@RequestParam	@PathVariable
입력방식	쿼리 파라미터 또는 폼 데이터	경로 변수
URL 형태	/product/detail?id=5	/products/5
Spring Mapping	@RequestParam("id")	@PathVariable("id")
사용 시점	선택적 파라미터 전달	필수 파라미터 전달
패턴	전통적 MVC 패턴	RESTful API

SpringBoot MVC & Actuator

Model

■ 개념

- Spring MVC 아키텍처에서 데이터를 저장하고 전달하는 객체
- Controller가 비즈니스 로직을 수행한 결과를 View로 전달할 때 중간 매개체 역할
- Model, ModelMap, ModelAndView, @ModelAttribute 등 사용

■ 특징

항목	설명
Spring이 자동 관리	요청마다 새로운 Model 객체를 생성 관리
데이터 전달	Controller 가 View에 데이터를 전달
key/value 구조	model.addAttribute(key, value) 형태
스코프 관리	request 범위 내에서 유효
단순 구조	복잡한 DTO 없이도 간단히 데이터 전달 가능

SpringBoot MVC & Actuator

Model 예시

```
@GetMapping("/info")
public String showInfo(Model model) {
    model.addAttribute("title", "Spring MVC");
    model.addAttribute("version", "3.x");
    return "info"; // View 정보
}
```

```
@GetMapping("/info")
public ModelAndView showInfo() {
    ModelAndView mav = new ModelAndView();
    mav.addObject("title", "Spring MVC");
    mav.addObject("version", "3.x");
    mav.setViewName("info"); // View 정보
    return mav;
}
```

```
@GetMapping("/login")
public String login(@ModelAttribute("login") LoginDTO dto) {
    //모델 저장
    dto.setUserid("inky4832");
    dto.setPasswd("1234");
    return "success";
}
```

Service

■ 개념

- Controller가 요청을 전달하면 Service는 필요한 데이터 처리/검증/계산 등을 수행
- 비즈니스 로직을 수행하는 계층으로서 Repository를 통해서 DB에 접근

■ 주요 역할

구분	설명
비즈니스 로직 처리	ex. 주문 시 재고 차감, 가격 계산, 유효성 검증
트랜잭션 처리	데이터 변경 시 오류가 생기면 롤백 처리 (@Transactional 사용)
Repository 호출	DB 저장, 수정, 삭제, 조회 등을 Repository에 위임
데이터 가공	DTO 변환, 계산, 검증 등을 수행
Controller와 Repository 사이의 중간 계층	역할 분리로 유지보수 용이

Service

■ 장점

- 코드 재사용
- 유지보수성
- 안정성
- 테스트 용이

■ 주요 어노테이션

어노테이션	설명
@Service	해당 클래스가 비즈니스 로직을 담당하는 Service 계층임을 명시
@Transactional	메서드 실행 중 오류가 발생하면 모든 변경사항을 되돌림

Service 예시

@Service // 서비스 계층임을 명시

```
public class ProductService {  
  
    private final ProductRepository productRepository;  
  
    // 생성자 주입  
    public ProductService(ProductRepository productRepository) {  
        this.productRepository = productRepository;  
    }  
  
    // 전체 상품 조회  
    public List<Product> getAllProducts() {  
        return productRepository.findAll();  
    }  
}
```

// 상품 등록

@Transactional

```
public void addProduct(Product product) {  
    if (product.getPrice() < 0) {  
        throw new IllegalArgumentException("가격은 0 이상이어야 합니다.");  
    }  
    productRepository.save(product);  
}
```

// 상품 삭제

@Transactional

```
public void deleteProduct(Long id) {  
    productRepository.deleteById(id);  
}  
}
```

Repository

■ 개념

- 데이터를 영구적으로 저장하고 조회하는 계층
- 어플리케이션에서 사용하는 도메인 객체(Entity, VO, DTO 등)와 실제 데이터 저장소(DB, 파일, 외부 API 등) 사이의 추상화 계층 (Abstract Layer) 역할

■ 역할

구분	설명
데이터 접근 (Data Access)	DB, 파일, 외부 API 등 다양한 저장소에 데이터를 읽고 쓰는 책임
추상화 (Abstraction)	구체적인 데이터 접근 기술(JDBC, MyBatis, JPA 등)을 숨기고 일관된 인터페이스 제공
Entity 관리	비즈니스 로직에서 사용하는 객체를 데이터 저장 구조에 맞게 변환

Repository

■ 계층 필요성

항목	설명
관심사 분리	Service는 비즈니스 로직에 집중하고 Repository는 데이터 접근에 집중
유지보수 용이성	데이터 접근 방식을 변경해도 Service 로직 변경 최소화
테스트 용이성	Mock 객체로 Repository 단위 테스트 가능
기술 교체 유연성	JPA -> MyBatis -> MongoDB 교체 시 Repository 인터페이스만 수정하면 됨

Repository

■ 주요 구성 요소

구성 요소	설명
Entity / Domain 객체	실제 저장할 데이터의 구조 정의 ex. Product, User 등
Repository Interface	데이터 접근 기능을 정의하는 추상 인터페이스
Repository 구현체	JDBC, MyBatis, JPA 등 실제 접근 기술을 사용해 구현. @Repository 이용

Repository 예시

```
// 도메인 객체
public class Product {
    private Long id;
    private String name;
    private int price;
}

// Repository 인터페이스
public interface ProductRepository {
    Product save(Product product);           // 등록/수정
    Optional<Product> findById(Long id);      // 단일 조회
    List<Product> findAll();                  // 전체 조회
    void deleteById(Long id);                 // 삭제
}
```

```
// JDBC 구현체
@Repository
public class JdbcProductRepository implements ProductRepository {

    private final JdbcTemplate jdbcTemplate;

    public JdbcProductRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @Override
    public Product save(Product product) {
        String sql = "INSERT INTO product(name, price) VALUES(?, ?)";
        jdbcTemplate.update(sql, product.getName(), product.getPrice());
        return product;
    }

    @Override
    public void deleteById(Long id) {
        jdbcTemplate.update("DELETE FROM product WHERE id = ?", id);
    }
}
```

Repository 예시

```
// MyBatis 구현체
@Mapper
public interface MyBatisProductRepository
    extends ProductRepository {

    @Override
    Product save(Product product);

    @Override
    Optional<Product> findById(Long id);

    @Override
    List<Product> findAll();

    @Override
    void deleteById(Long id);
}
```

```
// Memory 구현체
@Repository
public class MemoryProductRepository implements ProductRepository {

    private final Map<Long, Product> store = new HashMap<>();
    private final AtomicLong sequence = new AtomicLong(0);

    @Override
    public Product save(Product product) {
        long id = sequence.incrementAndGet();
        product = new Product(id, product.getName(), product.getPrice());
        store.put(id, product);
        return product;
    }

    @Override
    public List<Product> findAll() {
        return new ArrayList<>(store.values());
    }

    @Override
    public void deleteById(Long id) {
        store.remove(id);
    }
}
```

핵심기능 비교

구분	Controller	Service	Repository
주요 역할	요청/응답 처리	로직 수행	데이터 저장/조회
Spring 어노테이션	@Controller	@Service	@Repository
트랜잭션 관리	X	O	X
핵심 키워드	요청 흐름	비즈니스	데이터

SpringBoot MVC & Actuator

View

■ 개념

- Spring MVC 패턴에서 클라이언트에게 응답을 표현하는 계층
- Controller가 처리한 결과(Model 데이터)를 받아 사용자가 볼 수 있는 형태로 출력
ex. HTML, XML, JSON 등

■ 핵심 역할

구분	역할
응답 생성 (Presentation)	• Controller에서 전달된 Model 데이터를 화면(HTML) 또는 JSON데이터로 변환
표현 기술 독립성	• View 종류(JSP, Thymeleaf, JSON 등)는 자유롭게 교체 가능 • Controller는 View 기술에 독립적
Model 데이터 접근	• View는 Model 객체를 통해 데이터에 접근
표현 로직 처리	• 단순한 데이터 표시 (비즈니스 로직은 없음)

View 형태

View 유형	설명
HTML View	템플릿 엔진 (JSP, Thymeleaf, FreeMarker 등)을 이용해 화면 생성
JSON View	REST API 응답 시 객체를 JSON 형태로 반환
파일 기반 View	PDF, Excel 등의 파일 형식으로 응답 생성
Redirect View	다른 URL로 재요청을 보내는 리다이렉션 수행 ex. redirect:/path
Forward View	동일 요청 내에서 다른 컨트롤러로 제어 이동 ex. forward:/path

SpringBoot MVC & Actuator

View 이름 반환 방식

■ 개념

- Controller는 View 객체를 직접 반환하지 않음
- View의 논리적 이름(String)만 반환
- 논리적 이름은 ViewResolver가 실제 View 객체로 변환시킴
ex. ThymeleafView, JstlView, MappingJackson2JsonView

ex.

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("message", "환영합니다!");
        return "home"; // "home"은 논리적 View 이름
    }
}
```

ViewResolver

■ 개념

- Controller에서 반환된 View의 논리적 이름을 실제 View 객체로 변환

■ 핵심 유형

유형	설명
Internal Resource View Resolver	JSP 같은 서버 사이드 HTML 템플릿을 찾을 때 사용
Thymeleaf View Resolver	Thymeleaf 템플릿을 찾을 때 사용
Free Marker View Resolver	FreeMarker 템플릿을 찾을 때 사용
Content Negotiating View Resolver	요청 헤더(Content-Type)에 따라 HTML,JSON 등 View 자동 선택

SpringBoot MVC & Actuator

ViewResolver 동작 흐름

Controller 에서 View 이름 반환

```
@GetMapping("/info")
public String info(Model model) {
    model.addAttribute("title", "정보 페이지");
    return "info"; // View 이름만 반환
}
```



ViewResolver가 처리

입력	처리 결과
View 이름 "info"	실제 파일 경로 변환 JSP 경우: /WEB-INF/views/info.jsp Thymeleaf 경우: templates/info.html
Model 데이터	{ "title": "정보 페이지" }
최종 View	Model 데이터를 표현한 HTML 문서

최종 결과

클라이언트는 HTML, JSON 등 출력



ViewResolver가 내부에서 어떤 View기술을
사용했는지 완전히 숨김

핵심기능 비교

항목	View	ViewResolver
정의	사용자에게 보여줄 응답을 생성하는 객체	View 이름을 실제 View 객체로 변환
역할	Model 데이터 표시	View 선택 및 연결
Controller 관계	Controller가 View이름을 반환하면 실행됨	Controller와 View 사이 조정
표현 기술 의존	없음 (HTML, JSON, PDF 등 모두 가능)	없음 (등록된 Resolver가 선택)
Spring 내 위치	MVC 의 V 역할 (출력 담당)	MVC의 V 조정자 역할

SpringBoot MVC & Actuator

Actuator

■ 개념

- Spring Boot 어플리케이션의 운영, 모니터링, 진단을 위한 정보를 엔드포인트(Endpoint) 형태로 제공하는 기능
- /actuator/... 주소를 통해 내부 상태를 확인 가능

■ 주요 기능

기능	설명
모니터링과 제어	어플리케이션의 상태를 모니터링하고 필요시 제어 가능한 매커니즘 제공
시스템 상태보고	실시간으로 어플리케이션의 건강 상태, 매트릭, 환경정보 등을 제공
피드백 제공	시스템의 다양한 정보를 외부로 노출시켜 필요한 조치 가능하도록 지원
능동적 대응	단순한 모니터링을 넘어 로깅 레벨 변경, 상태확인과 같은 능동적인 작업 지원

Actuator

■ 의존성 설정

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    implementation 'org.springframework.boot:spring-boot-starter-actuator'  
}
```

■ 특징

항목	설명
어플리케이션 모니터링	어플리케이션의 상태, 건강 상태, 성능 지표 등을 실시간으로 모니터링
HTTP 엔드포인트	RESTful API 형태로 다양한 정보에 접근할 수 있는 엔드포인트 제공
운영 관리	로깅레벨 변경, 스레드 덤프 확인 등 운영 관련 작업을 수행 가능
보안 통합	Spring Security와 통합하여 엔드포인트에 대한 접근을 제어 가능

SpringBoot MVC & Actuator

Endpoints

■ 개념

- Actuator가 제공하는 내부 관리용 URL 주소(API)
- 어플리케이션의 상태, 환경, 설정, 로그, 매트릭 등을 외부에서 쉽게 확인하도록 지원
ex. `http://localhost:8080/actuator/health`

■ 특징

- 각 Endpoint에 대해서 액세스(access)를 제어 가능
- HTTP 또는 JMX를 통해 노출(expose)를 제어 가능
- Endpoint는 반드시 access가 허용되고 expose 되었을 때만 사용이 가능해짐
- 내장된 Endpoint는 사용 가능할 때만 자동으로 구성됨
- 대부분 HTTP를 통해서 노출을 선택하며 경로는 `/actuator/<endpoint ID>` 형식을 취함

내장 Endpoints

Endpoint ID	설명	최종 Endpoint
health	어플리케이션의 전반적인 상태 정보 ex. UP, DOWN, OUT_OF_SERVICE	/actuator/health
info	어플리케이션 정보 표시 ex. 버전, 이름 등	/actuator/info
metrics	다양한 측정 지표 제공 ex. JVM, CPU, HTTP 요청 등	/actuator/metrics
beans	컨텍스트에 등록된 Spring Bean 목록 및 관계 정보 제공	/actuator/beans
env	어플리케이션 환경 설정 정보 제공	/actuator/env
mappings	요청 매핑 정보 ex. @RequestMapping	/actuator/mappings
loggers	로거 구성 및 레벨 조회/변경 기능 제공	/actuator/loggers
configprops	@ConfigurationProperties 목록 표시	/actuator/configprops
logfile	로그 파일 내용 액세스 logging.file.name 또는 logging.file.path 속성 설정 필요	/actuator/logfile

Endpoint에 대한 expose

■ 개념

- Actuator의 Endpoint를 HTTP 요청으로 접근할 수 있게 노출시키는 설정
- 보안을 위해서 /actuator/health 만 노출되어 있고 거의 대부분 숨겨둠

■ 설정 방법

- 각각의 property는 포함하거나 제외할 endpoint 목록을 지정
- "*" 지정하면 모든 endpoint를 대상으로 함
- include와 exclude에 설정된 내용이 상충하면 exclude에 대한 내용이 우선 순위가 높음

property	설명	기본값
management.endpoint.web.exposure.include	외부에서 접근 가능한 목록 지정	health
management.endpoint.web.exposure.exclude	노출하지 않을 목록 지정	

SpringBoot MVC & Actuator

expose 예

```
management:
  endpoints:
    web:
      exposure:
        include: "*"
        exclude:
          - env
          - metrics
```

위 경우는 env와 metrics 를 제외한 모든 Endpoint가 expose 됨

SpringBoot MVC & Actuator

Endpoint에 대한 access

■ 개념

- 노출(expose)된 Actuator Endpoint에 대해 얼마나 접근을 허용할지 정하는 접근 수준
- 기본적으로 shutdown 과 heapdump를 제외한 나머지는 바로 접근이 가능
- none(차단) / read-only(읽기만) / unrestricted (제한 없음) 3가지 중에서 사용

■ 평가 순서

- 먼저 노출(expose)이 되어야 하고 그 다음 access가 적용됨
- 노출이 안되면 access 를 풀어도 외부에서 접근 불가

Endpoint에 대한 access

■ 설정 방법

property	설명
management.endpoints.access.default	기본값. 어떤 endpoint도 별도 설정이 없을 때 적용 none read-only unrestricted 중 하나 지정
management.endpoints.access.max-permitted	상한선. 어떠한 경우에도 이 수준을 넘을 수 없음
management.endpoint.<ID>.access	개별적인 endpoint 제어 방법 ex. management.endpoint.info.access=read-only

전역 차단(옵트인) 예

```
management:
  endpoints:
    access:
      default: none
  web:
    exposure:
      include: health,info,metrics
  endpoint:
    health:
      access: read-only
    info:
      access: read-only
    metrics:
      access: read-only
```

Sanitize (민감한 마스킹)

■ 개념

- Endpoint가 어플리케이션 설정 정보를 노출할 때 비밀번호/토큰 등 민감한 값을 자동으로 *** 로 대체 (masking)하는 기능
ex. /actuator/env 와 /actuator/configprops 등

■ 제어 방법

property	값
	never (***) 대체)
management.endpoint.<ID>.show-values	always (항상 실제값 표시)
	when-authorized (권한이 있으면 실제값 표시)

SpringBoot MVC & Actuator

실습 (Actuator 기본)

```
// build.gradle
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

SpringBoot MVC & Actuator

실습 (Actuator 기본)

```
# application.yml
```

```
server:
```

```
  port: 8090
```

```
spring:
```

```
  thymeleaf:
```

```
    cache: false
```

```
management:
```

```
  endpoint:
```

```
    info:
```

```
      access: none
```

```
  env:
```

```
    show-values: always
```

```
endpoints:
```

```
  web:
```

```
    exposure:
```

```
      include: "*"
```

```
      exclude:
```

```
        - beans
```

```
        - metrics
```

SpringBoot MVC & Actuator

health endpoint 심화

■ 개념

- Actuator는 /actuator/health 를 통해서 어플리케이션의 상태를 노출함
- 기본적으로 DB, 디스크 공간 등을 자동 감지함

■ 커스텀 구현

- 사용자가 직접 HealthIndicator 인터페이스를 구현하면 임의의 상태값 노출 가능

ex.

@Component

```
public class CustomServiceHealthIndicator implements HealthIndicator {
```

```
    @Override
```

```
    public Health health() {
```

```
        ..
```

```
        return Health.up().build();
```

```
    }
```

```
}
```

SpringBoot MVC & Actuator

실습 (health indicator)

```
# application.yml
```

```
server:
```

```
  port: 8090
```

```
spring:
```

```
  thymeleaf:
```

```
    cache: false
```

```
management:
```

```
  endpoints:
```

```
    web:
```

```
      exposure:
```

```
        include: health
```

```
  endpoint:
```

```
    health:
```

```
      show-details: always
```

SpringBoot MVC & Actuator

실습 (health indicator)

```
// CustomServiceHealthIndicator.java
package com.example.demo.service;

import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;

@Component
public class CustomServiceHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {

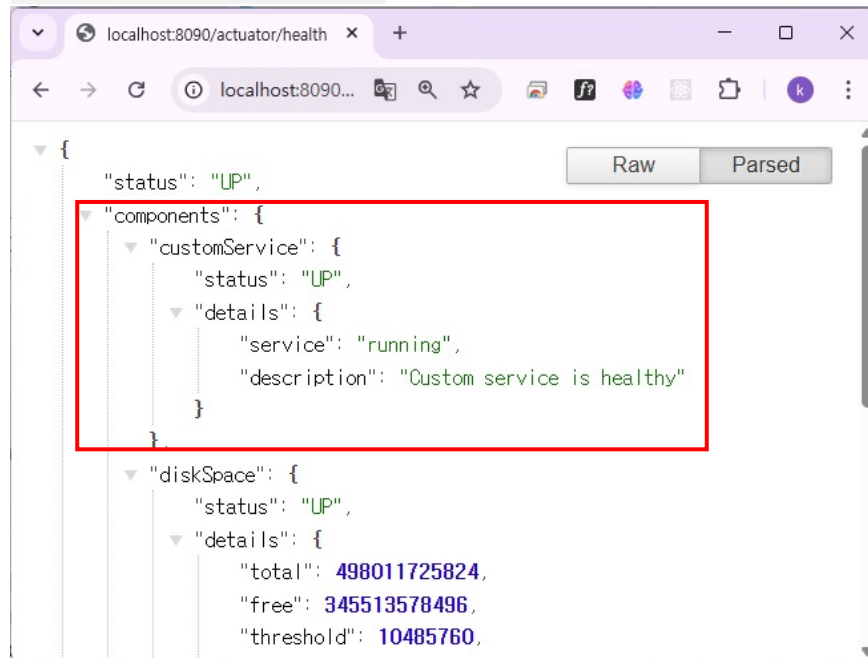
        // (1) 간단한 체크 로직: 임의로 true/false 지정 가능
        boolean serviceRunning = checkMyServiceStatus();
```

```
        if (serviceRunning) {
            return Health.up()
                .withDetail("service", "running")
                .withDetail("description", "Custom service is healthy")
                .build();
        } else {
            return Health.down()
                .withDetail("service", "stopped")
                .withDetail("description", "Custom service check failed")
                .build();
        }
    }

    private boolean checkMyServiceStatus() {
        // 실제 서비스 체크 로직 대신, 여기선 임의로 true 반환
        // ex) DB 연결, 외부 API 호출, 파일 존재 여부 등
        return true;
    }
}
```

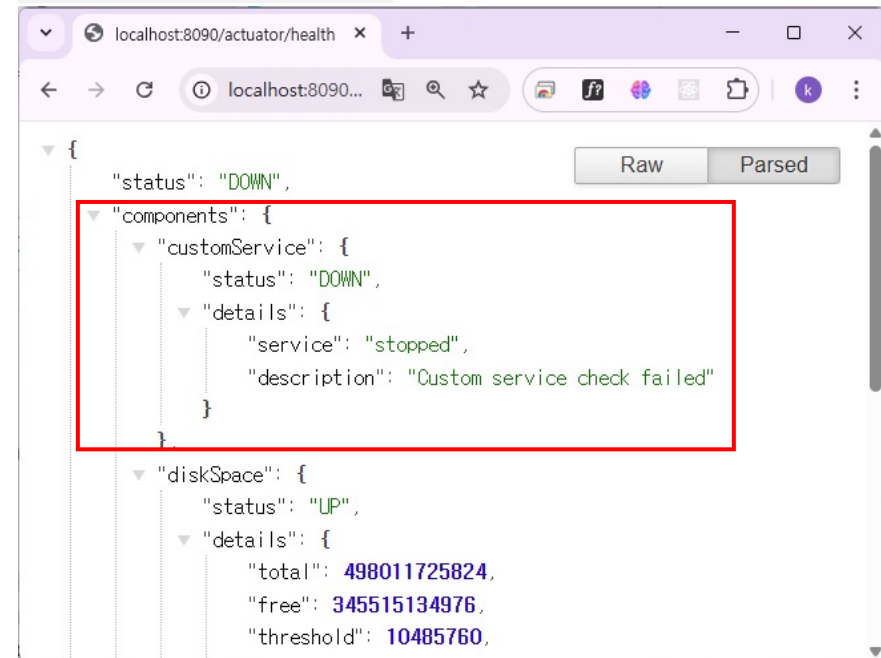

실행결과 (health indicator)

리턴값이 true 인 경우



```
{
  "status": "UP",
  "components": {
    "customService": {
      "status": "UP",
      "details": {
        "service": "running",
        "description": "Custom service is healthy"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 498011725824,
        "free": 345513578496,
        "threshold": 10485760,
      }
    }
  }
}
```

리턴값이 false 인 경우



```
{
  "status": "DOWN",
  "components": {
    "customService": {
      "status": "DOWN",
      "details": {
        "service": "stopped",
        "description": "Custom service check failed"
      }
    },
    "diskSpace": {
      "status": "UP",
      "details": {
        "total": 498011725824,
        "free": 345515134976,
        "threshold": 10485760,
      }
    }
  }
}
```

SpringBoot MVC & Actuator

info endpoint 심화

■ 개념

- Actuator는 /actuator/info 를 통해서 어플리케이션의 정보를 노출함 ex. 버전, 이름 등
- 실제 정보는 어플리케이션 컨텍스트에 정의된 모든 InfoContributor 타입의 빈에서 수집됨
- Spring Boot는 상황에 따라서 다양한 InfoContributor 빈을 자동으로 구성함

■ 커스텀 구현

- HealthIndicator 와 동일하게 직접 InfoContributor인터페이스를 구현하면 노출 가능

ex.

```
@Component
public class MyInfoContributor implements InfoContributor {
    @Override
    public void contribute(Info.Builder builder) {... }
}
```

SpringBoot MVC & Actuator

InfoContributor 종류 및 활성화

ID	Info Contributor	설명	제공정보
git	Git Info Contributor	git 저장소 정보	커밋ID, 브랜치, 시간 등
build	Build Info Contributor	빌드 정보	버전, 시간, artifact 정보 등
env	Environment Info Contributor	환경 정보	info. 으로 시작하는 속성들
java	Java Info Contributor	java 정보	java버전, JVM벤더, 런타임
process	Process Info Contributor	프로세스 정보	PID, 시작날짜, 실행 인수값
ssl	Ssl Info Contributor	SSL 인증정보	인증서 만료일, 발급자 등
ID	Info Contributor	설명	제공정보
git	Git Info Contributor	git 저장소 정보	커밋ID, 브랜치, 시간 등

info endpoint 활성화

management.info.env.enabled=true

management.info.java.enabled=true

management.info.os.enabled=true

management.info.process.enabled=true

management.info.ssl.enabled=true

SpringBoot MVC & Actuator

실습 (InfoContributor)

```
# application.yml
```

```
server:  
  port: 8090
```

```
spring:  
  thymeleaf:  
    cache: false
```

```
# EnvironmentInfoContributor 의해 수집
```

```
info:  
  app:  
    name: "주문 Service"  
    version: "1.2.4"  
    description: "주문 서비스 API"  
    author: "홍길동"
```

```
management:  
  endpoints:  
    web:  
      exposure:  
        include: health,info
```

```
info:  
  java:  
    enabled: true  
  os:  
    enabled: true  
  process:  
    enabled: true  
  env:  
    enabled: true
```

SpringBoot MVC & Actuator

실습 (InfoContributor)

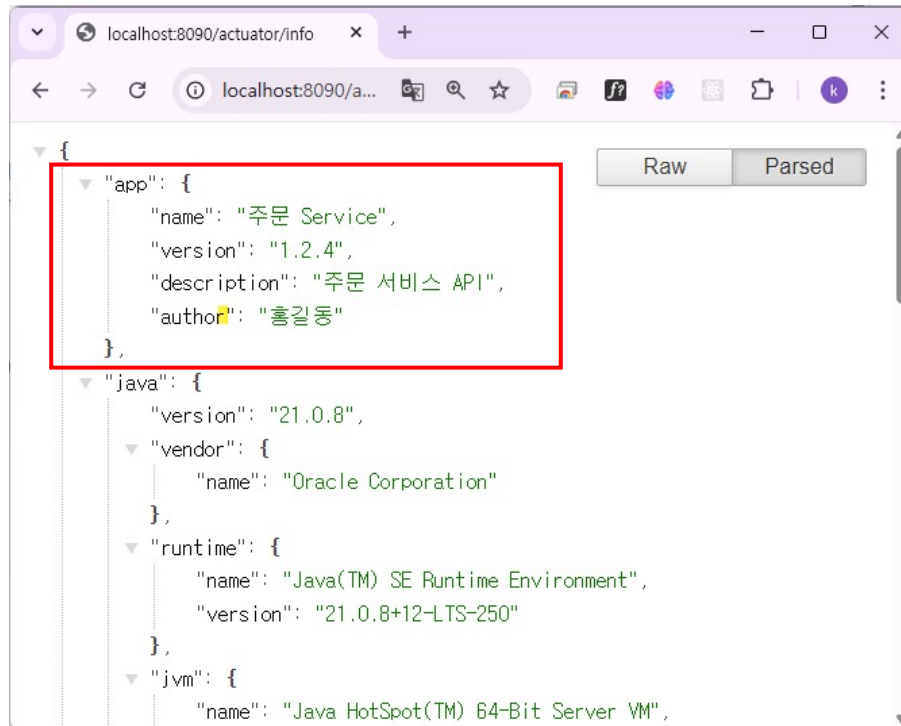
```
// CustomInfoContributor.java
package com.example.demo.contributor;

import org.springframework.boot.actuate.info.Info;
import org.springframework.boot.actuate.info.InfoContributor;
import org.springframework.stereotype.Component;

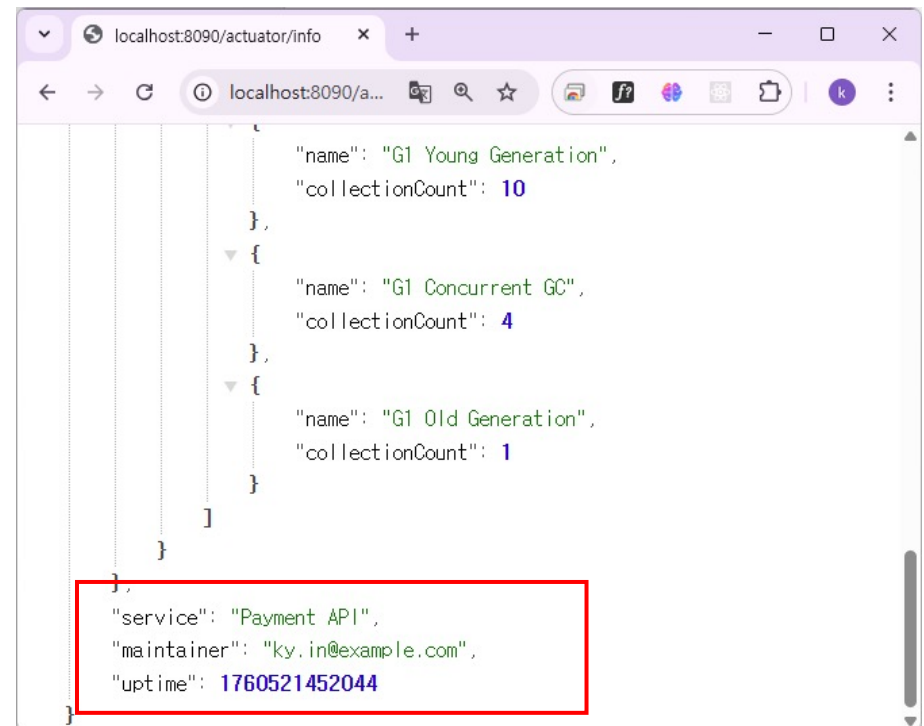
@Component
public class CustomInfoContributor implements InfoContributor {

    @Override
    public void contribute(Info.Builder builder) {
        builder.withDetail("service", "Payment API")
            .withDetail("maintainer", "ky.in@example.com")
            .withDetail("uptime", System.currentTimeMillis());
    }
}
```

실행결과 (InfoContributor)



```
{
  "app": {
    "name": "주문 Service",
    "version": "1.2.4",
    "description": "주문 서비스 API",
    "author": "홍길동"
  },
  "java": {
    "version": "21.0.8",
    "vendor": {
      "name": "Oracle Corporation"
    },
    "runtime": {
      "name": "Java(TM) SE Runtime Environment",
      "version": "21.0.8+12-LTS-250"
    },
    "jvm": {
      "name": "Java HotSpot(TM) 64-Bit Server VM",
```



```

    "name": "G1 Young Generation",
    "collectionCount": 10
  },
  {
    "name": "G1 Concurrent GC",
    "collectionCount": 4
  },
  {
    "name": "G1 Old Generation",
    "collectionCount": 1
  }
]
},
{
  "service": "Payment API",
  "maintainer": "ky.in@example.com",
  "uptime": 1760521452044
}
```

metrics endpoint 심화

■ 개념

- Actuator는 /actuator/metrics 를 통해서 다양한 측정 지표 제공
ex. JVM, CPU, HTTP 요청 등

■ 용어

구분	설명
Metrics	어플리케이션의 상태나 성능을 수치로 측정한 데이터
Meter	metrics를 수집하는 측정 도구

SpringBoot MVC & Actuator

metrics endpoint 심화

- 커스텀 구현

HealthIndicator 와 동일하게 직접 MeterBinder 인터페이스를 구현하여 노출 가능

ex.

```
@Component
```

```
public class SimpleCustomMetrics implements MeterBinder {
```

```
    @Override
```

```
    public void bindTo(MeterRegistry registry) {
```

```
        ...
```

```
    }
```

```
}
```


주요 Metrics 타입

분류	대표 매트릭 이름	설명
JVM	jvm.memory.used	사용된 메모리 크기
	jvm.threads.live	활성 스레드 수
HTTP	http.server.requests	요청 처리 시간
DataSource	hikaricp.connections.active	활성 DB 연결 수
	hikaricp.connections.timeout	DB 연결 타임아웃
Cache	cache.gets	캐시 조회 횟수
	cache.puts	캐시 저장 횟수
System	system.cpu.usage	시스템 CPU 사용률
	process.start.time	프로세스 시작 시간
Tomcat	tomcat.sessions.active.current	현재 활성 세션 수
	tomcat.sessions.expired	만료된 세션 수
Logback	logback.events	로그 이벤트 수

주요 Meter 타입

타입	설명	예시
Counter	단조 증가하는 누적 매트릭	HTTP 요청수, 에러발생 횟수
Gauge	현재 값을 나타내는 메트릭	메모리 사용량, 큐 사이즈
Timer	수행 시간 측정 매트릭	요청 처리 시간, 메서드 실행시간
Distribution Summary	데이터 분포 요약 매트릭	요청 크기, 응답 크기
Long Task Timer	장시간 작업 시간 측정	배치 작업 및 긴 실행 프로세스

SpringBoot MVC & Actuator

실습 (MeterBinder)

```
# application.yml
```

```
server:
```

```
  port: 8090
```

```
spring:
```

```
  thymeleaf:
```

```
    cache: false
```

```
management:
```

```
  endpoints:
```

```
    web:
```

```
      exposure:
```

```
        include: health,info, metrics
```

SpringBoot MVC & Actuator

실습 (MeterBinder)

```
// SimpleCustomMetrics.java
package com.example.demo.meter;

import io.micrometer.core.instrument.Gauge;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.binder.MeterBinder;
import org.springframework.stereotype.Component;

@Component
public class SimpleCustomMetrics implements MeterBinder {

    private int sampleValue = 10; // 임의의 값 (현재 상태를 의미)

    @Override
    public void bindTo(MeterRegistry registry) {
        Gauge.builder("custom.sample.value", this, SimpleCustomMetrics::getSampleValue)
            .description("샘플 상태값 (예: 현재 처리중인 작업 수)")
            .tag("service", "payment")
            .register(registry);
    }
}
```

```
// 단순 getter
public int getSampleValue() {
    return sampleValue;
}

// 값 변경 메서드 (테스트용)
public void increase() {
    sampleValue++;
}

// 값 감소
public void decrease() {
    if (sampleValue > 0) sampleValue--;
}
}
```

SpringBoot MVC & Actuator

실습 (MeterBinder)

```
//MetricTestController.java
package com.example.demo.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import com.example.demo.meter.SimpleCustomMetrics;

@Controller
public class MetricTestController {

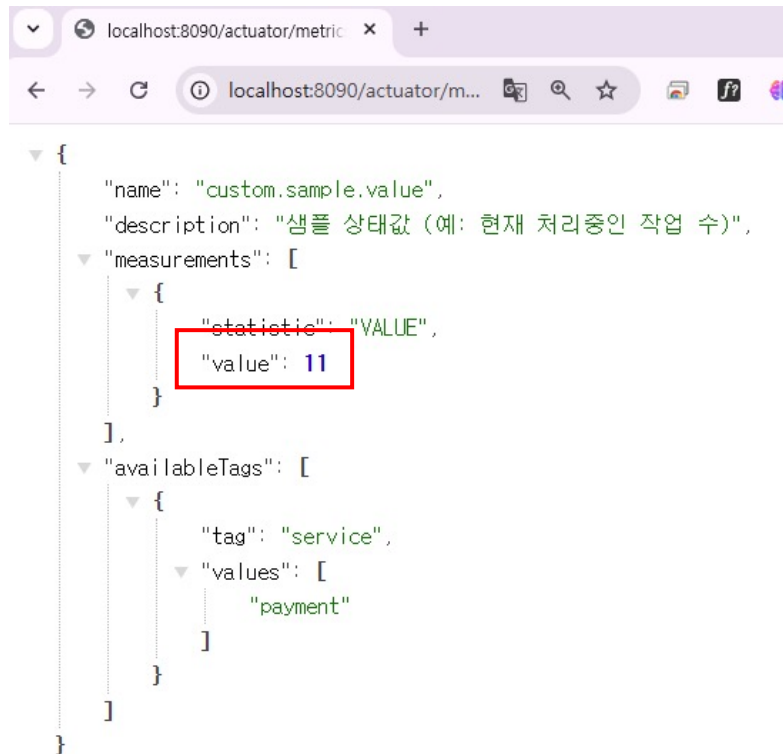
    private SimpleCustomMetrics metrics;

    public MetricTestController(SimpleCustomMetrics metrics){
        this.metrics = metrics;
    }
}
```

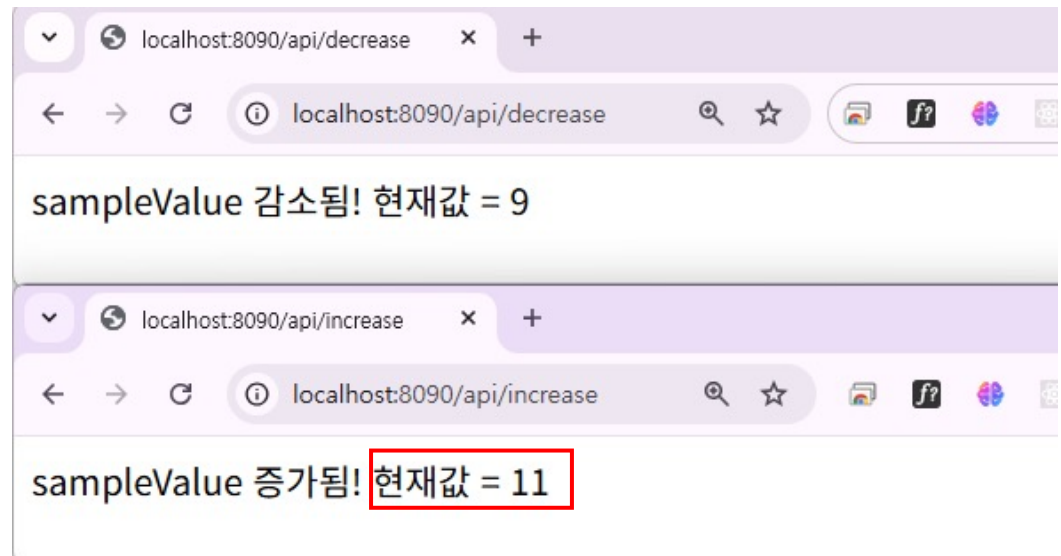
```
@GetMapping("/api/increase")
@ResponseBody
public String increaseValue() {
    metrics.increase();
    return "sampleValue 증가됨! 현재값 = "
        + metrics.getSampleValue();
}

@GetMapping("/api/decrease")
@ResponseBody
public String decreaseValue() {
    metrics.decrease();
    return "sampleValue 감소됨! 현재값 = "
        + metrics.getSampleValue();
}
}
```

실행결과 (MeterBinder)



```
{
  "name": "custom.sample.value",
  "description": "샘플 상태값 (예: 현재 처리중인 작업 수)",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 11
    }
  ],
  "availableTags": [
    {
      "tag": "service",
      "values": [
        "payment"
      ]
    }
  ]
}
```



localhost:8090/api/decrease

sampleValue 감소됨! 현재값 = 9

localhost:8090/api/increase

sampleValue 증가됨! 현재값 = 11

[샘플코드] Spring Boot MVC

```
@RestController
```

```
@RequestMapping("/api/products")
```

```
public class ProductController {
```

```
    private final ProductService service;
```

```
    public ProductController(ProductService service) {
```

```
        this.service = service;
```

```
    }
```

```
    @GetMapping("/{id}")                // GET /api/products/1
```

```
    public Product get(@PathVariable Long id) {
```

```
        return service.findById(id);
```

```
    }
```

```
}
```

```
# application.yml - Actuator (운영/모니터링)
```

```
management:
```

```
    endpoints:
```

```
        web.exposure.include: health,info,metrics
```

[핵심 요약] Spring Boot MVC & Actuator

핵심 포인트	반드시 숙지할 내용
MVC 아키텍처	Controller-Service-Repository-View 역할 분리 설계
요청 흐름	DispatcherServlet (Front Controller)이 요청을 받아 컨트롤러로 라우팅
자동 구성	spring-boot-starter-web 추가 시 MVC 자동 구성
계층 책임	Controller(요청/응답) · Service(비즈니스 로직) · Repository(데이터) · ViewResolver(화면)
Actuator	운영·모니터링 엔드포인트 health·info·metrics 제공, expose/access로 노출 제어

컴포넌트 스캔

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

컴포넌트 스캔(Component Scan)

- 컴포넌트 스캔은 스프링이 지정된 패키지(package) 이하를 탐색하여 특정 어노테이션(@Annotation)이 붙은 클래스들을 찾아내고, 스프링 빈(Bean)으로 등록하는 자동 설정 기능
- 컴포넌트 스캔의 기본 범위는 @SpringBootApplication이 선언된 클래스의 하위 패키지 전체

@SpringBootApplication 어노테이션의 조합 요소

구분	역할 / 기능
@SpringBootConfiguration	스프링부트의 설정 파일임을 명시 (내부적으로 @Configuration과 동일)
@EnableAutoConfiguration	자동 설정(Auto Configuration) 기능 활성화
@ComponentScan	해당 패키지 및 하위 패키지의 @Component, @Service, @Repository 등의 Bean 자동 탐색

❖ 스캔 범위 제한 : scanBasePackages 사용 (@ComponentScan에서는 basePackages)

```
@SpringBootApplication(scanBasePackages = {"com.skala.stock", "com.sk.common"})
public class StockApiApplication { ... }
```

컴포넌트 스캔 대상

- 스프링 부트의 스테레오타입(Stereotype) 어노테이션이 붙은 클래스들

어노테이션	설명	용도/예시
@Component	범용 컴포넌트 지정, 스프링이 빈으로 자동 등록	공통 모듈, 유틸리티 클래스 등
@Controller	MVC 패턴의 컨트롤러로 지정, 웹 요청 처리	REST API, 웹 페이지 처리용 컨트롤러 클래스
@RestController	@Controller + @ResponseBody, REST API 개발에 특화	RESTful 웹 서비스 컨트롤러
@Service	비즈니스 로직 담당 클래스 지정	서비스 계층 클래스(비즈니스 로직)
@Repository	데이터 액세스 계층(DAO) 지정, 예외 변환 기능 제공	JPA, MyBatis 등 DB 연동 클래스
@Configuration	자바 기반 설정 클래스 지정, @Bean 메서드 사용 가능	스프링 설정 및 빈 등록용 클래스
@ConfigurationProperties	외부 설정값(프로퍼티 등)을 객체에 바인딩	YAML, properties 파일을 자바 객체에 매핑
@EnableAutoConfiguration	스프링 부트 자동 설정 활성화 (주로 @SpringBootApplication에 포함)	스프링부트의 자동 설정 활성화

[실습] controller/HelloController.java

- HelloController에 /hello 엔트리 포인트를 통해 GET 메소드로 JSON 데이터를 전달
 - JSON 데이터는 message라는 key에 "SKALA에 오신 것을 환영합니다." value를 넣어서 전달
-
- "Run And Debug" 실행
 - 다른 터미널에서 curl 호출
 - Postman을 통해 API 호출

```
curl http://localhost:8080/hello  
  
{  
  "message": "SKALA에 오신 것을 환영합니다."  
}
```

[코드] controller/HelloController.java

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public Map<String, String> hello() {
        Map<String, String> response = new HashMap<>();
        response.put("message", "SKALA에 오신 것을 환영합니다.");
        return response;
    }
}
```

▪ @RestController

- REST API용 컨트롤러임을 의미하며 내부적으로 @Controller + @ResponseBody 역할을 조합
- 메서드가 반환하는 값을 HTTP 응답의 Body(본문)에 바로 담아서 JSON, XML 등으로 변환

▪ @GetMapping("/hello")

- HTTP GET 방식으로 /hello 경로로 요청이 들어왔을 때 이 메서드를 실행하도록 매핑

[실습] data/HelloResponse.java

- 문자열 message 속성을 갖는 HelloResponse DTO 클래스 작성
- 롬복(Lombok) @Data 어노테이션 설정으로 getter와 setter 구현
- HelloController의 hello에서 Map을 사용하지 않고 HelloResponse를 통해 응답 처리

❖ DTO(Data Transfer Object) : 데이터 전송을 위해 사용되는 객체

- 데이터를 담은 필드와 해당 필드에 접근할 수 있는 메서드를 정의
- 반복적인 Getter, Setter, 생성자 코드를 Lombok 어노테이션으로 대체하여 코드를 매우 간결하게 유지

[코드] data/HelloResponse.java

```
package com.skala.basic.data;

import lombok.Data;

@Data
public class HelloResponse {
    String message;
}
```

▪ @Data

- 롬복에서 제공하는 어노테이션
- 내부적으로 @Getter, @Setter, @ToString, @EqualsAndHashCode, @RequiredArgsConstructor 등을 포함

[참고] DTO 작성 시 주의점

- **Don't Include Business Logic (비즈니스 로직을 포함하지 마시오)**
 - DTO는 순수하게 데이터를 전달하는 "데이터 상자" 역할만 해야 함.
 - 계산, 로직 처리, 데이터베이스 접근 등의 코드는 서비스(Service)나 도메인(Domain) 계층에 위치
 - 나쁜 예: DTO 내부에 할인율을 계산하는 메서드를 만드는 것.
- **Don't Use Entities as DTOs (JPA 엔티티를 DTO로 사용하지 마시오) – 개인적으로 예제 작성시 무시하는 경향 (귀차니즘)**
 - 엔티티는 데이터베이스 테이블과 직접 매핑되는 핵심 도메인 객체
 - 엔티티를 클라이언트에 직접 노출하면 원치 않는 필드(비밀번호 등)가 외부에 노출될 수 있음
 - UI 요구사항이 변경될 때마다 데이터베이스 설계에 영향을 주게 되어 유연성이 떨어짐.
 - 양방향 연관관계가 설정된 엔티티를 직렬화할 때 무한 루프에 빠져 StackOverflowError가 발생할 수 있음
- **Name and Separate Appropriately (목적에 맞는 네이밍과 분리) – 귀차니즘**
 - DTO의 역할을 이름에 명시하면 가독성이 높아짐.
 - 요청(Request)용 DTO: UserCreateRequest, ProductUpdateRequest
 - 응답(Response)용 DTO: UserResponse, OrderDetailResponse
 - 하나의 DTO를 요청과 응답에 모두 사용하려고 하면, 용도가 다른 필드가 섞여 혼란을 줄 수 있음.
- **Consider Serialization (직렬화를 고려하시오)**
 - DTO는 네트워크를 통해 JSON 등의 형태로 변환되어 전송되는 경우가 많음
 - Jackson과 같은 라이브러리가 객체를 JSON으로 변환할 때 기본 생성자나 Getter가 없으면 오류 발생 가능 → Lombok으로 해결
- **Validation annotation belongs in the DTO (입력값 검증 어노테이션은 DTO에서)**
 - 클라이언트로부터 들어오는 데이터에 대한 검증은 DTO의 필드에 @NotNull, @Size, @Email 등의 어노테이션을 붙여 처리
 - 유효하지 않은 데이터가 서비스 계층으로 넘어가지 않도록 사전 차단

[참고] 데이터 getter와 setter 처리

```
public class HelloResponse {
    private String message;

    public HelloResponse() {
    }

    public HelloResponse(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    @Override
    public String toString() {
        return "HelloResponse{" +
            "message='" + message + '\'' +
            '}';
    }
}
```

[코드] controller/HelloController.java 수정

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public HelloResponse hello() {
        HelloResponse response = new HelloResponse();
        response.setMessage("SKALA에 오신 것을 환영합니다.");
        return response;
    }
}
```

[샘플코드] 컴포넌트 스캔

```
@RestController                // = @Controller + @ResponseBody
public class HelloController {
    @GetMapping("/hello")
    public HelloResponse hello() {
        return new HelloResponse("안녕하세요");
    }
}

@Service    public class HelloService { }           // 서비스 계층 빈
@Repository public interface HelloRepository { }     // 데이터 계층 빈
@Component  public class HelloUtil { }              // 일반 빈

// @SpringBootApplication 의 @ComponentScan 이
// 지정 패키지의 위 빈들을 자동으로 스캔/등록
```

[핵심 요약] 컴포넌트 스캔

핵심 포인트	반드시 숙지할 내용
@ComponentScan	지정 패키지를 탐색해 스테레오타입 어노테이션 클래스를 자동 빈 등록
스테레오타입	@Component · @Controller · @Service · @Repository
@RestController	@Controller + @ResponseBody → 반환값을 JSON 으로 응답
DTO	계층 간 데이터 전달 전용 객체 — 비즈니스 로직을 포함하지 않음

Spring 컨테이너

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Spring 컨테이너

- 스프링 빈의 생성, 설정, 관리, 소멸 등 생명주기(Lifecycle) 전반을 담당하는 핵심 엔진
- 객체의 생성과 관리, 흐름의 제어권을 개발자가 아닌 프레임워크(스프링 컨테이너)에서 처리(IOC 컨테이너)

구분	BeanFactory	ApplicationContext
설명	가장 기본적인 컨테이너	BeanFactory 확장판 (주로 사용)
주요 기능	Bean의 생성/조회/관리	BeanFactory의 모든 기능 + 부가기능 지원
Lazy Loading	지원 (요청 시 Bean 생성)	기본적으로 모든 싱글톤 빈을 컨테이너 시작 시 생성
실제 사용 예시	거의 사용하지 않음	대부분의 스프링 애플리케이션에서 사용
사용 예	옛날 레거시/경량 환경	웹/서버/실제 프로젝트 환경에서 표준적으로 사용

❖ **부가기능** : 국제화, 이벤트, AOP, 메시지 자원, 환경설정 등

빈(Bean)

■ 빈(Bean)이란?

- 스프링이 관리하는 객체(인스턴스)를 의미
- 스프링 컨테이너가 생성하고, 생명주기를 관리
- 일반적으로 개발자가 new로 직접 생성하지 않고, 스프링 컨테이너에게 위임함.

■ Bean의 특징

- **싱글톤(Singleton)** 방식으로 관리 (기본)
- 객체 생성, 의존성 주입, 소멸 등 생명주기를 컨테이너가 책임짐

싱글톤(Singleton) 방식

- 싱글톤(Singleton) 패턴이란 프로그램 실행 내내 단 하나의 객체만 생성해서 공유하는 디자인 패턴
- Spring 컨테이너는 기본적으로 모든 Bean을 싱글톤으로 관리
- 싱글톤 특징
 - 메모리 절약: 객체를 한 번만 생성해서 여러 곳에서 같이 쓰면 메모리를 아낄 수 있음.
 - 일관된 데이터 관리: 모든 사용자가 같은 객체(=동일 인스턴스)를 사용하므로 데이터 일관성이 높아짐.
 - 객체 생성 비용 감소: 반복적인 객체 생성 작업을 줄일 수 있음.

❖ 스프링에서의 싱글톤 Bean

- @Component, @Service, @Repository, @Controller 형태로 어노테이션으로 등록된 Bean을 컨테이너당 한 개만 생성 관리.

어노테이션(Annotation)

■ Java의 어노테이션(Annotation)

- 자바 코드에 '메타데이터'를 제공하는 특수한 표기법
- @ 기호로 시작 (예: @Override, @Component, @Autowired)
- 컴파일러, 개발 툴, 런타임 등에서 특별한 의미로 해석됨
- 주로 설정 파일을 줄이고 코드 가독성을 높이기 위해 사용

■ 스프링에서 어노테이션의 역할

- 설정 및 의존성 주입을 간편하게 처리하고 객체(Bean)를 자동으로 등록하거나 관리
- 다양한 기능(트랜잭션, 보안, 예외처리 등)을 선언적으로 적용하는 방식으로 사용

[참고] 스프링의 주요 어노테이션

어노테이션	용도/설명
@Component	스프링이 자동으로 Bean으로 등록해줌
@Service	비즈니스 로직을 담은 Service 클래스에 사용
@Repository	데이터 접근 계층(DAO)에 사용, 데이터 예외 변환 처리
@Controller	MVC 패턴에서 Controller 역할, 웹 요청 처리
@RestController	RESTful API Controller (JSON 반환)
@Autowired	필요한 의존성을 자동 주입 (DI)
@Bean	메서드의 반환 객체를 직접 Bean으로 등록

[실습] service/LifecycleBean.java

- @Component 어노테이션을 설정한 LifecycleBean 클래스 작성
- 생성자에 "[LifecycleBean] 생성자 호출됨 (Bean 생성) " 출력
- @PostConstruct를 붙인 init()에서 "[LifecycleBean] @PostConstruct 호출됨 (초기화)" 출력
- @PreDestroy를 붙인 destroy()에서 "[LifecycleBean] @PreDestroy 호출됨 (소멸 직전)" 출력

- "Run And Debug" 실행
- 다른 터미널에서 스프링 이력 확인

```
[LifecycleBean] 생성자 호출됨 (Bean 생성)
[LifecycleBean] @PostConstruct 호출됨 (초기화)
```

[코드] service/LifecycleBean.java

```
@Component
public class LifecycleBean {

    public LifecycleBean() {
        System.out.println("[LifecycleBean] 생성자 호출됨 (Bean 생성)");
    }

    @PostConstruct
    public void init() {
        System.out.println("[LifecycleBean] @PostConstruct 호출됨 (초기화)");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("[LifecycleBean] @PreDestroy 호출됨 (소멸 직전)");
    }
}
```

- 생성자: Bean이 생성될 때 호출됨
- @PostConstruct: Bean 생성 후 초기화 시점에 호출됨
- @PreDestroy: Bean이 소멸되기 직전(애플리케이션 종료시) 호출됨

[샘플코드] Spring 컨테이너 (Bean 생명주기)

@Component

```
public class LifecycleBean {  
  
    @PostConstruct                                // 빈 생성 직후 1회 호출  
    public void init() {  
        System.out.println("init - 자원 초기화");  
    }  
  
    @PreDestroy                                    // 빈 소멸 직전 호출  
    public void close() {  
        System.out.println("close - 자원 반납");  
    }  
}  
// 기본 스코프 = 싱글톤 (컨테이너당 1개 인스턴스를 공유)
```

[핵심 요약] Spring 컨테이너

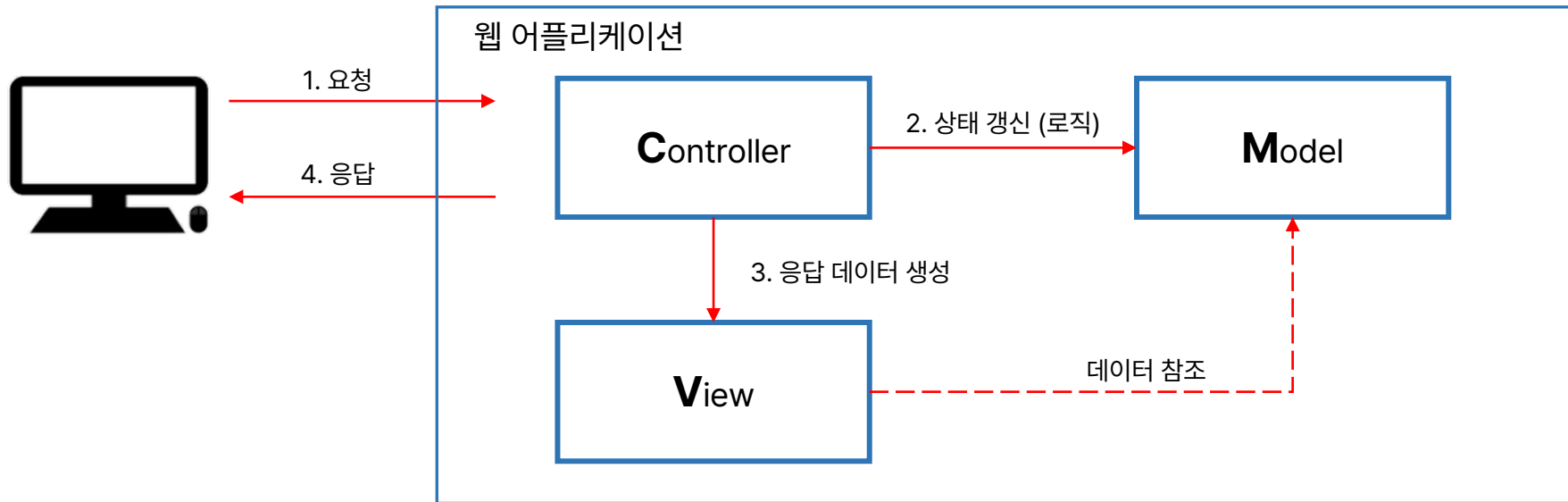
핵심 포인트	반드시 숙지할 내용
빈(Bean)	컨테이너가 생성·관리 하는 객체
싱글톤 스코프	기본 스코프 — 하나의 인스턴스를 애플리케이션 전체가 공유
어노테이션 등록	@Component 계열로 빈 등록, @Autowired로 주입
생명주기 콜백	@PostConstruct / @PreDestroy 로 초기화·소멸 시점 처리

한번 더 익히는 Spring MVC

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

스프링 MVC (Model-View-Controller)

- 스프링 MVC는 Spring Framework에서 제공하는 웹 애플리케이션 개발을 위한 핵심 모듈
- MVC 패턴을 기반으로 웹 요청을 처리하고, 데이터를 전달하며, 화면(View)을 렌더링
- ❖ 스프링 부트 애플리케이션은 내부적으로 스프링 MVC를 사용해서 웹 요청을 처리



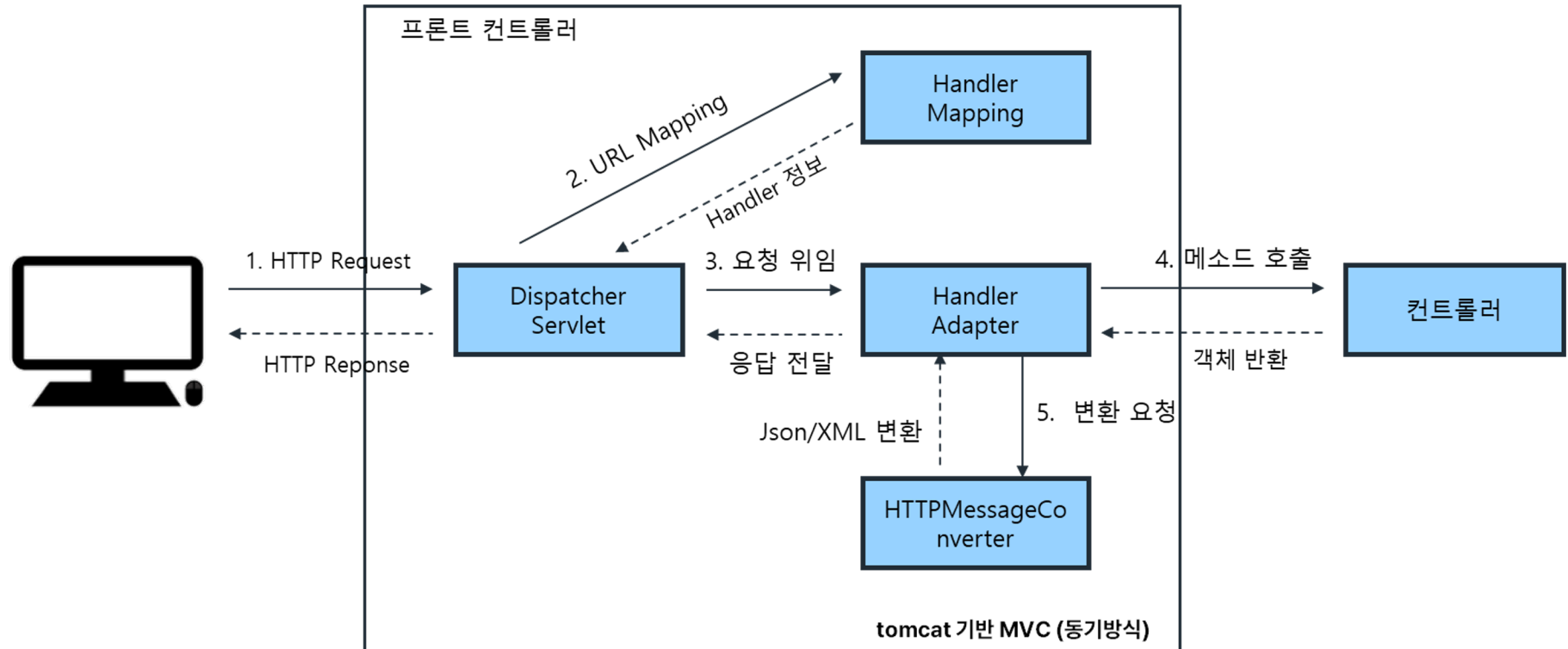
Spring Boot와 Spring MVC의 관계

구분	Spring MVC	Spring Boot
정의	Spring Framework의 웹 계층 모듈. MVC 패턴으로 HTTP 요청을 처리하는 웹 프레임워크	Spring · Spring MVC를 자동 설정 과 내장 서버로 손쉽게 구동하는 실행 플랫폼
계층 관계	Spring Boot가 포함·구동하는 하위 웹 기술	내부적으로 Spring MVC를 그대로 사용 (Boot ⊃ MVC)
설정 방식	DispatcherServlet · ViewResolver 등 수동 설정 필요	spring-boot-starter-web 추가 시 MVC 자동 구성(Auto-Configuration)
서버 구동	외부 WAS(Tomcat)에 WAR로 배포	내장 Tomcat으로 단독 실행 (java -jar)
핵심 관계	"웹 요청을 처리하는 방법"을 제공	"그 Spring MVC를 자동 설정으로 감싸 실행 "

스프링 부트 REST API 흐름도

스프링 MVC와 스프링 부트 REST API 처리의 차이

- 스프링 MVC View 반환: @Controller + return "뷰이름"; → 타임리프, JSP 등 ViewResolver가 동작.
- REST API (JSON 반환): @RestController(@Controller + @ResponseBody) → HTTPMessageConverter가 동작하여 JSON 변환.



MVC 아키텍처 흐름과 어노테이션

컴포넌트	역할 설명	관련 어노테이션
DispatcherServlet (요청)	모든 HTTP 요청을 처리하는 Front Controller	- @SpringBootApplication - DispatcherServlet
HandlerMapping	들어온 요청 URL, HTTP 메소드 등으로 Handler를 찾는 역할	- @RequestMapping - @GetMapping, @PostMapping 등
HandlerAdapter	실제 컨트롤러를 실행할 수 있도록 적합한 어댑터 선택 및 호출	- @Controller - @RestController
Controller	비즈니스 로직 처리, 요청 파라미터 바인딩, 응답 데이터 생성	- @Controller (뷰 반환) - @RestController (JSON 반환) - @ResponseBody - @RequestBody
HTTPMessageConverter	요청/응답의 Body(JSON, XML 등)를 객체/문자열로 변환	- @RequestBody, @ResponseBody - Jackson, Gson 등
DispatcherServlet (응답)	컨트롤러 결과를 받아 적절히 가공하여 클라이언트에 최종 응답	

스프링 어노테이션

역할	설명
컴포넌트 식별	클래스가 스프링이 관리할 Bean임을 표시
의존성 주입	Bean 간의 의존성을 자동으로 연결
웹 계층	컨트롤러, 요청 매핑 등 웹 요청 처리에 사용
설정 및 환경	설정 클래스 지정, 환경설정, 프로퍼티 주입 등에 사용
데이터 접근 / 트랜잭션	데이터베이스 접근 계층에서 활용, 트랜잭션 관리
예외 및 검증	예외 처리 및 데이터 검증 등

스프링 어노테이션 – 컴포넌트 식별 / 의존성 주입

분류	어노테이션	사용 위치	주요 역할/설명
컴포넌트 식별	@Component	클래스	스프링이 관리하는 일반 빈(Bean)으로 등록
	@Controller	클래스	MVC 패턴의 컨트롤러(웹 요청 처리) 빈으로 등록
	@Service	클래스	비즈니스 로직을 담는 서비스 계층 빈으로 등록
	@Repository	클래스	DAO 계층 빈, 예외 변환 처리(데이터 접근 계층)
	@RestController	클래스	REST API용 컨트롤러, 자동으로 @ResponseBody 포함
의존성 주입	@Autowired	생성자, 필드, 메소드	자동으로 의존성 주입
	@Qualifier	파라미터, 필드	동일 타입 빈 여러 개 존재 시 특정 빈을 지정
	@Inject	생성자, 필드	자바 표준 의존성 주입(스프링도 지원)
	@Resource	필드, 메소드	이름 기반 의존성 주입

스프링 어노테이션 – 환경설정 / 웹 계층

분류	어노테이션	사용 위치	주요 역할/설명
환경 설정	@Configuration	클래스	자바 기반 설정 클래스 표시
	@Bean	메소드	수동 빈 등록
	@Value	필드, 파라미터	프로퍼티 값 주입
	@PropertySource	클래스	프로퍼티 파일 위치 지정
	@Profile	클래스, 메소드	특정 프로파일(환경)에서만 빈 등록
웹 계층	@RequestMapping	클래스, 메소드	HTTP 요청 URL 매핑
	@GetMapping	메소드	GET 요청 매핑 (축약형)
	@PostMapping	메소드	POST 요청 매핑 (축약형)
	@PutMapping	메소드	PUT 요청 매핑 (축약형)
	@DeleteMapping	메소드	DELETE 요청 매핑 (축약형)
	@PathVariable	파라미터	URL 경로 변수 값 매핑
	@RequestParam	파라미터	쿼리 파라미터 값 매핑
	@RequestBody	파라미터	HTTP 요청 body(JSON 등) 매핑
	@ResponseBody	메소드, 클래스	리턴값을 HTTP 응답 body로 변환

스프링 어노테이션 – 데이터 접근 / 예외 검증

분류	어노테이션	사용 위치	주요 역할/설명
데이터 접근	@Entity	클래스	JPA 엔티티로 지정
	@Id	필드	엔티티의 기본키 지정
	@GeneratedValue	필드	기본키 값 자동 생성
	@Repository	클래스	(위 참조, 데이터 접근 계층)
	@Transactional	클래스, 메소드	트랜잭션 적용
예외 및 검증	@ControllerAdvice	클래스	전역 예외 처리, 바인딩 등 지원
	@ExceptionHandler	메소드	특정 예외 처리 메소드 지정
	@Valid, @Validated	파라미터, 클래스	Bean Validation 적용

[실습] service/HelloService.java

- @Service 어노테이션을 설정한 HelloService 클래스 작성
- 입력 받은 이름으로 HelloResponse 데이터를 생성하여 전달하는 createMessage() 메서드 구현
- HelloController는 HelloService를 생성자 주입을 통해 등록
- “/hello” 엔트리 포인트에 GET 매핑으로 name을 @RequestParam으로 전달 받도록 구현하고 HelloService에 전달하여 응답을 리턴

```
curl "http://localhost:8080/hello?name=홍길동"
```

```
{  
  "message": "안녕하세요, 홍길동님!"  
}
```


[코드] service/HelloService.java

```
import org.springframework.stereotype.Service;

@Service
public class HelloService {

    public HelloResponse createMessage(String name) {
        HelloResponse response = new HelloResponse();
        response.setMessage("안녕하세요, " + name + "님!");
        return response;
    }
}
```

- @Service : 비즈니스 로직(서비스 계층) 빈 등록

[코드] controller/HelloController.java

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private final HelloService helloService;

    // 생성자 주입
    public HelloController(HelloService helloService) {
        this.helloService = helloService;
    }

    @GetMapping("/hello")
    public HelloResponse hello(@RequestParam(defaultValue = "SKALA") String name) {
        // 서비스 호출하여 응답 객체 생성
        return helloService.createMessage(name);
    }
}
```

- **생성자 주입** : 스프링이 자동으로 HelloService를 **주입(IOC)하고 의존성(DI)을 관리**
- **@RequestParam(defaultValue = "SKALA") String name** : 쿼리 파라미터 값 name 받기

[샘플코드] Spring MVC 계층 구조

```
@RestController                                // Controller: 요청/응답
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService s) { this.service = s; }
    @GetMapping
    public List<Product> list() { return service.findAll(); }
}

@Service                                        // Service: 비즈니스 로직
public class ProductService {
    private final ProductRepository repo;
    public ProductService(ProductRepository r) { this.repo = r; }
    public List<Product> findAll() { return repo.findAll(); }
}
```

[핵심 요약] Spring MVC

핵심 포인트	반드시 숙지할 내용
Spring MVC	Model-View-Controller 패턴 기반의 Spring 웹 핵심 모듈
Boot와의 관계	Spring Boot는 내부적으로 Spring MVC를 그대로 사용 — Boot ⊃ MVC
어노테이션 분류	컴포넌트 식별·의존성 주입·환경설정· 웹 계층 ·데이터 접근·검증
REST 처리 흐름	요청 → Controller → Service → Repository → JSON 응답

IoC와 DI

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

IoC(Inversion of Control, 제어의 역전)

- 객체의 생성, 생명주기 관리, 의존성 주입 등을 개발자가 직접 하지 않고, 컨테이너가 대신 관리해주는 원리
- 전통적 방식(개발자 직접 제어)와 반대 개념 : 제어권이 개발자에서 프레임워크로 넘어감
- 코드 결합도를 낮추고, 유연한 아키텍처와 테스트가 가능한 구조로 만들 수 있음

[전통적 방식]

```
StockService stockService = new StockService(new StockRepository());
```

[IoC 적용]

```
@Autowired  
StockService stockService; // 객체 생성과 주입을 프레임워크가 대신함
```

DI(Dependency Injection, 의존성 주입)

- 객체가 의존하는 다른 객체를 "직접 생성"하지 않고, "외부(컨테이너)"에서 주입 받는 방식
- 객체 사이의 결합도를 낮추고, 테스트나 변경이 용이해 짐

DI 주입 방식	장점	단점
생성자 주입 (권장)	불변 객체 설계, 테스트 용이, 필수 의존성 보장	코드 다소 길어질 수 있음
필드 주입	코드 간결, 간단 실습에 적합	final 불가, 테스트 불편, 권장X
Setter 주입	선택적 의존성 주입에 적합	setter 오남용 가능, 불변성 약함

DI – 생성자 주입 (Constructor Injection)

- 주식(Stock) 정보를 조회하는 서비스(StockService)가 데이터 저장소(StockRepository)에 의존한다고 가정

```
@Service
public class StockService {
    private final StockRepository stockRepository;

    // 생성자 주입
    public StockService(StockRepository stockRepository) {
        this.stockRepository = stockRepository;
    }

    public String getStockInfo(String ticker) {
        return stockRepository.getStockInfo(ticker);
    }
}
```

❖ 생성자 주입에서 **private final** 선언의 필요성

- final로 선언하면 해당 필드는 초기화 이후 값이 변하지 않는 불변(immutable) 객체가 됨.
- 생성자를 통해 반드시 1번만 값이 할당되고, 이후 변경 불가 → 안정성과 신뢰성 확보
- Spring이 주입할 때도 반드시 생성자를 통해 값을 넣기 때문에, 필수 의존성이 반드시 주입 되었음을 컴파일 타임에 확인할 수 있음

DI – 필드 주입 (Field Injection)

- 주식(Stock) 정보를 조회하는 서비스(StockService)가 데이터 저장소(StockRepository)에 의존한다고 가정

```
@Service
public class StockService {

    @Autowired
    private StockRepository stockRepository;

    public String getStockInfo(String ticker) {
        return stockRepository.getStockInfo(ticker);
    }
}
```

DI – Setter 주입 (Setter Injection)

- 주식(Stock) 정보를 조회하는 서비스(StockService)가 데이터 저장소(StockRepository)에 의존한다고 가정

```
@Service
public class StockService {
    private StockRepository stockRepository;

    @Autowired
    public void setStockRepository(StockRepository stockRepository) {
        this.stockRepository = stockRepository;
    }

    public String getStockInfo(String ticker) {
        return stockRepository.getStockInfo(ticker);
    }
}
```

❖ Setter 주입에서도 @Autowired가 필요한 이유

- Setter 주입 방식에서는 객체 생성 후, Setter 메서드를 통해 Spring이 의존 객체를 “주입”
- 일반 자바 메서드는 Spring이 자동으로 호출해주지 않으므로, 이 메서드가 DI(의존성 주입) 용도임을 Spring에게 명확히 알려야 함 → @Autowired로 표시

[참고] 생성자 주입이 필드 주입보다 권장되는 이유

- 생성자 주입이 필드 주입보다 더 안전하고, 견고한 코드를 작성할 수 있게 해주기 때문에 생성자 주입을 권장

구분	상세 설명
불변성(Immutable) 보장	final 키워드 사용 가능. 객체 생성 후 의존성 변경 불가 → 안정성 증가
필수 의존성 보장	필수 의존성을 생성자 파라미터로 강제하여 누락 시 컴파일/런타임 에러 → 오류 조기 발견
테스트 용이성	단위 테스트 시 생성자를 통해 mock/fake 객체 직접 주입 가능 → 테스트 코드 작성 편리
순환 참조 조기 감지	순환 참조 발생 시 애플리케이션 구동 시점에 즉시 에러 발생 → 빠른 버그 확인
의존성 명확성 및 가독성	생성자 시그니처만 봐도 어떤 의존성이 필요한지 한눈에 파악 → 코드 가독성 및 유지보수 용이

[참고] 순환 참조 오류

- 생성자 주입은 객체를 생성하면서 필수로 모든 의존성을 동시에 주입해야 하므로, 서로 상대방을 필요로 하면 무한루프에 빠져 객체를 만들 수 없는 상태가 됨.
- 스프링은 이런 구조를 바로 감지해 시작 즉시 에러를 발생

```
*****
APPLICATION FAILED TO START
*****
```

Description:

The dependencies of some of the beans in the application context form a cycle:

```
aService (field private final BService aService.bService)
  ↓
bService (field private final AService bService.aService)
  ↓
aService (field private final BService aService.bService)
```

Action:

There is a circular dependency between two beans.
Consider revising your code.

❖ 해결 방법

- 순환 참조는 “비즈니스 설계” 자체를 다시 점검해야 한다는 신호
- 의존 구조를 리팩토링 하거나, 중간 계층(인터페이스, 이벤트 등)으로 분리해서 해결

[샘플코드] IoC & DI (@Qualifier / @Primary)

```
public interface PaymentGateway { void pay(int amount); }
```

@Component

@Primary // 기본 주입 대상

```
public class KakaoPay implements PaymentGateway {  
    public void pay(int amount) { }  
}
```

@Component("naver")

```
public class NaverPay implements PaymentGateway {  
    public void pay(int amount) { }  
}
```

@Service

```
public class OrderService {  
    private final PaymentGateway gateway;  
    public OrderService(@Qualifier("naver") PaymentGateway g) { // 특정 빈 지정  
        this.gateway = g;  
    }  
}
```

[핵심 요약] IoC와 DI

핵심 포인트	반드시 숙지할 내용
IoC	제어의 역전 — 객체 생성·의존성 관리를 컨테이너에 위임
DI	의존 객체를 직접 생성하지 않고 외부에서 주입 받는 방식
주입 방식	생성자 · 필드 · Setter — 생성자 주입 권장
순환 참조 주의	서로 의존 시 무한 루프 — 생성자 주입은 기동 시점에 오류를 드러냄

Controller와 HTTP 매개변수

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

@RestController

- RESTful 웹 서비스를 만들 때 사용하는 핵심 어노테이션
- 클래스 내의 모든 메서드가 HTTP 요청을 받아서 JSON, XML 등의 형태로 응답을 자동으로 반환

HTTP 메서드	어노테이션	설명	사용 예시
GET	@GetMapping	리소스 조회, 데이터 요청	@GetMapping("/users")
POST	@PostMapping	리소스 생성, 데이터 전송 및 등록	@PostMapping("/users")
PUT	@PutMapping	리소스 전체 수정	@PutMapping("/users/{id}")
PATCH	@PatchMapping	리소스 일부 수정	@PatchMapping("/users/{id}")
DELETE	@DeleteMapping	리소스 삭제	@DeleteMapping("/users/{id}")
(모든 메서드)	@RequestMapping	모든 HTTP 메서드에 사용 가능 - method로 지정	@RequestMapping(value="/users", method=RequestMethod.GET)

파라미터 바인딩 어노테이션

- 클라이언트가 보낸 HTTP 요청의 다양한 정보(URL 경로 변수, 쿼리 파라미터, 요청 본문 등)를 컨트롤러 메서드의 매개변수(파라미터)에 손쉽게 연결(바인딩)해주는 역할

어노테이션	역할/용도
@PathVariable	URL 경로의 값을 메서드 파라미터로 바인딩
@RequestParam	쿼리스트링, 폼 파라미터를 메서드 파라미터로 바인딩
@RequestBody	HTTP 요청 Body(JSON, XML 등)를 자바 객체로 변환
@ModelAttribute	폼 데이터/쿼리 파라미터를 객체에 바인딩
@RequestHeader	HTTP 요청 헤더 값을 파라미터로 바인딩
@CookieValue	쿠키 값을 파라미터로 바인딩

파라미터 바인딩 – @PathVariable

- URL 경로에 포함된 값을 메서드 파라미터로 바인딩

```
@GetMapping("/users/{id}")  
public User getUser(@PathVariable("id") Long id) {  
    // /users/100 → id = 100  
}
```

- URL의 **{id}** 부분이 파라미터로 전달됨
- 변수명이 같으면 **@PathVariable Long id**처럼 이름 생략 가능

파라미터 바인딩 – @RequestParam

- 쿼리 파라미터 또는 폼 데이터(POST form 등)를 바인딩 (?key=value 형태)

```
@GetMapping("/search")
public List<User> search(@RequestParam("keyword") String keyword) {
    // /search?keyword=Spring → keyword = "Spring"
}
```

여러 개 가능 : /search?name=kim&age=20

```
@GetMapping("/search")
public String search(@RequestParam String name, @RequestParam int age) {...}
```

필수 여부, 기본값 지정도 가능

```
@RequestParam(required=false, defaultValue="all") String type
```

[참고] 배열 형식의 파라미터 전달

- 같은 파라미터 이름을 반복해서 전달: **GET /search?category=java&category=spring&category=boot**

```
// 배열 형식으로 수신
@GetMapping("/search")
public String search(@RequestParam("category") String[] categories) {
    // categories: ["java", "spring", "boot"]
    return Arrays.toString(categories);
}

// List로 수신
@GetMapping("/search")
public String search(@RequestParam List<String> category) {
    // category: [java, spring, boot]
    return category.toString();
}
```

- coma(,)로 구분된 문자열 전달: **GET /search?category=java,spring,boot**

```
@GetMapping("/search")
public String search(@RequestParam("category") List<String> categories) {
    // categories: ["java", "spring", "boot"]
    return categories.toString();
}
```

❖ Spring은 기본적으로 coma(,)로 구분된 값을 분리해서 리스트/배열로 변환

파라미터 바인딩 – @RequestBody

- HTTP 요청의 본문(Body)에 담긴 JSON, XML 등 데이터를 자바 객체로 자동 변환(Mapping)
- 주로 POST, PUT, PATCH에서 사용

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
    // 요청 body의 JSON → User 객체 자동 매핑
}
```

파라미터 바인딩 – @ModelAttribute

- HTTP폼 데이터, 쿼리 파라미터 등을 자바 객체(모델)에 바인딩
- **폼 기반 페이지**에서 전달한 데이터 처리에 사용

```
@PostMapping("/users/form")
public String register(@ModelAttribute UserForm userForm) {
    // 폼의 name, age 필드 → UserForm 객체에 자동 바인딩
}
```

[참고] Form Data 처리

- 클라이언트에서 HTML 폼, 혹은 JavaScript의 FormData 형식의 데이터를 전송
 - `<form>` 태그를 통한 제출 (*submit*)
 - JS의 *FormData* 객체로 전송
 - Content-Type*이 *application/x-www-form-urlencoded* 또는 *multipart/form-data* 형태로 서버에 전달됨

```
// 개별 파라미터로 받기
```

```
@PostMapping("/register")
public String register(@RequestParam String username, @RequestParam String password) {
    // 각각의 폼 필드를 개별로 받음
}
```

```
// DTO(VO, Model) 객체로 한 번에 바인딩 (@ModelAttribute 사용)
```

```
public class UserForm {
    private String username;
    private String password;
    // getter, setter
}

@PostMapping("/register")
Public String register(@ModelAttribute UserForm form) {
    // 폼의 name과 동일한 필드명으로 자동 매핑됨
}
```

파라미터 바인딩 – @RequestHeader

- HTTP 요청 헤더 값을 파라미터로 받음 (ex. 인증, 토큰, User-Agent 등)

```
@GetMapping("/users/me")
public User getMe(@RequestHeader("Authorization") String token) {
    // Authorization: Bearer eyJhbGci... → token 파라미터로 전달
}
```


파라미터 바인딩 – @CookieValue

- 요청 쿠키 값을 파라미터로 받음

```
@GetMapping("/visit")
public String visit(@CookieValue("visitTime") String visitTime) {
    // 쿠키에 저장된 visitTime 값 사용
}
```

[실습] “courses” 엔트리 포인트 추가

- HelloController에 “courses” 엔트리 포인트 추가하고 CourseService 추가로 주입
 - @PathVariable로 {이름} 받기
 - @RequestParam List<String> topics에 콤마로 구분된 관심 분야 배열 받기
- CourseService에 createCourse()를 통해 바인딩 파라미터 전달하면 CourseResponse 생성하여 리턴
- 응답 데이터 CourseResponse는 name, topics, description 속성 포함

POST /courses/홍길동?topics=Spring,Java,AI

```
{
  "name": "홍길동",
  "topics": ["Spring", "Java", "AI"],
  "description": "홍길동님이 관심 있는 분야: Spring, Java, AI"
}
```

[코드] data/CourseResponse.java

```
import lombok.Data;
import java.util.List;

@Data
public class CourseResponse {
    private String name;
    private List<String> topics;
    private String description;
}
```

[코드] service/CourseService.java

```
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class CourseService {

    public CourseResponse createCourse(String name, List<String> topics) {
        CourseResponse response = new CourseResponse();
        response.setName(name);
        response.setTopics(topics);

        String desc = String.format(
            "%s님이 관심 있는 교과과정: %s",
            name, String.join(", ", topics)
        );
        response.setDescription(desc);

        return response;
    }
}
```

[코드] controller/HelloController.java

```
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
public class HelloController {

    private final HelloService helloService;
    private final CourseService courseService;

    // 생성자 주입
    public HelloController(HelloService helloService, CourseService courseService) {
        this.helloService = helloService;
        this.courseService = courseService;
    }

    @GetMapping("/hello")
    public HelloResponse hello(@RequestParam(defaultValue = "SKALA") String name) {
        // 서비스 호출하여 응답 객체 생성
        return helloService.createMessage(name);
    }

    @PostMapping("/courses/{name}")
    public CourseResponse getClassInfo(@PathVariable String name, @RequestParam List<String> topics) {
        // Service에서 CourseResponse 생성
        return courseService.createCourse(name, topics);
    }
}
```

[샘플코드] Controller & HTTP 매개변수

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping("/{id}")                // 경로 변수
    public Product get(@PathVariable Long id) { ... }

    @GetMapping                        // 쿼리 ?keyword=..&page=0
    public List<Product> search(@RequestParam String keyword,
                               @RequestParam(defaultValue = "0") int page) { ... }

    @PostMapping                      // JSON 본문
    public Product create(@RequestBody ProductRequest req) { ... }

    @GetMapping("/auth")
    public String auth(@RequestHeader("X-Auth") String token) { ... }
}
```

[핵심 요약] Controller와 HTTP 매개변수

핵심 포인트	반드시 숙지할 내용
@RestController	REST API 엔드포인트 정의(JSON 반환)
경로/쿼리	@PathVariable(경로 변수) · @RequestParam(쿼리 파라미터)
본문/폼	@RequestBody(JSON 본문) · @ModelAttribute(폼 데이터)
헤더/쿠키	@RequestHeader · @CookieValue 로 헤더·쿠키 값 바인딩

Lombok

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Lombok 어노테이션

- 개발자의 **boilerplate code**를 줄이고 가독성을 높일 수 있도록 지원하는 Java 라이브러리

❖ **Boilerplate Code(보일러플레이트 코드)는 ?**

- 반복적이고 본질적인 로직과 무관하지만 반드시 작성해야 하는 코드
- Java에서는 객체의 필드를 다룰 때 getter, setter, 메서드 등이 대표적인 보일러플레이트 코드

주요 Lombok 어노테이션

어노테이션	역할/설명	코드 사용 위치
@Getter	모든 필드에 대해 getter 메서드 자동 생성	클래스
@Setter	모든 필드에 대해 setter 메서드 자동 생성	클래스
@ToString	toString() 메서드 자동 생성	클래스
@EqualsAndHashCode	equals()와 hashCode() 메서드 자동 생성	클래스
@NoArgsConstructor	기본 생성자(파라미터 없는 생성자) 자동 생성	클래스
@AllArgsConstructor	모든 필드를 파라미터로 받는 생성자 자동 생성	클래스
@RequiredArgsConstructor	final, @NonNull 필드만을 파라미터로 받는 생성자 자동 생성	클래스
@Data	@Getter, @Setter, @ToString, @EqualsAndHashCode, @RequiredArgsConstructor를 한 번에 적용	클래스
@Builder	빌더 패턴 자동 생성 (객체 생성 유연화)	클래스, 생성자, 메서드
@Value	모든 필드를 final로 하여 불변 객체 생성, getter만 생성	클래스
@Slf4j	로그 변수(log) 자동 생성 (Log 객체 주입)	클래스

Lombok 적용 예시

```
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import lombok.Builder;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    private Long id;
    private String name;
}
```

```
public class User {
    private Long id;
    private String name;

    // 기본 생성자
    public User() {}

    // 모든 필드 초기화 생성자
    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getter & Setter
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    // toString() : 객체 정보를 문자열로 반환
    @Override
    public String toString() {
        return "User{id=" + id + ", name='" + name + "'}";
    }

    // equals() : 두 객체가 같은지 비교 (주로 id와 name이 모두 같을 때)
    // hashCode() : 객체를 해시 기반 컬렉션(Map, Set)에 넣을 때 사용
}
```

Lombok - @Value

- 불변(immutable) 객체를 만들 때 사용
- 모든 필드가 private final이 되고, setter는 미생성 - 생성자, getter, toString, equals, hashCode가 자동 생성

```
import lombok.Value;

@Value
public class Stock {
    String symbol;
    String name;
    int price;
}

//객체 생성 후 필드 값을 변경 불가

Stock stock = new Stock("005930", "삼성전자", 80000);
// stock.setPrice(90000); // 컴파일 오류 (setter 없음, 값 변경 불가)
System.out.println(stock.getName()); // "삼성전자"
```

Lombok - @Slf4j

- 로그(Logger) 객체를 자동으로 주입 - 클래스에 붙이면 `private static final org.slf4j.Logger log` 필드가 생성
- 로그 레벨별로 출력: `log.info()`, `log.warn()`, `log.error()`, `log.debug()`, `log.trace()`
- `System.out.println` 대신 사용

```
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class StockService {

    public void printStock(Stock stock) {
        log.info("Stock 정보: {}", stock);
        log.debug("디버깅 로그입니다");
        log.error("에러 상황입니다");
    }
}

// 에러 출력 시에는 마지막 인자에 Exception 전달
log.error("에러 발생: {}", userId, e); // SLF4J 권장 방식
```

- SLF4J 로그 메시지 포맷은 {}만 지원.
- %s, %d 등은 지원하지 않으므로 필요하면 `String.format()` 사용 가능 (비권장 - 문자열 결합 및 객체 생성 연산이 발생)

❖ 애플리케이션 관점의 로그 관리는 별도 세션에서 상세히 설명

[참고] 로그 생성 기준

- **TRACE**: 가장 상세한 정보. 특정 작업의 시작과 끝, 변수 값의 변화 등 아주 세밀한 추적이 필요할 때 사용(주로 개발 단계)
- **DEBUG**: 개발 단계에서 내부 시스템의 동작을 확인하고 디버깅하는 데 필요한 정보를 기록 (특정 메소드의 파라미터나 반환 값, 내부 변수 값 등).
- **INFO**: 시스템의 주요 동작 상태나 중요한 비즈니스 프로세스의 진행 상황을 표시하여 운영 환경에서도 추적이 용이하도록 핵심적인 정보를 기록 ('사용자 A가 로그인', '주문 B가 생성'과 같은 정보)
- **WARN**: 심각한 문제는 아니지만, 잠재적인 위험이나 예상치 못한 상황을 알릴 때 사용 (당장 에러는 아니지만 주의 깊게 살펴봐야 할 상황을 의미 - '지원 중단 예정인 API가 호출되었습니다'와 같은 경우)
- **ERROR**: 심각한 오류가 발생하여 애플리케이션의 정상적인 기능 수행에 문제가 생긴 경우 사용 (즉시 조치가 필요한 상황이며, 문제 해결을 위한 충분한 정보를 포함해야 함 - 예외(Exception)가 발생했을 때 주로 사용)

• Controller 계층에서의 로깅 패턴

- **INFO 레벨**: 어떤 요청이 들어왔는지 기록 - HTTP 메소드, 요청 URI, 요청을 보낸 클라이언트의 IP 주소 등
예시: `log.info("[REQUEST] {} {} clientIp={}", request.getMethod(), request.getRequestURI(), clientIp);`
- **DEBUG 레벨**: 요청에 포함된 파라미터나 `@RequestBody`의 내용을 기록하여 디버깅 시 활용 (민감한 개인정보는 반드시 마스킹 처리)
예시: `log.debug("Request DTO: {}", requestDto.toString());` (DTO의 `toString()` 메소드에 민감 정보가 포함되지 않도록 주의)

• Service 계층에서의 로깅 패턴

- **INFO 레벨**: 어떤 사용자가 어떤 작업을 수행했는지 명확히 기록
예시: `log.info("Creating a new order for user: {}", userId);`
- **DEBUG 레벨**: 메소드의 주요 파라미터와 반환 값, 그리고 비즈니스 로직 중간의 중요한 데이터 상태를 기록
예시: `log.debug("Calculated total price for order {}: {}", orderId, totalPrice);`

[실습] “@Slf4j” 를 이용한 로깅 설정

- Controller와 Service 클래스에 @Slf4j 어노테이션을 설정하고 info() 및 debug() 이력 생성

[샘플코드] Lombok

```
@Data                // getter/setter/toString>equals 자동 생성
@Builder             // 빌더 패턴
@NoArgsConstructor
@AllArgsConstructor

public class Product {
    private Long id;
    private String name;
    private int price;
}

@Slf4j                // log 필드 자동 생성
@Service
public class ProductService {
    public void save(Product p) {
        log.info("save product: {}", p.getName());    // 로깅
    }
}

// 사용: Product.builder().name("mouse").price(15000).build();
```


[핵심 요약] Lombok

핵심 포인트	반드시 숙지할 내용
목적	boilerplate 코드 제거 — getter/setter/생성자 등 자동 생성
주요 어노테이션	@Data · @Getter · @Setter · @NoArgsConstructor 등
@Value / @Slf4j	@Value(불변 객체) · @Slf4j (로거 자동 생성)
로그 레벨	TRACE · DEBUG · INFO · WARN · ERROR 단계 구분

입력값 검증

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

입력값 검증이란?

- 클라이언트(사용자)로부터 받은 데이터가 사전에 정의된 규칙에 맞는지 확인하는 과정
- 검증 과정 없이 잘못된 데이터가 시스템으로 들어오면, 예상치 못한 오류가 발생하거나 데이터베이스에 원치 않는 값이 저장될 수 있음
- 입력값 검증 과정을 통해 애플리케이션의 안정성과 데이터의 신뢰성을 확보

❖ 의존성 추가 설정 필요

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

입력값 검증 - @Valid vs @Validated

@Valid : Java 표준 어노테이션

- 간단한 객체 검증에 사용
- 그룹 지정 불가(무조건 기본 그룹만 사용)
- 대부분의 일반적인 검증은 이것로 충분함

@Validated : Spring 제공 확장 검증 어노테이션

- 그룹을 지정하여 상황별로 다르게 검증 가능
- 그룹을 구분해서 메서드나 클래스에 적용할 수 있음
- Spring AOP 기반으로 컨트롤러/서비스 계층에서도 사용 가능

❖ 검증 실패 시 발생하는 예외 (@Valid와 @Validated 모두 동일)

어노테이션 적용 위치	발생 예외
@RequestBody	MethodArgumentNotValidException
@ModelAttribute	BindException
@RequestParam	ConstraintViolationException

입력값 검증 어노테이션

- 개별 데이터에 설정하고 @Valid 또는 @Validated를 통해 검증

어노테이션	설명	예시
@NotNull	null이 아니어야 함	필수 입력값
@NotEmpty	null 또는 빈 문자열("")이 아니어야 함	공백 불가(문자열, 컬렉션)
@NotBlank	null, 빈문자, 공백문자만 있는 값 불가	공백불가(문자열)
@Size	문자열, 컬렉션, 배열의 길이/크기 제한	@Size(min=2, max=20)
@Min	최소값(숫자)	@Min(18)
@Max	최대값(숫자)	@Max(100)
@Positive	양수여야 함	1, 2, ...
@PositiveOrZero	0 또는 양수여야 함	0, 1, 2, ...
@Negative	음수여야 함	-1, -2, ...
@NegativeOrZero	0 또는 음수여야 함	0, -1, -2, ...
@Email	이메일 형식	@Email
@Pattern	정규표현식 패턴 일치	@Pattern(regexp="^[0-9]+\$")
@Past	과거 날짜여야 함	생년월일 등
@PastOrPresent	과거나 오늘 날짜	
@Future	미래 날짜여야 함	예약일 등
@FutureOrPresent	오늘 또는 미래 날짜여야 함	
@Digits	자릿수 및 소수점 자리 제한	@Digits(integer=5, fraction=2)
@AssertTrue	반드시 true	체크박스 등
@AssertFalse	반드시 false	
@Null	null이어야 함	

@Valid 검증 예시

- 일반적인 단일 검증 방식 - 유효성 검증은 Service가 아닌 Controller에서 처리하는 것이 표준 아키텍처 패턴

```
// DTO
public class UserRequest {
    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(18)
    private int age;
}
```

```
// 컨트롤러에서 @Valid 사용 검증
@PostMapping("/users")
public ResponseEntity<?> createUser(@Valid @RequestBody UserRequest request) {
    // 유효성 검증 통과 시 로직 진행
    return ResponseEntity.ok("가입 완료");
}
```

❖ 검증 실패 시 **400 Bad Request**로 예외 발생

[참고] Controller에서 검증하는 이유

- **역할과 책임의 분리 (관심사의 분리) :** 각 계층은 자신만의 명확한 역할과 책임이 있음
 - Controller 계층: HTTP 요청을 받아들이는 애플리케이션의 관문 역할로 요청으로 들어온 데이터(Payload, Parameter)의 형식, 구문, 값의 유효성을 검증하고 인증/인가 여부를 확인한 후 유효한 데이터를 Service 계층으로 전달
 - Service 계층: Controller로부터 받은 데이터를 가지고 핵심 비즈니스 로직을 수행하면서 데이터의 형식이 아닌, 비즈니스 규칙에 맞는지를 검증 (예: "이미 존재하는 이메일인가?", "재고가 충분한가?")과 트랜잭션 처리
- **빠른 실패(Fail-Fast)의 원칙 :** 잘못된 데이터는 가능한 한 시스템의 가장 바깥쪽에서, 가장 빨리 차단하는 것이 효율적
- **예외 처리의 일관성**
 - 컨트롤러에서 검증이 실패하면 전역적 예외 처리(**MethodArgumentNotValidException 발생**)를 통해 일관된 형식의 응답(어떤 필드가 왜 잘못되었는지)을 클라이언트에게 보내주기 용이
 - 만약 서비스 계층에서 검증을 한다면 ConstraintViolationException이 발생하는데, 이는 비즈니스 로직 예외와 입력값 검증 예외가 섞여 예외 처리 로직이 더 복잡해질 수 있음.

구분	입력값/형식 검증 (Syntactic Validation)	비즈니스 검증 (Business Validation)
담당 계층	Controller	Service
목적	요청 데이터가 약속된 형식과 규칙에 맞는지 확인	데이터가 비즈니스 규칙에 부합하는지 확인

@Validated 검증 예시

■ 그룹별 조건 검증

```
// 그룹 인터페이스 선언
public interface CreateGroup {}
public interface UpdateGroup {}
```

```
// DTO에 그룹 지정
public class UserRequest {

    @NotBlank(groups = CreateGroup.class)
    private String name;

    @Email(groups = {CreateGroup.class, UpdateGroup.class})
    private String email;

    @Min(value = 18, groups = CreateGroup.class)
    private int age;
}
```

```
// Controller에서 그룹 지정하여 검증
```

```
// 생성(Create) 요청: 모든 값 검증
@PostMapping("/users")
public ResponseEntity<?> createUser(
    @Validated(CreateGroup.class) @RequestBody UserRequest
    request) {
    // name, email, age 모두 검증됨
    return ResponseEntity.ok("회원 등록");
}
```

```
// 수정(Update) 요청: 이메일만 검증
@PutMapping("/users/{id}")
public ResponseEntity<?> updateUser(
    @Validated(UpdateGroup.class) @RequestBody
    UserRequest request) {
    // email만 검증됨
    return ResponseEntity.ok("회원 정보 수정");
}
```


[실습] 요청 데이터 검증

- data/HelloRequest.java DTO를 생성하고 입력값 검증 어노테이션 설정
- HelloController에 POST 방식의 "/hello" 엔트리 포인트 추가하고 @Valid를 사용하여 HelloRequest 입력값 검증 처리

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;

@Data
public class HelloRequest {
    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(18)
    private int age;
}
```

```
POST /hello

{
  "name": "홍길동",
  "email": "gdhong@savingpoors.org",
  "age": 25
}
```

[코드] HelloController.java

```
@GetMapping("/hello")
public HelloResponse hello(@RequestParam(defaultValue = "SKALA") String name) {
    // 서비스 호출하여 응답 객체 생성
    log.info("/hello: GET {}", name);
    return helloService.createMessage(name);
}

@PostMapping("/hello")
public HelloResponse postHello(@Valid @RequestBody HelloRequest body) {
    log.info("/hello: POST {}", body.getName());
    return helloService.createMessage(body.getName());
}
```

[코드] 입력값 검증 요청 결과

POST http://localhost:8080/hello

Params Authorization Headers (9) **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1 {
2   "name": "",
3   "email": "",
4   "age": 0
5 }

```

Body Cookies Headers (4) Test Results **400 Bad Request**

{} JSON ▾ ▶ Preview 🔄 Visualize ▾

```

1 {
2   "timestamp": "2025-08-06T01:24:29.399+00:00",
3   "status": 400,
4   "error": "Bad Request",
5   "path": "/hello"
6 }

```

검증 → 실패

POST http://localhost:8080/hello

Params Authorization Headers (9) **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1 {
2   "name": "홍길동",
3   "email": "gdhong@poors.org",
4   "age": 25
5 }

```

Body Cookies Headers (5) Test Results **200 OK**

{} JSON ▾ ▶ Preview 🔄 Visualize ▾

```

1 {
2   "message": "안녕하세요, 홍길동님!"
3 }

```

검증 → 성공

검증 결과 로그

Validation failed for argument [0] in public com.skala.basic.data.HelloResponse
 com.skala.basic.controller.HelloController.postHello(com.skala.basic.data.HelloRequest) with 2 errors: [Field error
 in object 'helloRequest' on field 'name': rejected value []; codes
 [NotBlank.helloRequest.name,NotBlank.name,NotBlank.java.lang.String,NotBlank]; arguments
 [org.springframework.context.support.DefaultMessageSourceResolvable: codes [helloRequest.name,name]; arguments [];
 default message [name]]; default message [공백일 수 없습니다]] [Field error in object 'helloRequest' on field 'age':
 rejected value [0]; codes [Min.helloRequest.age,Min.age,Min.int,Min]; arguments
 [org.springframework.context.support.DefaultMessageSourceResolvable: codes [helloRequest.age,age]; arguments [];
 default message [age],18]; default message [18 이상이어야 합니다]]]

❖ 검증 실패 시 **400 Bad Request** 대신 검증 조건을 전달할 수 있을까?

[샘플코드] 입력값 검증 (@Valid)

```
public class ProductRequest {
    @NotBlank(message = "이름은 필수입니다")    // 빈 문자열 불가
    private String name;
    @Min(value = 0, message = "0 이상")          // 최소값
    private int price;
    @Email                                       // 이메일 형식
    private String sellerEmail;
    // getter / setter
}

@RestController
public class ProductController {
    @PostMapping("/api/products")
    public Product create(@RequestBody @Valid ProductRequest req) {
        return service.create(req);              // 검증 통과 시 실행
    }
}
```

[핵심 요약] 입력값 검증

핵심 포인트	반드시 숙지할 내용
검증이란	요청 데이터가 규칙에 맞는지 확인 → 안정성·데이터 신뢰성 확보
@Valid vs @Validated	@Valid(Java 표준) vs @Validated (Spring, 그룹 검증 지원)
제약 어노테이션	@NotNull · @NotBlank · @Size · @Min/@Max 등
검증 위치	Controller 계층 에서 검증 — 관심사(역할) 분리

예외 처리 (ExceptionHandler)

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

[참고] Java의 예외 처리 (1/2)

- 일반적으로 예외를 처리할 블록에 try-catch 구문을 통해 처리
- ❖ **파일, 네트워크 소켓, 데이터베이스 커넥션 등과 같은 자원**은 사용이 완료되면 반드시 명시적으로 닫아야 함.
→ 그렇지 않으면 자원 누수(Resource Leak)가 발생

finally를 이용한 자원 해제(전통적인 방식)

```
public class FinallyExample {  
    public static void main(String[] args) {  
        BufferedReader reader = null;  
        try {  
            reader = new BufferedReader(new FileReader("test.txt"));  
            String line = reader.readLine();  
            System.out.println(line);  
        } catch (IOException e) {  
            e.printStackTrace();  
        } finally {  
            // 자원 해제  
            if (reader != null) {  
                try {  
                    reader.close();  
                } catch (IOException e) {  
                    // 닫을 때도 예외가 발생할 수 있음  
                    e.printStackTrace();  
                }  
            }  
        }  
    }  
}
```

[참고] Java의 예외 처리 (2/2)

- `try()` 괄호 안에서 선언한 자원은 블록이 끝나면 자동으로 `close()` 되어 코드가 간결하고 실수할 위험이 적음
- 자원을 여러 개 선언할 수도 있음: `try (Resource1 r1 = ...; Resource2 r2 = ...)`

try-with-resources(자동 자원 해제, Java 7+)

```
import java.io.*;

public class TryWithResourcesExample {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("test.txt"))) {
            String line = reader.readLine();
            System.out.println(line);
        } catch (IOException e) {
            e.printStackTrace();
        }
        // reader.close() 자동 호출됨
    }
}
```


스프링의 예외 처리 방식

- 별도의 예외 처리 설정을 하지 않았을 때, 스프링 부트는 `BasicExceptionHandler`를 사용하여 처리되지 않은 예외를 자동으로 처리
- 웹 브라우저에서 페이지를 요청했을 때 오류가 발생하면 'Whitelabel Error Page' 형식으로 응답
- API 클라이언트가 서버에 요청(요청 헤더의 `Accept` 값이 `application/json`일 때) JSON 형식으로 응답

응답의 내용은 동일

```
{
  "timestamp": "2025-08-06T02:00:15.931+00:00",
  "status": 404,
  "error": "Not Found",
  "path": "/api/non-existent-endpoint"
}
```

@ControllerAdvice와 @ExceptionHandler

- 스프링 부트에서 제공하는 강력하고 유연한 예외 처리 메커니즘
- 가장 일반적이고 권장되는 전역(Global) 예외 처리 방식 - 애플리케이션의 모든 처리과정에서 발생하는 예외를 한 곳에서 중앙 집중적으로 처리

❖ 예외 처리 우선순위

- ① **컨트롤러 내 @ExceptionHandler**: 현재 컨트롤러에 해당 예외를 처리하는 핸들러가 있는지 먼저 확인
- ② **@ControllerAdvice 내 @ExceptionHandler**: 적절한 핸들러를 찾지 못하면 전역 예외 처리기에서 핸들러를 검색 - 여러 @ControllerAdvice가 있다면 @Order 어노테이션으로 순서를 지정
- ③ **ResponseStatusException**: 위의 두 경우에 해당하지 않고 ResponseStatusException이 던져지면 스프링이 직접 처리
- ④ **스프링 부트 기본 예외 처리**: 위의 어떤 방법으로도 처리되지 않은 예외는 스프링 부트의 기본 BasicErrorController가 처리 - 기본적인 오류 페이지나 JSON 응답

@ControllerAdvice를 통한 예외 처리

- 스프링(Spring) 애플리케이션에서 전역적으로 예외(Exception)를 처리하기 위해 사용하는 어노테이션
- 역할에 따라 예외 처리를 분리하도록 여러 개를 사용할 수 있으나, 충돌을 피하고 의도한 대로 동작하게 하려면 @Order로 우선 순위 설정 필요

@ControllerAdvice // @RestControllerAdvice로 변경 가능

```
public class GlobalExceptionHandler {

    // @Valid, @Validated(@RequestBody) 예외 처리
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors);
    }

    // 기타 예외 (예: @ModelAttribute, @RequestParam 등)
    // 필요 시 추가 처리
}
```

- ❖ 컨트롤러에서 검증이 실패하면 전역적 예외 처리(**MethodArgumentNotValidException**) 발생
- ❖ **@RestControllerAdvice**는 내부적으로 @ControllerAdvice + @ResponseBody 기능을 모두 포함 : REST API에서는 대부분 @RestControllerAdvice 사용을 권장

[실습] exception/GlobalExceptionHandler.java

- /exception폴더를 생성하고 전역 예외처리를 위한 GlobalExceptionHandler.java 클래스 생성
- 컨트롤러의 @Valid에 의해 발생한 MethodArgumentNotValidException을 @ExceptionHandler 를 통해 처리
- JSON 형식의 응답으로 검증 실패한 속성의 오류 원인을 전달

```
{  
    "name": "공백일 수 없습니다",  
    "email": "올바른 형식의 이메일 주소여야 합니다"  
}
```

[코드] exception/GlobalExceptionHandler.java

```
package com.skala.basic.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

import java.util.HashMap;
import java.util.Map;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors);
    }
}
```

[샘플코드] 예외 처리 (@RestControllerAdvice)

```
@RestControllerAdvice                                // 전역 예외 처리
public class GlobalExceptionHandler {

    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<String> handleNotFound(NoSuchElementException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class) // 검증 실패
    public ResponseEntity<String> handleValid(MethodArgumentNotValidException e) {
        return ResponseEntity.badRequest()
            .body(e.getBindingResult().getFieldError().getDefaultMessage());
    }
}
```

[핵심 요약] 예외 처리

핵심 포인트	반드시 숙지할 내용
기본 처리	미처리 예외는 스프링 부트의 BasicExceptionHandler 가 처리
전역 예외 처리	@ControllerAdvice + @ExceptionHandler 로 예외를 한 곳에서 중앙 집중 처리
일관된 응답	예외 유형별로 일관된 에러 응답(코드·메시지) 구성
자원 관리	try-with-resources로 자원 자동 close — 간결·안전

자동 설정 (Auto Configuration)

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

자동 설정(Auto-configuration)

- "Convention over Configuration(설정보다 관례)" 철학을 기반
- 개발자의 특별한 설정이 없는 이상, 가장 보편적이고 표준적인 방식으로 애플리케이션 동작 방식을 구성
- 애플리케이션의 메인 클래스에 붙는 @SpringBootApplication 안에 내장된 @EnableAutoConfiguration : 스프링 부트의 "클래스 패스"를 확인하고 필요한 설정들을 자동으로 수행

스프링 스타터의 자동 설정

- 스프링 부트의 **스타터(Starter)**는 특정 기능 개발에 필요한 의존성 모음
- 개발자가 스타터를 추가하면, 스프링 부트의 자동 설정(Auto-configuration) 기능을 통해 필요한 빈(Beans)들을 자동으로 등록하고 설정함

Starter 의존성	자동 설정 기능 예시	설명
spring-boot-starter-web	- 내장 톰캣(Embedded Tomcat) - DispatcherServlet- WebMvcConfigurer - 기본 에러 페이지 핸들링	웹 어플리케이션 개발에 필요한 핵심 컴포넌트 자동 등록
spring-boot-starter-data-jpa	- DataSource - EntityManagerFactory - JpaRepository 구현체 - 트랜잭션 관리(TransactionManager)	JPA/Hibernate 기반 DB 연동을 위한 설정 자동 구성
spring-boot-starter-security	- Spring Security Filter Chain - 기본 로그인 폼 - PasswordEncoder - UserDetailsService	보안, 인증/인가를 위한 필수 컴포넌트 자동 구성
spring-boot-starter-thymeleaf	- ThymeleafViewResolver - TemplateEngine - 템플릿 캐싱 설정	Thymeleaf 템플릿 엔진 연동 자동 설정
spring-boot-starter-validation	- Validator - MethodValidationPostProcessor	Bean Validation 자동 적용
spring-boot-starter-actuator	- /actuator 엔드포인트 자동 등록 - 헬스 체크, 모니터링, 메트릭스 제공	운영/모니터링 관련 기능을 손쉽게 추가

애플리케이션 설정 정보 관리

- resources 디렉토리의 application.properties 또는 application.yml 파일은 Spring Boot에서 애플리케이션의 설정 정보를 관리하는 표준 방식
- application.properties / application.yml 역할
 - 환경설정 관리:** 데이터베이스 접속 정보, 서버 포트, 로깅 레벨, 외부 API 키, 커스텀 프로퍼티 등 애플리케이션 동작에 필요한 다양한 설정을 정의
 - 프로젝트의 런타임 환경별 설정 관리:** 개발, 테스트, 운영 환경에 따라 각기 다른 설정값을 분리하고 관리

application.properties

```
server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/stockdb
spring.datasource.username=stockuser
spring.datasource.password=stockpw

logging.level.org.springframework=INFO

# 사용자 정의 값
stock.api.base-url=https://api.stock.com
```

application.yml

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/stockdb
    username: stockuser
    password: stockpw

logging:
  level:
    org.springframework: INFO

stock:
  api:
    base-url: https://api.stock.com
```

설정 정보 사용 – @Value

- 장점: 간단하게 한두 개 값 읽을 때 적합
- 단점: 값이 많아지면 관리가 어렵고, 계층구조 표현이 불편

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class StockApiClient {
    @Value("${stock.api.base-url}")
    private String baseUrl;

    // baseUrl을 사용하는 코드
    public void connect() {
        System.out.println("API URL: " + baseUrl);
    }
}
```

설정 정보 사용 – @ConfigurationProperties

- 계층적 구조의 설정을 객체로 묶어 사용할 때 활용

```
# application.yml

server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/stockdb
    username: stockuser
    password: stockpw

stock:
  api:
    base-url: https://api.stock.com
    timeout: 1000
```

```
stock:
  api:
    base-url: https://api.stock.com
    timeout: 1000
  caching:
    enabled: true
```

```
@Data
@Component
@ConfigurationProperties(prefix = "stock.api")
public class StockApiProperties {
    private String baseUrl;
    private int timeout;
}
```

```
@Data
@Component
@ConfigurationProperties(prefix = "stock")
public class StockProperties {
    private Api api;
    private Caching caching;

    @Data
    public static class Api {
        private String baseUrl;
        private int timeout;
    }

    @Data
    public static class Caching {
        private boolean enabled;
    }
}
```

[참고] 설정 정보 클래스 서비스 사용 예시

```
@Service
public class StockApiClient {
    private final StockApiProperties stockApiProperties;

    public StockApiClient(StockApiProperties stockApiProperties) {
        this.stockApiProperties = stockApiProperties;
    }

    public void connect() {
        System.out.println("API URL: " + stockApiProperties.getBaseUrl());
    }
}
```

런타임 환경별 설정 정보 관리

- 런타임 환경(개발, 테스트, 운영 등)에 따라 설정 파일명을 구분하여 관리
 - application-**dev**.yml (개발)
 - application-**prod**.yml (운영)
 - application-**test**.yml (테스트)
- 실행 명령어에서 환경(profile) 지정 방법

JVM 옵션

```
java -Dspring.profiles.active=dev -jar skala-stock-api.jar
```

Spring Boot 옵션

```
java -jar skala-stock-api.jar --spring.profiles.active=dev
```

- ❖ **JVM 옵션(-Dspring.profiles.active=dev)**은 반드시 java 명령어 바로 뒤에 와야 함 → JAVA 환경변수 처리(Spring Boot가 내부적으로 System.getProperty를 통해 참조)
- ❖ **Spring Boot 옵션(--spring.profiles.active=dev)**은 jar 뒤에 사용 → 애플리케이션 아규먼트로 전달

[참고] VS Code 런타임 설정 실행

- .vscode/launch.json 파일에 환경변수나 아규먼트를 지정

환경변수 방식

```
{
  "type": "java",
  "name": "BasicApplication",
  "request": "launch",
  "mainClass": "com.skala.basic.BasicApplication",
  "projectName": "basic",
  "env": {
    "SPRING_PROFILES_ACTIVE": "dev"
  }
}
```

아규먼트 방식

```
{
  "type": "java",
  "name": "BasicApplication",
  "request": "launch",
  "mainClass": "com.skala.basic.BasicApplication",
  "projectName": "basic",
  "args": "--spring.profiles.active=dev"
}
```


[실습] 스타터에 필요한 정보 설정

- Maven 프로젝트 pom.xml에는 spring-boot-starter-actuator가 등록되어 있음.
- Actuator에 의해 제공되는 health 정보를 상세히 보기 위해 application.yml의 설정 정보 변경

```
http://localhost:8080/actuator/health
```

```
{"status":"UP"}
```

- ❖ **application.properties와 application.yml이 동시에 존재할 때**, Spring Boot는 두 파일을 모두 읽고 동일한 key가 있으면 application.properties가 application.yml보다 우선 적용(properties가 yml 값을 덮어씌움)
- ❖ 유지보수, 혼란 방지를 위해 한 가지 포맷만 사용하는 것이 안전 (**application.yml 사용 권장**)

[코드] resources/application.yml

```
server:  
  port: 8080  
  
management:  
  endpoint:  
    health:  
      show-details: always
```

```
http://localhost:8080/actuator/health
```

[참고] 스프링 부트 Actuator

- Spring Boot 애플리케이션의 상태 확인, 시스템 모니터링, 관리(Management), 운영 자동화 등을 위한 운영용 엔드 포인트 (REST API)를 자동으로 제공하는 모듈

기능 구분	설명	엔드 포인트
헬스 체크	시스템, DB 등 주요 컴포넌트 상태 확인	/actuator/health
애플리케이션 정보	버전, 이름 등 info 정보	/actuator/info
메트릭	JVM, CPU, 메모리, 트래픽, GC, 요청 수, response time 등	/actuator/metrics
환경변수	환경, 프로퍼티, 시스템 변수	/actuator/env
빈 정보	스프링 빈 목록, 상태, 의존관계	/actuator/beans
스레드 덤프	스레드 상태/스택 추적	/actuator/threaddump
로그 레벨 동적 변경	런타임에 로그 레벨 변경	/actuator/loggers
캐시 정보	Spring Cache 정보	/actuator/caches
HTTP 트레이스	최근 HTTP 요청/응답 추적	/actuator/httptrace
Mappings	RequestMapping 정보	/actuator/mappings

[샘플코드] 자동 설정 & @Value

```
# application.yml
shop:
  welcome-message: "환영합니다"

// ---- Java ----
@Service
public class WelcomeService {

    @Value("${shop.welcome-message}")    // 설정값 주입
    private String message;

    @Value("${server.port:8080}")        // 기본값 8080
    private int port;
}
// spring-boot-starter-web 만 추가하면 내장 Tomcat/Jackson 자동 설정
```

[핵심 요약] 자동 설정(Auto-Configuration)

핵심 포인트	반드시 숙지할 내용
자동 설정	" 설정보다 관례 " 철학 — Starter 의존성만으로 관련 빈 자동 구성
설정 관리	application.yml / properties 로 환경 정보 관리
값 주입	@Value(단일) vs @ConfigurationProperties (구조적)
환경별 설정	런타임 환경(dev/test/prod)별 설정 파일 분리
Actuator	spring-boot-starter-actuator로 상태·모니터링 정보 노출

로그 관리

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

애플리케이션 로그 관리의 중요성

■ 1. 문제 진단 및 원인 분석

- 장애 발생 시 로그를 통해 언제, 어떤 오류가 발생했는지 빠르게 파악할 수 있다.
- Stack Trace, 에러 메시지 등 상세 정보를 확인하여 근본 원인을 추적할 수 있다.
- 버그 재현이 어려운 경우에도 로그를 통해 상황을 복원할 수 있다.

■ 2. 시스템 모니터링 및 운영 효율화

- 애플리케이션의 상태, 성능, 사용량 등을 실시간으로 모니터링할 수 있다.
- 요청 처리 시간, 에러 발생률, 리소스 사용 현황 등을 수집하여 운영 지표로 삼을 수 있다.

■ 3. 보안 및 컴플라이언스 대응

- 접근 기록, 실패한 로그인 시도 등 보안 이벤트 기록을 남겨 침해 사고 대응 및 추적에 활용할 수 있다.
- 개인정보 처리, 트랜잭션 기록 등 컴플라이언스(준수) 목적에도 필요하다.

■ 4. 사용자 행동 및 서비스 개선

- 사용자 요청, API 호출 패턴 등 사용자 행동 분석 데이터를 확보할 수 있다.
- 인기 기능, 오류가 많은 기능 등 서비스 개선에 필요한 인사이트를 도출할 수 있다.

■ 5. 감사 및 법적 증빙

- 중요한 비즈니스 이벤트, 트랜잭션 로그는 감사 및 법적 분쟁 시 증빙 자료로 활용된다

스프링부트의 로그 관리 구조

- Spring Boot는 로그 시스템을 자동 구성 해주어 초기 설정만 하면 별도 복잡한 작업 없이 로그를 관리
 - 기본적으로 **SLF4J(Simple Logging Facade for Java)**와 Logback이 함께 동작
 - 개발자는 SLF4J 인터페이스만 사용하면, 내부적으로 Logback이 로그를 처리
-
- ❖ **SLF4J**는 자바 생태계에서 가장 널리 사용되는 로그 파사드(Facade) 라이브러리
 - 다양한 자바 로깅 프레임워크(Logback, Log4j, java.util.logging 등)와 일관된 API로 통합해서 사용할 수 있도록 해주는 일종의 인터페이스
 - 직접 로그를 기록하지 않고, 내부적으로 실제 로깅 구현체에 위임(delegate)
-
- ❖ **참고** : 2021년 12월 Log4j2에서 심각한 보안 취약점(CVE-2021-44228, "Log4Shell")이 공개되어 전 세계적으로 큰 이슈가 됨.

로그 파사드(Log Facade)

- 로그 파사드(Facade)란 여러 로깅 프레임워크(Logback, Log4j, java.util.logging 등)를 일관된 API로 감싸서 사용할 수 있게 해 주는 **중간 계층**
- 로깅 구현체가 달라져도(예: Logback → Log4j2) 코드 변경 없이 파사드 인터페이스만 사용하면 됨
- 라이브러리 간 로깅 의존성 충돌을 방지

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class SampleService {
    private static final Logger log = LoggerFactory.getLogger(SampleService.class);

    public void doSomething() {
        log.info("작업 시작");
        log.debug("상세 정보 출력");
        log.error("에러 발생!", new RuntimeException());
    }
}
```

❖ Spring Boot 기본 로깅 구현체와 교체 방법

- **기본(Logback)** : spring-boot-starter-logging에 포함
- **변경(Log4j2)** : spring-boot-starter-logging 의존성 제거후 spring-boot-starter-log4j2 추가

[참고] Lombok @Slf4j 활용

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class SampleService {
    private static final Logger log = LoggerFactory.getLogger(SampleService.class);

    public void doSomething() {
        log.info("작업 시작");
        log.debug("상세 정보 출력");
        log.error("에러 발생!", new RuntimeException());
    }
}
```

```
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class SampleService {

    public void doSomething() {
        log.info("작업 시작");
        log.debug("상세 정보 출력");
        log.error("에러 발생!", new RuntimeException());
    }
}
```

❖ **@Slf4j** 어노테이션은 자동으로 **log** 객체를 생성해 줌

[샘플코드] 로그 관리 (@Slf4j)

```
@Slf4j
@Service
public class OrderService {
    public void order(Long id) {
        log.trace("가장 상세");
        log.debug("디버깅 id={}", id);
        log.info("주문 처리 시작 id={}", id);    // 운영 기본 레벨
        log.warn("재고 부족 경고");
        log.error("주문 실패", new RuntimeException());
    }
}
```

```
# application.yml (패키지별 로그 레벨)
logging:
  level:
    com.sk.skala: debug
```

[핵심 요약] 로그 관리

핵심 포인트	반드시 숙지할 내용
로그의 중요성	문제 진단·원인 분석의 핵심 근거
자동 구성	Spring Boot가 로그 시스템을 자동 구성 (초기 설정만으로 사용)
로그 파사드	SLF4J (파사드) + Logback (구현체) 조합
@Slf4j 활용	Lombok @Slf4j로 로거 자동 생성, 레벨별 기록

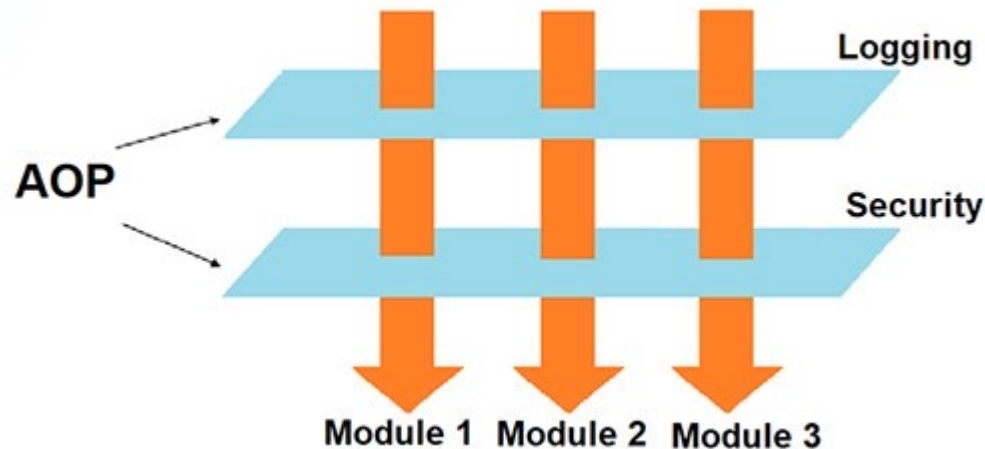


Aspect Oriented Programming

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Java의 AOP(Aspect-Oriented Programming)

- **AOP(관점 지향 프로그래밍)는 OOP(객체지향 프로그래밍)의 한계를 보완하는 프로그래밍 패러다임**
 - OOP는 주로 '기능' 단위로 클래스를 구현
 - AOP는 '공통 관심사(Aspect)'를 모듈화 하여 코드 중복을 줄이고 유지보수가 쉽도록 구현
- **대표적인 공통 관심사(Aspect) 예시**
 - 로깅(Logging)
 - 트랜잭션 처리(Transaction)
 - 보안(Security)
 - 성능 측정(Profiling)



Spring Boot의 AOP

- 별도 설정 없이 **spring-boot-starter-aop** 의존성만 추가하면 바로 사용
- 선언적 **AOP** 지원: @Aspect 어노테이션 기반으로 Aspect 작성, Pointcut, Advice(@Before, @After, @Around 등) 선언 가능

키워드	역할	비고
Aspect	공통 관심사를 모아둔 클래스, Advice + Pointcut의 결합체	@Aspect @Component public class LogAspect { }
Advice	언제 공통 로직을 실행할 지 정의, 실제 실행되는 코드	@Before @After @AfterReturning @AfterThrowing @Around
Joint Point	Advice가 적용될 수 있는 지점, Spring AOP에서는 메서드 실행시점만 지원	메서드 호출 전/후, 예외 발생 시
Pointcut	어디에 Advice를 적용할지 정의하는 표현식, Join Point 중에서 실제 적용 대상만 선택	execution(* com.example.service.*(..))
Target	실제 비즈니스 로직을 담고 있는 객체	
Proxy	Target 객체를 감싸는 대리 객체	
Weaving	Advice를 Target에 적용하는 과정	

Spring Boot의 AOP

- 별도 설정 없이 **spring-boot-starter-aop** 의존성만 추가하면 바로 사용
- 선언적 **AOP** 지원: @Aspect 어노테이션 기반으로 Aspect 작성, Pointcut, Advice(@Before, @After, @Around 등) 선언 가능

✅ Advice (언제 공통 로직을 실행할 지 정의)

어노테이션	역할
@Aspect	해당 클래스가 Aspect임을 선언
@Before	메서드 실행 전에 Advice 실행
@After	메서드 실행 후에 Advice 실행
@AfterReturning	메서드 정상 실행 후 Advice 실행
@AfterThrowing	예외 발생 후 Advice 실행
@Around	메서드 실행 전후 모두 Advice 실행 (실무에서 가장 많이 사용됨)

Pointcut 표현식



- **execution(* com.example.user.*.*(..))**
user 패키지 아래의 모든 클래스의 전체 메소드를 대상이며, 모든 반환값 유형 (void, String, int ...)
- **execution(* com.example.user.UserService.*(..))**
UserService 클래스의 전체 메소드를 대상
- **execution(* com.example.user.UserService.find*(..))**
UserService 클래스에서 find 로 시작하는 메소드를 대상
- **execution(String com.example.user.UserService.*(..))**
UserService 클래스에서 반환값이 String 인 메소드를 대상

[참고] @Around 구현 예시

- 메서드 실행 전/후, 예외 발생 여부 등 "전체 실행 흐름"을 모두 제어

```
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.example.service.*.*(..))")
    public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("메서드 실행 전");
        Object result = joinPoint.proceed(); // 실제 메서드 실행
        System.out.println("메서드 실행 후");
        return result;
    }
}
```

- **@Aspect** : Spring AOP는 @Aspect가 붙은 클래스를 읽어 Advice(부가기능 코드)와 Pointcut(적용대상) 정보를 스캔
- **@Component** : Spring의 Bean(Component)으로 등록, @Aspect 단독으로 Bean이 아니므로 동작하지 않음.
- **@Around("execution(* com.example.service.*.*(..))")** : logAround() 메서드를 Advice(부가기능 코드)로 지정하고, Pointcut 표현식에 해당하는 모든 메서드 실행에 적용하라는 의미
 - **"execution(* com.example.service.*.*(..))"**: 메서드 실행 지점(Pointcut) 지정
 - *****: 반환 타입 무관
 - **com.example.service.***: com.example.service 패키지 내 모든 클래스
 - **.*(..)**: 모든 메서드, 모든 파라미터
 - 동작: com.example.service 패키지의 모든 메서드 실행시 logAround()가 먼저 실행 → joinPoint.proceed() 호출 시 실제 원본 메서드가 실행

[실습] AOP 기반 API 요청/응답 로깅

- aop/ApiLoggingAspect.java 클래스를 구현하여 모든 API 요청에 대해 요청 파라미터, 응답 데이터, 처리 시간(ms)을 로깅
- 모든 Controller의 public 메서드를 대상으로 AOP 적용 : @Around Advice를 사용하여 메서드 실행 전 파라미터 로깅, 메서드 실행 후 응답/처리시간 로깅

```
<!-- pom.xml 파일에 AOP 의존성 추가 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

[코드] aop/ApiLoggingAspect.java 구현

```
@Aspect
@Component
public class ApiLoggingAspect {

    private static final Logger log = LoggerFactory.getLogger(ApiLoggingAspect.class);

    // 모든 RestController의 public 메서드에 적용
    @Around("execution(public * com.skala.basic.controller..*Controller.*(..))")
    public Object logApiResponse(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();

        // 1. 요청 파라미터 로깅
        String methodName = joinPoint.getSignature().toShortString();
        Object[] args = joinPoint.getArgs();
        log.info("[API REQUEST] {} | Params: {}", methodName, args);

        // 2. 실제 메서드 실행
        Object result = joinPoint.proceed();

        long end = System.currentTimeMillis();
        long duration = end - start;

        // 3. 응답, 처리시간 로깅
        log.info("[API RESPONSE] {} | Result: {} | Duration: {}ms", methodName, result, duration);

        return result;
    }
}
```

[참고] 스프링의 프록시(Proxy)

- 스프링 프록시는 스프링 컨테이너가 Bean(원본 객체) 대신 프록시 객체를 생성하여 외부에서는 원본처럼 보이지만, 부가기능(AOP, 트랜잭션 등)을 자동으로 추가할 수 있도록 만들어 주는 것.
- 동작 방식:
 - ① Bean 등록 시점에 프록시 객체가 실제 객체 대신 컨테이너에 등록됨
 - ② 클라이언트가 Bean을 사용할 때 프록시가 먼저 실행
 - ③ 전처리(Advice) 실행
 - ④ 실제 원본 메서드 호출
 - ⑤ 후처리(Advice) 실행
 - ⑥ 결과 반환

[참고] Java의 Reflection과 스프링의 활용

- Java 리플렉션은 "실행 중에 클래스/메서드/필드 정보를 읽거나 조작할 수 있게 해주는 기술"
- Spring은 거의 모든 핵심 기능(빈 생성, DI, 어노테이션 처리, AOP, 컨트롤러 매핑 등)에 리플렉션을 적극

// Java의 클래스 정보 얻기

```
Class<?> clazz = Class.forName( " com.example.User " ); // 클래스 이름으로 Class 객체 획득
```

// Java의 메서드 동적 호출

```
Method method = clazz.getMethod("setName", String.class);
method.invoke(obj, "홍길동"); // obj의 setName("홍길동") 실행
```

// 스프링의 DI(의존성 주입) - 빈 생성/주입 - 스프링 내부 코드 (간단화)

```
Class<?> clazz = ...; // 등록된 빈의 클래스 타입
```

```
Object bean = clazz.getDeclaredConstructor().newInstance(); // 리플렉션으로 인스턴스 생성
```

// 스프링의 @Autowired, @Value 어노테이션 처리

```
Field[] fields = clazz.getDeclaredFields();
```

```
for (Field field : fields) {
```

```
    if (field.isAnnotationPresent(Autowired.class)) {
```

```
        field.setAccessible(true);
```

```
        field.set(bean, 의존성객체);
```

```
    }
```

```
}
```

[샘플코드] AOP

```
// build.gradle: implementation 'org...:spring-boot-starter-aop'
@Aspect
@Component
@Slf4j
public class ApiLoggingAspect {

    @Around("execution(* com.sk.skala..controller..*(..))") // 포인트컷
    public Object logApi(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();           // 원본 메서드 실행
        long time = System.currentTimeMillis() - start;
        log.info("{} - {}ms", pjp.getSignature(), time);
        return result;
    }
}
```

[핵심 요약] AOP(관점 지향 프로그래밍)

핵심 포인트	반드시 숙지할 내용
AOP란	로깅·트랜잭션 등 공통 관심사(횡단 관심사) 를 분리 — OOP 보완
핵심 용어	Aspect · Advice · Pointcut · JoinPoint
Advice/Pointcut	@Around 등 Advice, execution(...) Pointcut 표현식으로 적용 지점 지정
동작 원리	스프링 프록시(Proxy) 가 원본 빈을 감싸 부가기능 수행
적용	spring-boot-starter-aop 의존성만 추가하면 사용

JPA (Java Persistence API)

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

JPA(Java Persistence API)

- 관계형 데이터베이스(RDBMS)에 데이터를 저장하고, 조회하고, 수정하고, 삭제(CRUD)할 수 있도록 표준화한 Java의 ORM(Object-Relational Mapping) 프레임워크
 - **ORM(Object-Relational Mapping):** 객체(자바 클래스)와 데이터베이스 테이블을 매핑(연결)하는 기술로 개발자는 SQL이 아닌, 자바 코드(객체)로 DB 작업이 가능
 - **표준 API:** JPA는 "인터페이스"와 "어노테이션"만 제공하고 Hibernate, EclipseLink 등의 구현체가 별도로 존재

[참고] JPA vs JDBC

- JDBC: Java Database Connectivity의 약자로, 자바에서 데이터베이스에 접근하기 위한 표준 API

구분	JPA (Java Persistence API)	JDBC (Java Database Connectivity)
목적	자바 객체와 DB 테이블 매핑(ORM)	자바에서 SQL을 직접 실행하여 DB와 연동
코딩 방식	객체지향적(Entity/Repository/Service 계층)SQL 최소화	SQL을 직접 작성Connection/Statement/ResultSet 사용
자동화 수준	CRUD, 트랜잭션, 매핑, 변경감지 등 많은 부분 자동화	모든 DB 작업을 개발자가 직접 처리
SQL 작성	메서드 자동 생성, @Query를 통한 쿼리 작성 가능	모든 쿼리를 직접 작성
트랜잭션	@Transactional로 선언적 관리	Connection 객체로 직접 트랜잭션 제어
예외 처리	DataAccessException 등 추상화된 예외 사용	SQLException 사용, 직접 예외 처리 필요
데이터 매핑	객체(Entity)와 테이블 자동 매핑	ResultSet에서 직접 필드 추출, 수동 매핑
학습 난이도	개념(ORM, 엔티티, 영속성 컨텍스트 등) 필요	JDBC와 SQL만 알면 바로 사용 가능
성능 최적화	캐시, 배치 처리 등 별도 튜닝 필요	SQL 및 JDBC 커넥션 풀 등 직접 튜닝
대표 사용 예	Spring Data JPA, Hibernate	JdbcTemplate, MyBatis, 순수 JDBC 코드
커넥션 관리	스프링이 자동 관리, 커넥션 풀 연동	직접 관리(try-finally 등으로 close 필수)

JPA를 위한 DB 정보 구성

- 스프링부트에서 JPA 및 데이터베이스 연결 정보는 application.yml에 설정

```
spring:
  datasource:
    url: jdbc:h2:mem:skala-stock
    driver-class-name: org.h2.Driver
    username: sa
    password:
  h2:
    console:
      enabled: true
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

- ❖ ddl-auto: update, create, create-drop, validate 등
- ❖ h2.console 기본 path: /h2-console

JPA를 위한 DB 정보 구성

- 스프링부트에서 JPA 및 데이터베이스 연결 정보는 application.yml에 설정

기본 설정

```
spring:
  datasource:
    url: jdbc:h2:mem:skala-stock
    driver-class-name: org.h2.Driver
    username: sa
    password:
  h2:
    console:
      enabled: true
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

❖ **ddl-auto: update, create, create-drop, validate 등**

❖ **H2 console 기본 path: /h2-console**

HikariCP 튜닝 (옵션)

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2
      connection-timeout: 30000
      idle-timeout: 600000
```

❖ Spring Boot는 2.0 버전부터 기본 데이터베이스 커넥션 풀로

HikariCP를 사용 - 세밀한 커넥션 풀 설정 필요 시 hikari 속성으로 조정 가능

[실습] H2 DB 접속 정보 설정

- 구성 정보 파일 application.yml 에 H2 DB 접속 정보 설정
- H2 Console (/h2-console)을 통해 접속 확인

[코드] resources/application.yml - H2 DB 설정

```
spring:
  datasource:
    url: jdbc:h2:mem:skala-stock
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: update
  h2:
    console:
      enabled: true
```

← → ↻ ⓘ localhost:8080/h2-console/login.jsp?jsessionid=6f

☰ | 🌐 SK OA VDI 🌐 Google ⓘ 홈 | Udemy Business 🌐 ChatGPT

English ▾ Preferences Tools Help

Login

Saved Settings: Generic H2 (Embedded) ▾

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:mem:skala-stock

User Name: sa

Password:

Connect Test Connection

H2 콘솔 (/h2-console) 확인

JPA 주요 어노테이션

어노테이션	목적/용도 (상세 설명)
@Entity	클래스를 JPA가 관리하는 엔티티로 지정 JPA는 이 클래스를 테이블에 매핑해서 CRUD 등 데이터베이스 작업을 처리할 수 있게 됨.
@Table	엔티티와 매핑할 데이터베이스 테이블의 이름·스키마 지정 클래스명과 실제 테이블명이 다르거나, DB 스키마를 지정해야 할 때 사용
@Id	기본키(Primary Key)로 사용할 필드 지정 각 엔티티마다 식별자가 필요하고 이를 통해 객체의 고유성을 보장
@GeneratedValue	기본키 값을 자동 생성하는 전략 설정 DB의 auto-increment나 시퀀스 등을 활용해 PK를 자동 할당하고 싶을 때 사용
@Column	클래스 필드와 DB 컬럼을 세부적으로 매핑 컬럼명, 길이, null 허용, 유니크 등 제약조건 지정(필드가 DB 컬럼명과 다를 때도 사용)
@Enumerated	Enum 타입 필드를 DB에 저장할 때 방식 지정 값 자체를 숫자로 저장할지(ORDINAL), 문자열로 저장할지(String) 선택
@Lob	대용량 데이터(텍스트/이미지 등)를 DB에 저장 일반적인 문자열이 아닌, 긴 텍스트(CLOB)나 바이너리 데이터(BLOB) 매핑
@Transient	DB에 저장하지 않는 필드 선언 계산된 값, 일시적 데이터 등 DB 컬럼으로 만들 필요 없는 필드에 사용
@{One/Many}To{One/Many}	엔티티 간의 관계를 명시적으로 선언 관계형 DB의 외래키와 매핑하며, 객체지향적으로 연관관계를 관리(1:1, 1:N, N:1, N:N 등)
@JoinColumn	관계 매핑시 외래키 컬럼을 직접 지정 관계 엔티티의 어떤 컬럼이 외래키인지 명확히 표기할 때 사용
@Embedded	내장 값 타입을 엔티티에 포함 별도의 클래스를 필드로 사용하여, 컬럼을 그룹화해서 재사용
@Embeddable	내장 값 타입 클래스 지정 @Embedded로 포함될 클래스에 선언 (여러 엔티티에서 재사용 가능)

JPA 어노테이션 - @Entity

- 해당 클래스를 JPA가 관리하는 엔티티(테이블)로 지정 - 자바 클래스와 데이터베이스의 테이블을 1:1로 매핑하는 역할
- @Entity가 선언된 클래스는 JPA의 CRUD 대상이 됨.

```
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity // 이 클래스를 JPA가 관리하는 엔티티로 지정
@Table(name = "stocks") // (선택) stocks 테이블에 매핑
public class Stock {

    @Id
    private String ticker; // 주식의 고유 코드 (ex: "AAPL")

    private String name; // 주식 이름 (ex: "Apple Inc.")

    private String market; // 시장 구분 (ex: "NASDAQ")

    // getter, setter 등 생략
}
```

- @Entity가 붙은 클래스는 반드시 기본 생성자(파라미터 없는 생성자)가 필요
- 클래스는 final, abstract, interface, inner class에는 설정할 수 없음.
- @Entity만 선언하면 클래스 이름이 그대로 테이블명으로 매핑
- 실제 운영에서는 테이블명과 클래스명이 다를 수 있으니, 보통 @Table을 함께 사용해서 명시적으로 테이블명을 지정하는 것이 안전

JPA 어노테이션 - @Table

- @Entity로 지정한 클래스가 어느 DB 테이블과 매핑될지 명시
- 주로 테이블 이름이 클래스명과 다를 때 사용하거나, 스키마 지정 등 추가 설정이 필요할 때 사용(생략 가능)

속성	설명	예시
name	매핑할 DB 테이블 이름을 지정	@Table(name = "stocks")
schema	사용할 DB 스키마 지정 (DBMS가 지원할 때만)	@Table(name = "stocks", schema = "skala")
catalog	DB 카탈로그 지정 (거의 사용하지 않음)	@Table(name = "stocks", catalog = "stockdb")
uniqueConstraints	유니크 제약조건 지정 (복합 유니크 키 등)	@UniqueConstraint(columnNames = {"ticker", "market"})

```
@Entity
@Table(
    name = "stocks",
    schema = "skala",
    uniqueConstraints = @UniqueConstraint(columnNames = {"ticker",
"market"})
)
public class Stock {
    @Id
    private String ticker;
    private String name;
    private String market;
}
```

- schema 속성은 DB 스키마(예: skala.stocks)에 매핑이 필요할 때 사용.
- uniqueConstraints는 테이블 차원에서 유니크 제약조건을 부여할 때 사용 - (예: ticker + market 조합이 중복되면 안 되는 경우)

❖ 단일 컬럼 유니크와 테이블 단위 복합 유니크 제약 조건의 차이

- @Column(unique = true) : 한 컬럼이 중복되지 않아야 할 때 사용.
- @Table(uniqueConstraints = ...) : 두 개 이상 컬럼 조합이 중복되지 않도록 보장하고 싶을 때 사용.

JPA 어노테이션 - @Id

- 엔티티의 "기본키(Primary Key)" 필드를 지정
- JPA는 모든 엔티티에 반드시 하나 이상의 @Id가 필요

```
// 기본키가 하나인 경우 (단일 필드)
// ticker 필드가 stocks 테이블의 Primary Key
```

```
@Entity
@Table(name = "stocks")
public class Stock {
    @Id
    private String ticker; // 예: "AAPL"

    private String name;
    private String market;
}
```

```
// 자동 생성되는 숫자 기본키
// MySQL 등에서 auto_increment로 PK가 자동 할당
```

```
import javax.persistence.*;

@Entity
@Table(name = "transactions")
public class Transaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // 자동 증가되는 기본키

    private String ticker; // 주식 코드
    private int quantity; // 거래 수량
    private double price; // 거래 가격
    private LocalDateTime date; // 거래 일시
}
```

❖ 두 개 이상의 컬럼 조합이 고유해야 하는 경우 **복합키(Composite Key)**로 구성

JPA 어노테이션 - @Id (복합키)

- 주로 두 개 이상의 컬럼 조합이 고유해야 하는 경우 사용

@Embeddable

```
public class UserStockId implements Serializable {  
    private Long userId;  
    private String ticker;  
    // equals, hashCode 구현 필수!  
}
```

@Entity

```
@Table(name = "user_stocks")  
public class UserStock {  
    @EmbeddedId  
    private UserStockId id;  
  
    private int quantity;  
}
```

- @Embeddable 클래스로 복합키 타입 정의
- @EmbeddedId로 엔티티에 복합키 사용

JPA 어노테이션 - @GeneratedValue

- 기본키(PK) 값을 자동으로 생성하는 전략을 지정할 때 사용 - 반드시 @Id와 함께 사용
- 데이터베이스의 auto_increment, 시퀀스, 별도 테이블 등 다양한 자동 생성 방식을 지원
- 주요 속성
 - strategy : PK 자동 생성 방식 지정
 - generator : 시퀀스/테이블 방식일 때 커스텀 생성기 이름 지정

[strategy : GenerationType.IDENTITY]

- AUTO: DB에 맞는 전략을 JPA가 알아서 선택 (기본값)
- IDENTITY: MySQL의 auto_increment처럼, DB가 PK를 자동 증가
- SEQUENCE: 오라클, PostgreSQL 등에서 시퀀스 객체 사용

// MySQL, MariaDB 기준 - DB의 auto_increment 사용

```
@Entity
@Table(name = "transactions")
public class Transaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // 자동 생성 PK

    private String ticker;
    private int quantity;
    private double price;
    private LocalDateTime tradedAt;
}
```

// Oracle, PostgreSQL 등 - 시퀀스 전략 사용

```
@Entity
public class TradeHistory {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "trade_seq_gen")
    @SequenceGenerator(name = "trade_seq_gen", sequenceName = "trade_seq", allocationSize = 1)
    private Long id;

    // 기타 필드
}
```

JPA 어노테이션 - @Column

- 엔티티 필드를 데이터베이스의 컬럼과 매핑할 때 사용
- 컬럼 이름, 길이, null 허용, 유니크 여부, 숫자 자릿수 등 DB 컬럼 속성을 세밀하게 조정 가능

속성	설명	예시
name	컬럼명 지정	@Column(name = "user_name")
nullable	null 허용 여부 (기본값: true)	@Column(nullable = false)
length	문자 길이(문자열 필드에만 사용)	@Column(length = 50)
unique	유니크 제약조건 부여(단일 컬럼일 때)	@Column(unique = true)
updatable	UPDATE 쿼리에서 수정 가능 여부	@Column(updatable = false)
insertable	INSERT 쿼리에서 포함 여부	@Column(insertable = false)
precision, scale	소수점 숫자 자리수 지정(숫자형만)	@Column(precision = 10, scale = 2)

@Column 속성 매핑

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_name", nullable = false, length = 30)
    private String name; // DB 컬럼: user_name, NOT NULL, 최대 30자

    @Column(unique = true, length = 50)
    private String email; // DB 컬럼: email, UNIQUE, 최대 50자

    @Column(length = 100)
    private String password; // 비밀번호 컬럼, 길이 100자 제한

    @Column(updatable = false)
    private LocalDateTime createdAt; // 최초 생성일(수정 불가)
}
```

@Column 속성 매핑 - 숫자 자리 지정

```
@Entity
public class Transaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(precision = 12, scale = 4)
    private BigDecimal price; // 최대 9999999999.9999까지 표현 가능
}
```


JPA 어노테이션 - @Enumerated

- Enum(열거형) 타입 필드를 데이터베이스에 어떻게 저장할지 지정할 때 사용
- 자바의 enum 타입은 RDB에는 직접 저장할 수 없으므로, 숫자(ORDINAL)나 문자열(String)로 변환하여 저장
- 실무에서는 String 방식을 강력 권장

```
public enum StockStatus {  
    ACTIVE,  
    INACTIVE  
}  
  
@Entity  
public class Stock {  
    @Id  
    private String ticker;  
  
    @Enumerated(EnumType.STRING)  
    @Column(length = 10)  
    private StockStatus status; // "ACTIVE" 또는 "INACTIVE"로 저장  
}
```

JPA 어노테이션 - @Lob

- 대용량 데이터(텍스트 또는 바이너리 데이터)를 데이터베이스에 저장할 때 사용 - 필드 타입에 따라 자동으로 CLOB 또는 BLOB으로 매핑
 - **CLOB(Character Large Object)**: 대용량 텍스트 (ex. 긴 설명, 본문 등)
 - **BLOB(Binary Large Object)**: 대용량 바이너리 (ex. 이미지, 파일 등)

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @Lob
    private String description; // @Lob + String 타입 → CLOB(대용량 텍스트)로 매핑됨

    @Lob
    private byte[] profileImage; // @Lob + byte[] 타입 → BLOB(대용량 바이너리)로 매핑됨
}
```

❖ @Lob vs @Column(columnDefinition = "TEXT")

- @Lob은 JPA 표준 방식, DB 독립적 → 유지보수·확장성에 유리
- columnDefinition은 DB 종속적, 유연성 적음 → 특별한 경우에만 사용

JPA 어노테이션 - @Transient

- 엔티티 필드 중에서 DB 컬럼과 매핑하지 않을 필드에 사용 - 해당 필드는 DB 테이블에 컬럼으로 생성되지 않으며, JPA의 영속화(저장/조회) 대상이 아님을 명시
- 계산값, 임시 데이터, 화면 출력 전용 속성 등에 활용

```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    private String password;

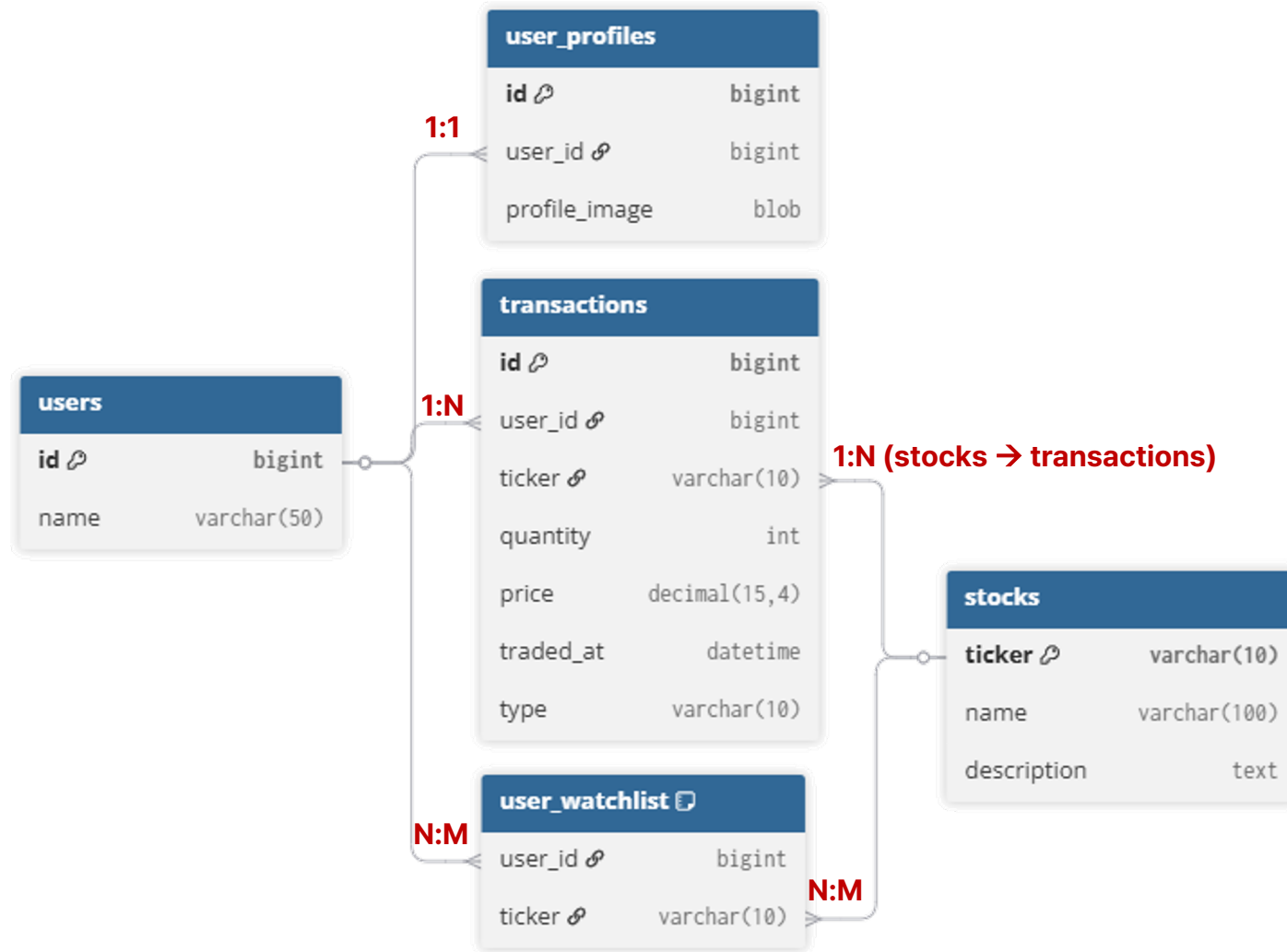
    @Transient
    private String passwordConfirm; // 입력폼 전용, DB 저장 X
}
```

JPA 어노테이션 - 관계 매핑

- 객체 간의 관계를 데이터베이스 테이블의 외래 키(FK)와 연결하는 방법을 정의
- 다중성(1:1, 1:N, N:1, N:M)과 방향성(단방향, 양방향)을 조합하여 설정

어노테이션	관계	대표 예시 (stock-api)
@ManyToOne	N:1	여러 거래 → 하나의 주식
@OneToMany	1:N	하나의 사용자 → 여러 거래
@OneToOne	1:1	유저 ↔ 프로필
@ManyToMany	N:M	유저 ↔ 관심종목(주식)
@JoinColumn	외래키 지정	거래에 user_id, ticker 등
@JoinTable	중간테이블	user_watchlist(user_id + ticker)

관계 매핑 예시 - 데이터 모델



관계 매핑 예시 – Users.java

```

@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "user")
    private List<Transaction> transactions = new ArrayList<>();

    @OneToOne(mappedBy = "user")
    private UserProfile profile;

    @ManyToMany
    @JoinTable(
        name = "user_watchlist",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "ticker")
    )
    private List<Stock> watchList = new ArrayList<>();
}

```

- **mappedBy**는 "이 관계의 주인은 반대편 엔티티의 'user' 필드"라는 뜻 :
"외래키/연관관계의 소유자는 상대방" 임을 선언

- **@JoinTable**은 두 테이블의 다대다 관계를 위한 중간 테이블(**user_watchlist**)을 정의
 - joinColumns: 이 엔티티(User)에서 중간 테이블로 연결되는 외래키(user_id)
 - inverseJoinColumns: 반대 엔티티(Stock)에서 중간 테이블로 연결되는 외래키(ticker)
 - 결과적으로 user_watchlist(user_id, ticker)가 중간테이블로 생성
 - 한 User는 여러 Stock을 watchList로, 한 Stock도 여러 User의 watchList에 포함 가능

관계 매핑 예시 – Stocks.java

```
@Entity
@Table(name = "stocks")
public class Stock {
    @Id
    private String ticker;

    private String name;

    @Lob
    private String description;

    @OneToMany(mappedBy = "stock")
    private List<Transaction> transactions = new ArrayList<>();

    @ManyToMany(mappedBy = "watchList")
    private List<User> fans = new ArrayList<>();
}
```

관계 매핑 예시 – Transactions.java

```
@Entity
@Table(name = "transactions")
public class Transaction {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user; // 거래한 유저

    @ManyToOne
    @JoinColumn(name = "ticker")
    private Stock stock; // 거래한 주식

    private int quantity;

    @Column(precision = 15, scale = 4)
    private BigDecimal price;

    private LocalDateTime tradedAt;

    @Enumerated(EnumType.STRING)
    @Column(length = 10)
    private TransactionType type;
}
```

```
public enum TransactionType {
    BUY, SELL
}
```


관계 매핑 예시 – UserProfile.java

```
@Entity
@Table(name = "user_profiles")
public class UserProfile {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    @JoinColumn(name = "user_id", unique = true)
    private User user;

    @Lob
    private byte[] profileImage;
}
```

- **@JoinColumn**은 관계의 주인이 되는 엔티티에서 외래키 컬럼명을 직접 지정
 - name = "user_id": 프로필 테이블의 user_id 컬럼이 users.id를 참조하는 외래키임을 지정
 - unique = true: user_id 값이 프로필 테이블에서 유일해야 함(한 유저에 하나의 프로필만)
 - DB에는 user_profiles 테이블에 user_id 컬럼이 생성되고, 유니크 제약조건이 설정됨

관계 매핑 예시 – 관계 요약

관계	DBML	JPA 매핑
User-Transaction	users.id → transactions.user_id (1:N)	User: @OneToMany(mappedBy="user")
		Transaction: @ManyToOne(user_id)
Stock-Transaction	stocks.ticker → transactions.ticker(N:1)	Stock: @OneToMany(mappedBy="stock")
		Transaction: @ManyToOne(ticker)
User-Profile	users.id → user_profiles.user_id (1:1)	User: @OneToOne(mappedBy="user")
		UserProfile: @OneToOne + @JoinColumn
User-Stock(관심종목)	user_watchlist (user_id, ticker) (N:M)	User: @ManyToMany + @JoinTable
		Stock: @ManyToMany(mappedBy="watchList")

JPA 어노테이션 - @Embedded와 @Embeddable

- @Embeddable : "값 타입(Value Type)" 객체를 정의할 때 사용
 - "독립적인 엔티티"가 아니라, 다른 엔티티에 포함되어 함께 저장되는 복합 값 객체임을 표시
 - DB에는 별도의 테이블 없이, 엔티티 테이블의 컬럼으로 녹아 들어감
- @Embedded : 엔티티에서 값 타입(즉, @Embeddable로 선언된 클래스)을 필드로 사용할 때 설정
 - 실제로는 엔티티 테이블에 이 값 타입의 각 필드가 컬럼으로 생성됨

```
@Embeddable
public class Address {
    private String city;
    private String street;
    private String zipcode;

    // getter/setter 등
}
```

```
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;

    private String name;

    @Embedded
    private Address address; // User 테이블에 city, street, zipcode 생성
}
```

[실습] Course 엔터티 생성

- table/Course.java 클래스를 구현하여 JPA에서 사용할 수 있는 엔터티로 지정
- Long 형식의 id는 기본키로 자동 증가 전략 설정
- Name, topic, description: 모두 String 타입으로 필드 구조를 단순화

- ❖ JPA에서는 엔티티에 기본 생성자가 반드시 필요
- ❖ 객체 생성 편의를 위해 모든 필드를 파라미터로 갖는 생성자도 추가

[코드] table/Course.java

```
package com.skala.basic.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String topic; // 여러 개의 주제가 필요하다면 콤마(,)로 구분된 문자열로 저장

    private String description;
}
```

[샘플코드] JPA 엔티티

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // PK 자동 증가
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;
    private int price;

    @Enumerated(EnumType.STRING) // enum -> 문자열 저장
    private Category category;

    @ManyToOne(fetch = FetchType.LAZY) // N:1 관계
    @JoinColumn(name = "seller_id")
    private Customer seller;
}
```

[핵심 요약] JPA (Java Persistence API)

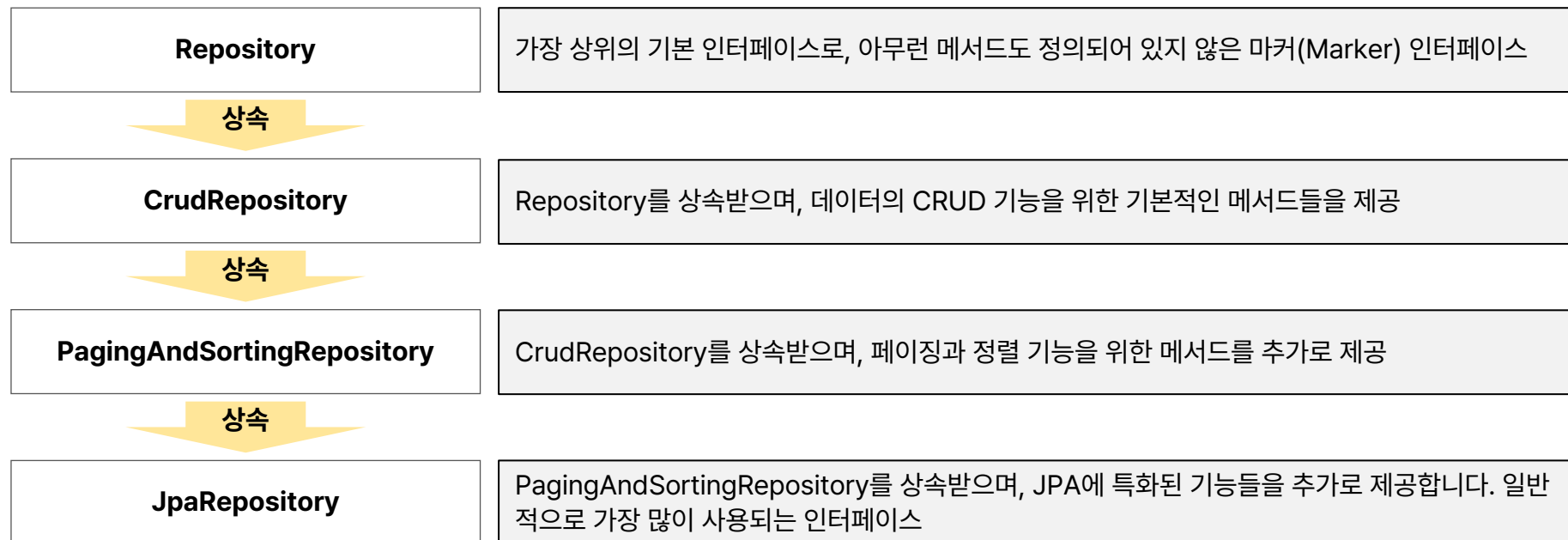
핵심 포인트	반드시 숙지할 내용
JPA	객체-테이블 매핑(ORM) 표준 — SQL 없이 CRUD 처리
JPA vs JDBC	JDBC는 직접 SQL, JPA는 엔티티 중심 의 객체지향 영속화
매핑 어노테이션	@Entity · @Table · @Id · @GeneratedValue · @Column
필드 매핑	@Enumerated · @Lob · @Transient · @Embedded/@Embeddable
관계 매핑	1:N 등 객체 관계를 외래 키(FK) 로 연결, DB 연결은 application.yml

JPA Repository

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

JPA Repository

- Spring Data JPA가 제공하는 인터페이스로 JPA를 더 쉽게 다루기 위한 추상화 계층
- 개발자가 데이터베이스에 접근하는 코드 DAO(Data Access Object)를 직접 구현하지 않고도 데이터베이스 조작을 가능하게 하는 인터페이스 기반의 프레임워크



JpaRepository 주요 메서드 – 조회

메서드 시그니처	설명	리턴값
<i>Optional<T> findById(ID id)</i>	PK로 엔티티 단건 조회	<i>Optional<T></i>
<i>List<T> findAll()</i>	모든 엔티티 조회	<i>List<T></i>
<i>List<T> findAllById(Iterable<ID> ids)</i>	여러 PK로 엔티티 조회	<i>List<T></i>
<i><S extends T> List<S> findAll(Example<S> example)</i>	Example 조건에 맞는 전체 엔티티 조회	<i>List<S></i>
<i><S extends T> Optional<S> findOne(Example<S> example)</i>	Example 조건에 맞는 엔티티 1개 조회	<i>Optional<S></i>
<i>Long count()</i>	전체 엔티티 개수 반환	<i>Long</i>
<i><S extends T> Long count(Example<S> example)</i>	Example 조건에 맞는 엔티티 개수 반환	<i>Long</i>
<i>boolean existsById(ID id)</i>	해당 PK 엔티티 존재 여부	<i>boolean</i>
<i><S extends T> boolean exists(Example<S> example)</i>	Example 조건에 맞는 엔티티 존재 여부	<i>boolean</i>
<i>T getReferenceById(ID id)</i>	프록시 엔티티 반환(JPA Lazy Loading)	<i>T</i>

JpaRepository 주요 메서드 – 저장 및 수정

메서드 시그니처	설명	리턴값
<code><S extends T> S save(S entity)</code>	엔티티 저장 또는 병합(수정)	<i>S</i>
<code><S extends T> List<S> saveAll(Iterable<S> entities)</code>	여러 엔티티 저장/수정	<i>List<S></i>
<code><S extends T> S saveAndFlush(S entity)</code>	엔티티 저장/수정 후 즉시 flush	<i>S</i>
<code>void flush()</code>	영속성 컨텍스트 변경 내용을 DB에 즉시 반영	<i>void</i>

❖ **flush()** : JPA에서 영속성 컨텍스트(Persistence Context)의 변경 내용을 즉시 DB에 반영(동기화)하도록 하는 메서드

- JPA는 기본적으로 트랜잭션이 커밋(commit) 되는 시점에 자동으로 flush를 호출하여 DB에 쿼리를 실행
- 저장, 수정, 삭제를 해도 즉시 DB 쿼리가 실행되는 것이 아니라, 영속성 컨텍스트에 반영된 뒤, 트랜잭션이 커밋될 때 한 번에 반영(Flush)되는 구조

❖ flush()를 직접 호출하는 대표적 상황

- DB에 즉시 쿼리를 반영해야 할 때 : 예를 들어, 바로 이후에 Native Query 또는 다른 JDBC 작업을 수행해야 할 때, 변경 내용을 DB에 먼저 반영해야 할 수 있음
- 트랜잭션 내에서 쿼리 동작 순서를 명확히 해야 할 때: 예를 들어, insert 후 select를 실행해야 할 때, 영속성 컨텍스트의 변경이 DB에 반영되어야 원하는 결과가 나올 때
- JPA 이벤트 리스너나 복잡한 이벤트 처리 로직: 엔티티의 상태를 DB에 반영해야만 이벤트가 정상적으로 처리되는 경우

JpaRepository 주요 메서드 – 삭제

메서드 시그니처	설명	리턴값
<code>void deleteById(ID id)</code>	PK 기준 엔티티 삭제	<code>void</code>
<code>void delete(T entity)</code>	엔티티 단건 삭제	<code>void</code>
<code>void deleteAllById(Iterable<? extends ID> ids)</code>	여러 PK 엔티티 삭제	<code>void</code>
<code>void deleteAll(Iterable<? extends T> entities)</code>	여러 엔티티 삭제	<code>void</code>
<code>void deleteAll()</code>	모든 엔티티 삭제	<code>void</code>
<code>void deleteAllInBatch()</code>	모든 엔티티 일괄(벌크) 삭제	<code>void</code>
<code>void deleteAllInBatch(Iterable<T> entities)</code>	전달받은 엔티티만 일괄(벌크) 삭제	<code>void</code>
<code>void deleteAllByIdInBatch(Iterable<ID> ids)</code>	여러 PK 엔티티만 일괄(벌크) 삭제	<code>void</code>

JpaRepository 주요 메서드 – 페이징 및 정렬

메서드 시그니처	설명	리턴값
<code>List<T> findAll(Sort sort)</code>	정렬 조건에 맞게 전체 엔티티 조회	<code>List<T></code>
<code>Page<T> findAll(Pageable pageable)</code>	페이징 조건에 맞는 엔티티 페이지 조회	<code>Page<T></code>
<code><S extends T> List<S> findAll(Example<S> example, Sort sort)</code>	Example + Sort 조건으로 엔티티 조회	<code>List<S></code>

[참고] JPA 쿼리 메서드 키워드(Keyword)

구분	키워드(메서드 내 단어)	의미 / 동작	예시 메서드명
조회	<i>find, read, get, query, search, stream, count, exists, delete, remove</i>	조회, 개수, 존재 여부, 삭제 등	<i>findByName, countByAge, existsByEmail</i>
조건	<i>By</i>	뒤에 오는 조건 필드를 지정	<i>findByName, findByAge</i>
	<i>And, Or</i>	조건 연결(AND/OR)	<i>findByNameAndAge, findByNameOrEmail</i>
	<i>Is, Equals</i>	같음(=)	<i>findByNameIs, findByNameEquals</i>
	<i>Between</i>	범위(사이)	<i>findByAgeBetween</i>
	<i>LessThan, Before</i>	미만(<)	<i>findByAgeLessThan</i>
	<i>GreaterThan, After</i>	초과(>)	<i>findByAgeGreaterThan</i>
	<i>LessThanEqual</i>	이하(<=)	<i>findByAgeLessThanEqual</i>
	<i>GreaterThanEqual</i>	이상(>=)	<i>findByAgeGreaterThanEqual</i>
	<i>IsNull, IsNotNull</i>	널/널이 아님	<i>findByEmailIsNull, findByEmailIsNotNull</i>
	<i>In, NotIn</i>	컬렉션 내 포함/불포함	<i>findByStatusIn, findByAgeNotIn</i>
	<i>StartingWith</i>	문자열로 시작하는(Like 'xxx%')	<i>findByNameStartingWith</i>
	<i>EndingWith</i>	문자열로 끝나는(Like '%xxx')	<i>findByNameEndingWith</i>
	<i>Containing, Contains</i>	포함(Like '%xxx%')	<i>findByNameContaining</i>
	<i>Like, NotLike</i>	Like, Not Like	<i>findByNameLike, findByNameNotLike</i>
	<i>True, False</i>	불리언 값	<i>findByActiveTrue</i>
정렬	<i>OrderBy</i>	정렬 조건	<i>findByNameOrderByAgeDesc</i>
페이징	<i>Pageable, Slice</i>	페이징 파라미터 사용	<i>findByName(String name, Pageable pageable)</i>

JpaRepository 사용자 정의 메서드

- JPA Repository 기본 제공 메서드나 간단한 쿼리 메서드만으로 해결하기 어려운, 복잡한 조회 조건 또는 특정 데이터 형태의 조회가 필요할 때 작성
- 여러 테이블을 조인하거나, 특정 조건에 따라 동적으로 쿼리를 만들어야 하거나, 전체 엔티티가 아닌 일부 데이터만 DTO 형태로 가져와 성능을 개선하고 싶을 때 필요

구분	특징	코드 작성 방식
쿼리 메서드	메서드 명 규칙만으로 자동 쿼리 생성 - 장점: 간단, 빠름, 실수 적음 - 단점: 복잡조건/조인 한계	<code>findByNameAndAge(String, int)</code>
@Query (JPQL)	JPQL(엔티티 기반)로 커스텀 쿼리 작성 - 장점: 조인/집계 등 복잡 쿼리 지원 - 단점: 동적쿼리 어렵, Entity 기준	<code>@Query("select u...")</code>
@Query (Native)	실제 DB SQL로 작성 - 장점: DB최적화, 고급쿼리, DB함수 사용 - 단점: DB종속, 유지보수 어려움	<code>@Query(value="SQL", nativeQuery=true)</code>
QueryDSL	자바 코드 기반 타입 안전 동적 쿼리 - 장점: 동적, 복잡 조건, 타입 안전 - 단점: 세팅 필요, 러닝커브	QueryDSL 코드

사용자 정의 메서드 – 쿼리 메서드

- 쉽고 빠르며, 실수 방지. 단순 조건, 검색, 정렬에 매우 효율적
- 조인, 복잡 논리(AND/OR 혼합), 그룹/집계, 복잡 동적 쿼리에는 적합하지 않음
- 이름이 너무 길어질 땐 @Query로 전환 고려

```
// 1. 이름과 나이로 사용자 찾기 (자동 쿼리 생성)
List<User> findByNameAndAgeGreaterThan(String name, int age);

// 2. 상태가 활성화고 이메일이 특정 값인 사용자
Optional<User> findByStatusAndEmail(UserStatus status, String email);

// 3. 생성일 기준 정렬, 페이징
Page<User> findByCreatedAtAfter(LocalDate date, Pageable pageable);
```


사용자 정의 메서드 – @Query 어노테이션

- 복잡 조건, 조인, 집계(AVG, COUNT), 부분 컬럼 추출 등에 적합
- 동적 조건이 많으면 코드가 지저분해짐, 컴파일러 타입 체크 어려움
- 반복되는 복잡 쿼리는 QueryDSL/커스텀 리포지토리로 이전 고려

```
// JPQL (엔티티 객체 대상으로 쿼리)
@Query("select u from User u where u.status = :status and u.age >= :minAge")
List<User> findActiveUsers(@Param("status") UserStatus status, @Param("minAge") int minAge);

// 조인 포함 (User, Order)
@Query("select u from User u join u.orders o where o.price > :minPrice")
List<User> findByOrderPriceGreaterThan(@Param("minPrice") int minPrice);

// 특정 컬럼만 추출
@Query("select u.name, u.email from User u where u.status = :status")
List<Object[]> findUserNamesAndEmails(@Param("status") UserStatus status);
```

```
// Native SQL (DB 테이블/컬럼명 직접 사용)
@Query(value = "SELECT * FROM users WHERE status = ?1 AND created_at > ?2", nativeQuery = true)
List<User> findActiveUsersNative(String status, LocalDateTime createdAt);

// DB 함수, 윈도우 함수, 복잡 집계 등
@Query(value = "SELECT u.name, count(*) OVER() as total FROM users u WHERE u.status = :status", nativeQuery = true)
List<Object[]> findUsersWithTotalCount(@Param("status") String status);
```

[참고] JPQL(Java Persistence Query Language)

- JPA의 객체(Entity)를 대상으로 쿼리하는 객체 지향 쿼리 언어
- SQL과 유사하지만, "테이블/컬럼"이 아니라 "엔티티/필드"를 기준으로 작성
- DB 독립적이며, JPA 구현체(Hibernate 등)가 SQL로 변환하여 실행

```
// 1. 전체 조회
@Query("select u from User u")
List<User> findAllUsers();

// 2. 조건 검색
@Query("select u from User u where u.age > :age")
List<User> findUsersOlderThan(@Param("age") int age);

// 3. 조인 (User와 Order를 조인)
@Query("select u from User u join u.orders o where o.status = :status")
List<User> findUsersByOrderStatus(@Param("status") OrderStatus status);

// 4. 집계/그룹
@Query("select u.name, count(o) from User u join u.orders o group by u.name")
List<Object[]> countOrdersByUserName();
```

[참고] JPQL 주요 함수

분류	함수명	설명/용도	예시 코드/비고
집계함수	<i>COUNT</i>	행 개수 집계	<i>select count(u) from User u</i>
	<i>SUM</i>	합계	<i>select sum(o.amount) from Order o</i>
	<i>AVG</i>	평균	<i>select avg(u.age) from User u</i>
	<i>MAX</i>	최대값	<i>select max(u.age) from User u</i>
	<i>MIN</i>	최소값	<i>select min(u.age) from User u</i>
문자열	<i>CONCAT</i>	문자열 연결	<i>select concat(u.firstName, ' ', u.lastName) from User u</i>
	<i>SUBSTRING</i>	문자열 자르기	<i>select substring(u.name, 1, 3) from User u</i>
	<i>LENGTH</i>	문자열 길이	<i>select length(u.name) from User u</i>
	<i>LOWER</i>	소문자 변환	<i>select lower(u.name) from User u</i>
	<i>UPPER</i>	대문자 변환	<i>select upper(u.name) from User u</i>
	<i>TRIM</i>	앞뒤 공백 제거	<i>select trim(u.name) from User u</i>
날짜/시간	<i>CURRENT_DATE</i>	현재 날짜	<i>select current_date from User u</i>
	<i>CURRENT_TIME</i>	현재 시간	<i>select current_time from User u</i>
	<i>CURRENT_TIMESTAMP</i>	현재 날짜/시간	<i>select current_timestamp from User u</i>

사용자 정의 메서드 – QueryDSL

- 타입 안전, 동적 쿼리, 복잡 로직, 유지보수/재사용성 탁월
- 초기 셋업 필요(**gradle/maven 설정, Q클래스 생성**), 코드가 길어질 수 있음
- 반복/변경 가능한 동적 쿼리는 반드시 QueryDSL

❖ QueryDSL을 설정하려면 라이브러리 종속성을 추가하고, JPA 엔티티를 기반으로 쿼리 타입(Q-Type) 클래스를 생성하도록 빌드 도구를 구성한 뒤, JPAQueryFactory를 빈으로 등록하는 과정이 필요

[실습] Course Repository 작성

- H2 DB 접속 정보를 resources/application.yml 파일에 설정
- table/Course.java 클래스를 엔터티로 하는 repository/CourseRepository.java 인터페이스 작성
- 사용자 정의 메서드 추가
- findByName : 이름으로 조회
- findByTopicContaining : 특정 토픽을 포함하는 코스 조회 (콤마로 구분된 문자열이므로 like 쿼리 사용)

```
spring:
  datasource:
    url: jdbc:h2:mem:scala-stock
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: update
  h2:
    console:
      enabled: true
```

[코드] repository/CourseRepository.java

```
package com.skala.basic.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String topic; // 여러 개의 주제가 필요하다면 콤마(,)로 구분된 문자열로 저장

    private String description;
}
```

[실습] Course Service 개선

- CourseService에서 새로운 course 가 등록되면 CourseRepository를 통해 H2 DB에 저장되도록 구현
- 생성자 주입 방식으로 레파지토리를 설정
- 배열(List) 형식의 topics는 Course 엔터티의 topic으로 콤마가 붙은 단일 문자열로 저장

HelloController의 엔트리 포인트는 변경없이 유지한 상태에서 API 호출

POST http://localhost:8080/courses/홍길동?topics=Spring,Java,AI

```
{
  "name": "홍길동",
  "topics": [
    "Spring",
    "Java",
    "AI"
  ],
  "description": "홍길동님이 관심 있는 분야: Spring, Java, AI"
}
```

❖ H2 Console에서 데이터 확인

The screenshot shows the H2 Console interface. On the left, the database structure is displayed, including the 'COURSE' table with columns 'ID', 'DESCRIPTION', 'NAME', and 'TOPIC'. The 'SQL statement' field contains the query 'select * from course'. Below the query, the results are shown in a table format.

ID	DESCRIPTION	NAME	TOPIC
1	홍길동님이 관심 있는 분야: Spring, Java, AI	홍길동	Spring,Java,AI

(1 row, 1 ms)

[코드] service/CourseService.java

```
@Service
@Slf4j
@RequiredArgsConstructor // 생성자 주입 자동 생성
public class CourseService {

    private final CourseRepository courseRepository;

    public CourseResponse createCourse(String name, List<String> topics) {
        log.info("topics-size={}", topics.size());

        // topics 리스트를 콤마로 구분된 문자열로 변환
        String topic = String.join(",", topics);
        String desc = String.format("%s님이 관심 있는 분야: %s", name, String.join(", ", topics));

        // Course 엔터티 생성
        Course course = new Course();
        course.setName(name);
        course.setTopic(topic);
        course.setDescription(desc);

        // H2 DB에 저장
        Course savedCourse = courseRepository.save(course);

        // 응답 객체 생성
        CourseResponse response = new CourseResponse();
        response.setName(savedCourse.getName());
        response.setTopics(topics); // 원본 List로 반환
        response.setDescription(savedCourse.getDescription());

        return response;
    }
}
```


JPA 트랜잭션 (Transaction)

- 트랜잭션은 데이터베이스에서 일련의 작업들을 하나의 논리적 단위로 묶어서 처리하는 것
- 모든 작업이 성공해야만 실제로 DB에 반영되고, 중간에 오류가 발생하면 전체 작업이 취소(롤백) 됨
- JPA의 변경 내용(저장/수정/삭제)은 트랜잭션이 있을 때만 데이터베이스에 안전하게 반영됨

트랜잭션의 원칙 (ACID)	설명
Atomicity(원자성)	모두 반영되거나 모두 반영되지 않아야 한다.(올 오어 낫싱)
Consistency(일관성)	트랜잭션 전후로 데이터가 일관된 상태 유지해야 한다.
Isolation(격리성)	여러 트랜잭션이 동시에 실행돼도 서로 영향 없어야 한다.
Durability(지속성)	커밋된 결과는 영구적으로 저장되어야 한다.

❖ JPA에서는 서비스 계층에서 트랜잭션을 시작 – 스프링은 **@Transactional** 어노테이션을 사용해 트랜잭션을 자동 관리

→ 실제 데이터베이스 커밋은 @Transactional로 선언된 작업 범위가 성공적으로 끝나는 시점에 발생하며, 개발자가 명시적으로 커밋 시점을 코드로 호출하는 경우는 거의 없음.

JPA 트랜잭션 커밋(Commit) 시점

- JPA에서 트랜잭션이 커밋 되는 시점은 기본적으로 트랜잭션을 시작한 메서드가 성공적으로 종료될 때
- 스프링 환경에서는 @Transactional 어노테이션이 붙은 메서드가 **예외 없이** 실행을 마치는 시점에 커밋 발생

❖ @Transactional 어노테이션

- 스프링에서 서비스 클래스의 메서드에 @Transactional을 붙이면, 스프링 컨테이너는 해당 서비스 객체 대신 트랜잭션 기능이 포함된 **프록시 (Proxy)** 객체를 생성하여 빈(Beans)으로 등록 - 서비스 메서드를 호출하면 실제로는 이 프록시 객체의 메서드가 먼저 호출됨.
- **메서드 시작 시점**: 프록시는 데이터베이스 트랜잭션을 시작(BEGIN)
- **메서드 종료 시점**: 메서드가 정상적으로 종료되면, 프록시는 트랜잭션을 커밋(COMMIT) 하고, 메서드 실행 중 예외(기본적으로 RuntimeException)가 발생하면, 트랜잭션을 롤백(ROLLBACK) 처리

❖ 영속성 컨텍스트(Persistence Context)와 쓰기 지연

- 트랜잭션이 시작되면, JPA는 **영속성 컨텍스트**라는 논리적인 영역을 생성 : 이곳은 엔티티(Entity)를 임시로 저장하고 관리하는 일종의 1차 캐시이자 작업 공간
- **repository.save(entity)**를 호출하면, 엔티티는 데이터베이스에 바로 저장되는 것이 아니라 **영속성 컨텍스트에 저장**되고, 실행해야 할 SQL(INSERT, UPDATE, DELETE 등)은 쓰기 지연 **SQL 저장소**에 쌓이게 됨.
- 이렇게 **SQL을 바로 실행하지 않고 모아두는 것을 쓰기 지연(Write-Behind)** 이라고 함.

@Transactional

- 트랜잭션 관리를 지원하는 선언적 어노테이션
- 메서드(또는 클래스)에 붙이면, 해당 범위 내의 모든 데이터베이스 작업을 트랜잭션 단위로 처리
- 모든 작업이 성공하면 커밋(commit), 중간에 예외가 발생하면 자동으로 롤백(rollback) 처리됨

속성	기본값	설명
propagation	REQUIRED	트랜잭션 전파 방식(예: REQUIRED, REQUIRES_NEW 등)
isolation	DEFAULT	트랜잭션 격리 수준(READ_COMMITTED, REPEATABLE_READ 등)
readOnly	false	읽기 전용 트랜잭션으로 설정. select만 하고 update, insert, delete 안할 때 성능 향상
timeout	-1	트랜잭션 제한 시간(초), 초과시 롤백
rollbackFor	{}	지정한 예외 발생 시에만 롤백 (기본은 RuntimeException, Error 롤백)
noRollbackFor	{}	지정한 예외 발생 시에는 롤백하지 않음

❖ **@Transactional import 위치** : org.springframework.transaction.annotation.Transactional을 사용해야 함.

@Transactional 설정 기준

- DB 변경이 있는 오퍼레이션을 수행하는 서비스 메서드에는 @Transactional을 꼭 사용하는 것이 권장
- @Transactional을 너무 남발하면 트랜잭션 범위가 넓어져 DB lock, 동시성 저하 등 성능 문제가 생길 수 있음

케이스	트랜잭션 적용 방법
데이터 변경(insert/update/delete)	반드시 @Transactional 사용
단순 조회(select)	@Transactional(readOnly = true) 또는 아예 미사용 (가능하면 readOnly 권장)
외부 API/파일 등 장시간 IO 포함	트랜잭션 범위를 DB 오퍼레이션만 포함하도록 최대한 좁게 유지

[참고] @Transactional 설정 범위 예시

```
@Service
public class FileService {

    @Transactional
    public void processAndSave(User user, MultipartFile file) throws IOException {

        // 1. 파일 저장 (네트워크/디스크 IO) - 작업 시간이 길 수 있음
        String filePath = saveFileToDisk(file);

        // 2. DB 저장 (트랜잭션)
        user.setProfileImagePath(filePath);
        userRepository.save(user);

        // 3. 외부 API 호출 (트랜잭션) - 네트워크 작업
        notifyExternalSystem(user);
    }
}
```

트랜잭션 범위가 너무 넓음

```
@Service
public class FileService {

    public void processAndSave(User user, MultipartFile file) throws IOException {
        String filePath = saveFileToDisk(file);    // 파일 저장
        saveUserProfileImagePath(user, filePath);  // DB 저장
        notifyExternalSystem(user);                // 외부 API 호출
    }

    @Transactional // DB 오퍼레이션에만 트랜잭션 적용
    public void saveUserProfileImagePath(User user, String filePath) {
        user.setProfileImagePath(filePath);
        userRepository.save(user);
    }
}
```

트랜잭션 범위를 DB 작업에 한정

→ 내부 사용 함수지만 public 선언

트랜잭션 전파 (Propagation)

- 트랜잭션이 이미 존재할 때, 현재 실행 중인 메서드에서 트랜잭션을 어떻게 처리할지를 결정하는 옵션

→ 기존 트랜잭션에 참여할지, 새로운 트랜잭션을 생성할지, 아니면 트랜잭션 없이 실행할지 선택

옵션명	설명
REQUIRED (기본값)	기존 트랜잭션이 있으면 참여, 없으면 새로 생성 - 대부분의 서비스 로직
REQUIRES_NEW	항상 새로운 트랜잭션 생성, 기존 트랜잭션은 일시 중단(suspend) - 로그 기록, 감사(audit) 등
SUPPORTS	트랜잭션이 있으면 참여, 없으면 트랜잭션 없이 실행 - 트랜잭션 있어도 되고 없어도 되는 조회성 로직 등
MANDATORY	반드시 트랜잭션 내에서만 실행, 없으면 예외 발생 - 트랜잭션 필수적인 보안 로직 등
NOT_SUPPORTED	트랜잭션이 있으면 일시 중단, 트랜잭션 없이 실행 - 외부 시스템 연동, 대용량 데이터 조회 등
NEVER	트랜잭션 없이만 실행, 트랜잭션이 있으면 예외 발생 - 트랜잭션과 같이 동작하면 안 되는 로직 등
NESTED	기존 트랜잭션이 있으면 중첩(새 savepoint), 없으면 새 트랜잭션 - 부분 롤백 지원이 필요한 경우

트랜잭션 전파 오류 – Self Invocation 문제

- 하나의 클래스 내에서 @Transactional이 붙은 다른 메소드를 호출하면 트랜잭션 전파 설정이 적용되지 않음.

← 프록시를 거치지 않고 실제 객체의 내부 메소드를 직접 호출하기 때문

```
@Service
public class UserService {
    @Transactional // (propagation = Propagation.REQUIRED) 기본값
    public void outerMethod() {
        // DB 작업 1
        innerMethod(); // 내부 호출 (프록시 X)
    }

    @Transactional (propagation = Propagation.REQUIRES_NEW)
    public void innerMethod() {
        // DB 작업 2
    }
}
```

❖ **해결 방안** : 트랜잭션 단위가 다른 메소드는 **별도의 클래스(Bean)**로 분리하여 주입받아 사용

트랜잭션 전파 오류 – 해결 방안

- 트랜잭션 단위가 다른 메소드는 별도의 클래스(Bean)로 분리하여 주입받아 사용

```
@Service
public class OuterService {
    @Autowired
    private InnerService innerService;

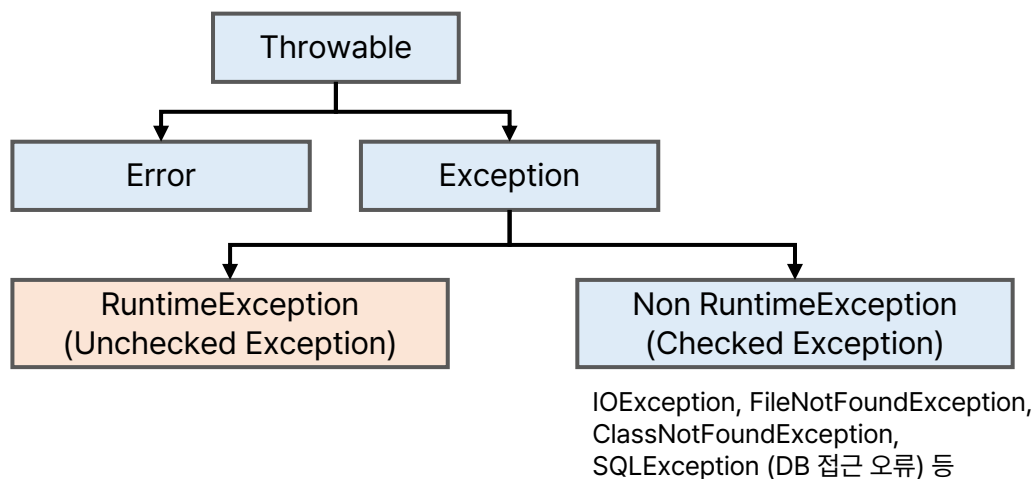
    @Transactional // (propagation = Propagation.REQUIRED)
    public void outerMethod() {
        // ... DB 작업 1 ...
        innerService.innerMethod();
        // ... DB 작업 3 ...
    }
}
```

```
@Service
public class InnerService {
    @Transactional // (propagation = Propagation.REQUIRED)
    public void innerMethod() {
        // ... DB 작업 2 ...
        // 만약 여기서 예외가 발생하여 롤백 되면, outerMethod의 'DB 작업 1'까지 모두 롤백 된다.
    }
}
```


트랜잭션 예외와 롤백 규칙

- @Transactional은 기본적으로 RuntimeException(Unchecked Exception)과 Error가 발생했을 때만 롤백
- Checked Exception: Exception을 상속받는 Checked Exception을 try-catch로 잡고 아무것도 하지 않으면, 트랜잭션은 정상적으로 커밋 되어 데이터 정합성 문제 발생 가능
- 해결 방안: 모든 예외(Exception)에 대해 롤백하고 싶다면 rollbackFor 속성을 사용

```
@Transactional(rollbackFor = Exception.class)
public void process() throws MyCustomException {
    // ... 로직 ...
}
```



트랜잭션 동시성 제어 (Consistency Control)

- 여러 사용자가 동시에 데이터에 접근하고 수정할 때 발생할 수 있는 데이터 불일치 문제를 방지하기 위한 제어
- JPA는 낙관적(Optimistic) Lock과 비관적(Pessimistic) Lock을 제공

구분	개념
낙관적 Lock	- 트랜잭션 충돌이 자주 발생하지 않을 것이라고 가정하는 방식 - DB 레코드 자체에 lock을 걸지 않고 Version을 사용하여 데이터의 변경 여부를 확인
비관적 Lock	- 트랜잭션 충돌이 자주 발생할 것이라고 가정 - 데이터에 접근 시점부터 데이터베이스 레코드에 직접 Lock을 거는 방식 - 다른 트랜잭션은 해당 락이 해제될 때까지 접근 불가

트랜잭션 동시성 제어 – 낙관적 Lock

- **장점:** DB에 직접적인 lock을 걸지 않으므로 부하가 적고 성능이 좋음 (읽기 작업이 많은 환경에 유리)
- **단점:** 충돌이 발생하면 개발자가 직접 예외를 처리하고 재시도 로직 등을 구현해야 함.

```
@Entity
public class Article {
    @Id
    @GeneratedValue
    private Long id;

    private String content;

    @Version
    private Long version; // 버전을 관리할 필드
}
```

- 엔티티에 @Version 어노테이션이 붙은 필드(주로 숫자 타입)를 추가
- 데이터를 조회할 때 이 버전 정보도 함께 가져옴.
- 데이터를 수정하고 트랜잭션을 커밋할 때, 현재 데이터베이스의 버전과 내가 조회했던 시점의 버전을 비교
- 버전이 일치하면 데이터를 수정하고 버전을 1 증가시킴
- 버전이 일치하지 않으면 (다른 트랜잭션이 먼저 수정했다는 의미) `ObjectOptimisticLockingFailureException` 예외를 발생시키고 수정을 실패 처리

트랜잭션 동시성 제어 – 비관적 Lock

- **장점:** 데이터 정합성을 확실하게 보장 (충돌이 잦은 환경에서 데이터 무결성을 유지하는 데 효과적)
- **단점:** 데이터베이스에 직접 락을 걸기 때문에 성능 저하가 발생 가능(다른 트랜잭션의 대기 시간이 길어져 전체적인 시스템 성능에 영향을 주고, 데드락(Deadlock) 발생 가능)
- **Lock 모드:**
 - **PESSIMISTIC_WRITE** : 데이터를 수정하기 위해 설정(다른 트랜잭션은 해당 데이터를 읽거나 쓸 수 없음)
 - **PESSIMISTIC_READ** : 다른 트랜잭션이 해당 데이터를 수정하는 것을 방지(DB에 따라 읽기는 허용될 수 있음)

```
public interface ArticleRepository extends JpaRepository<Article, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Article a where a.id = :id")
    Optional<Article> findByIdWithWriteLock(@Param("id") Long id);
}
```

```
@Repository
public class ArticleRepository {

    @PersistenceContext
    private EntityManager em;

    // PESSIMISTIC_WRITE 락을 사용하여 조회
    public Optional<Article> findByIdWithWriteLock(Long id) {
        return Optional.ofNullable(em.find(Article.class, id, LockModeType.PESSIMISTIC_WRITE));
    }
}
```

[샘플코드] JPA Repository

```
public interface ProductRepository extends JpaRepository<Product, Long> {

    List<Product> findByNameContaining(String keyword);    // 쿼리 메서드
    List<Product> findByNameContainingIgnoreCase(String keyword);

    List<Product> findByPriceLessThanOrderByPriceAsc(int max);

    @Query("select p from Product p where p.price between :min and :max")
    List<Product> findInRange(@Param("min") int min, @Param("max") int max);
}

@Service
public class ProductService {
    private final ProductRepository repo;

    @Transactional                                // 트랜잭션 경계
    public void raisePrice(Long id, int amount) {
        Product p = repo.findById(id).orElseThrow();
        p.setPrice(p.getPrice() + amount);    // 변경감지 -> 자동 UPDATE
    }
}
```

[핵심 요약] JPA Repository

핵심 포인트	반드시 숙지할 내용
JpaRepository	CRUD·페이징·정렬 기본 메서드 자동 제공
커스텀 조회	쿼리 메서드(이름 규칙) · @Query(JPQL) · QueryDSL
@Transactional	트랜잭션 경계 지정 — 커밋/롤백 관리
전파 주의	전파(Propagation), Self-Invocation 시 트랜잭션 미적용 문제
동시성 제어	낙관적 Lock / 비관적 Lock 으로 정합성 보장

REST API 문서화

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

REST API 문서화

- Spring Boot 3.0 이상 버전에서는 springdoc-openapi 라이브러리를 사용하여 API 문서 자동화 지원
- OpenAPI 3 명세 기반의 문서와 Swagger UI를 자동 생성

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

❖ 별도의 추가 설정 없이 위 의존성만 추가하면 기본 문서 생성

- Swagger UI: <http://localhost:8080/swagger-ui/index.html>
- OpenAPI JSON 문서: <http://localhost:8080/v3/api-docs>

OAS 어노테이션 태그

어노테이션	적용 위치	주요 용도/설명
@Tag	클래스(Controller)	API 그룹 및 설명. Swagger UI에서 그룹핑
@Operation	메서드	개별 API의 요약/설명/주석 작성
@Parameter	메서드 파라미터	개별 파라미터 설명 및 예시/기본값 지정
@RequestBody	메서드 파라미터	요청 본문 객체의 설명 및 스키마 자동생성
@ApiResponse	메서드	응답 코드별 설명 및 스키마 지정
@Schema	DTO 클래스/필드	필드별 설명, 예시, 타입, 필수여부, 형식 등
@Hidden	클래스/메서드/필드	문서화에서 숨기기(표시되지 않음)

OAS 문서 작성 예시

- 컨트롤러, 메서드, DTO에 어노테이션으로 부가정보

```
@Tag(name = "Hello API", description = "Hello 엔드포인트 관련 API")
@RestController
public class HelloController {

    @Operation(summary = "Hello 메시지 반환", description = "입력받은 이름을 이용하여 환영 메시지를 반환합니다.")
    @GetMapping("/hello")
    public HelloResponse hello(
        @Parameter(description = "사용자 이름", example = "홍길동")
        @RequestParam String name
    ) {
        return new HelloResponse("안녕하세요, " + name + "님!");
    }
}

public class HelloResponse {
    @Schema(description = "환영 메시지", example = "안녕하세요, 홍길동님!")
    private String message;

    // getter, setter
}
```

Servers

https://petstore3.swagger.io/api/v3 ▾

Authorize 

pet

Everything about your Pets

Find out more 

PUT

/pet

Update an existing pet

 ▾

POST

/pet

Add a new pet to the store

 ▾

GET

/pet/findByStatus

Finds Pets by status

 ▾

GET

/pet/findByTags

Finds Pets by tags

 ▾

GET

/pet/{petId}

Find pet by ID

 ▾

POST

/pet/{petId}

Updates a pet in the store with form data

 ▾

[샘플코드] REST API 문서화 (OpenAPI)

```
// build.gradle:
// implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui'

@Tag(name = "상품 API", description = "상품 CRUD") // 컨트롤러 설명
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Operation(summary = "상품 상세 조회") // 엔드포인트 설명
    @ApiResponse(responseCode = "200", description = "성공")
    @GetMapping("/{id}")
    public Product get(@Parameter(description="상품 ID") @PathVariable Long id) {
        return service.findById(id);
    }
}

// 문서 확인: http://localhost:8080/swagger-ui.html
```

[핵심 요약] REST API 문서화

핵심 포인트	반드시 숙지할 내용
문서화 필요성	API 사용법을 표준 형식으로 제공 → 협업·유지보수 향상
OAS 기반	OpenAPI Specification(Swagger) 기반 문서화
OAS 어노테이션	어노테이션·태그로 엔드포인트·파라미터·응답 명세 작성
자동 생성	코드로부터 API 문서 자동 생성

JWT (JSON Web Token)

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

JSON Web Token(JWT)

- 웹 애플리케이션에서 인증(Authentication) 및 정보 전달을 위해 널리 사용되는 토큰 기반 인증 방식
- JWT는 서버와 클라이언트 간에 인증 정보를 *Base64Url 방식으로 인코딩하여 전달

JWT 구조

[Header].[Payload].[Signature]

부분	설명
Header	토큰의 타입(JWT)과 서명 알고리즘(HS256, RS256 등)을 명시
Payload	사용자 정보(Claims)가 들어가는 부분. 예: 사용자 ID, 권한, 만료시간 등
Signature	Header와 Payload를 인코딩한 후 비밀 키로 서명한 값. 무결성 검증에 사용됨

* Base64Url 인코딩은 Base64 인코딩과 구분됨.

❖ JSON Web Token (JWT) Debugger : <https://jwt.io/>

JWT Header

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

필드	설명
alg	Algorithm: 서명에 사용된 알고리즘 (예: HS256, RS256 등)
typ	Type: 토큰의 타입 (보통 "JWT"로 고정)
kid	(선택) Key ID: 여러 키 중 어떤 키로 서명 되었는지 식별할 때 사용

JWT Payload (Claims)

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1716249022,
  "exp": 1716252622
}
```

표준 등록 필드	설명
iss	Issuer: 토큰 발급자
sub	Subject: 토큰 주제 (예: 사용자 ID)
aud	Audience: 토큰 대상자 (예: 서비스 이름)
exp	Expiration: 만료 시간 (UNIX timestamp)
nbf	Not Before: 이 시간 이전에는 유효하지 않음
iat	Issued At: 토큰 발급 시간
jti	JWT ID: 토큰 고유 식별자 (재사용 방지에 유용)

- ❖ 사용자 정의 필드는 임의로 추가 가능
- ❖ 민감한 정보(비밀번호, 주민번호 등)는 절대 포함하지 말 것! JWT Payload는 인코딩 일 뿐 암호화되지 않음.

Spring Security

- Spring Boot MVC 기반 REST API 애플리케이션의 Spring Security 설정 범위

범위	설명
인증(Authentication)	사용자가 누구인지 확인 (예: 로그인)
인가(Authorization)	특정 리소스에 접근 가능한지 판단 (예: ROLE 기반 접근 제한)
세션/토큰 관리	JWT, 세션 쿠키 등 인증 상태 유지 방식
CORS 정책	다른 도메인에서의 요청 허용 여부
CSRF 보호	주로 웹 브라우저 환경에서 사용됨, REST API에서는 보통 비활성화
예외 처리	인증/인가 실패 시 응답 형식 지정 (예: 401, 403 JSON 응답 등)
HTTP 요청 보안 설정	<code>http.authorizeHttpRequests()</code> 등으로 경로별 권한 제어

- ❖ MSA(Micro-service Architecture) 구조에서 BFF(Backend For Frontend) 노드가 인증/인가를 전담할 경우, 그 뒤에 위치한 일반 REST API 애플리케이션에서 **CORS**를 포함한 모든 보안 정책을 허용 가능

[참고] CORS (Cross-Origin Resource Sharing)

- 웹 브라우저는 보안상의 이유로 다른 출처(origin)의 리소스 요청을 기본적으로 차단
- CORS는 서버가 "내 리소스를 특정 외부 도메인에서 접근해도 된다"고 명시해주는 방식
 - CORS는 클라이언트 측의 보안 제약이며, 서버 보안 정책은 아님.
 - 서버는 Access-Control-Allow-Origin: * 설정 시 모든 도메인 허용

// WebMvcConfigurer를 통한 전역 CORS 설정 예시

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("*") // 모든 도메인 허용
            .allowedMethods("*") // 모든 HTTP 메서드 허용
            .allowedHeaders("*") // 모든 헤더 허용
            .allowCredentials(true);
    }
}
```

JWT를 이용한 인증/인가 방식 (예시)

1. 로그인	서버가 JWT 발급 후 HttpOnly 쿠키로 반환
2. 인증 처리	서비스에서 세션 관리자를 통해 쿠키에서 토큰 추출 → 서명 검증 → 사용자 정보 확인
3. 로그아웃	쿠키 만료 (Set-Cookie: access_token=; Max-Age=0)로 클라이언트 토큰 제거

로그인 하지 않은 사용자 (유효하지 않은 JWT)	<pre>POST /api/players/buy { "playerId": "postman", "stockId": 1, "stockQuantity": 1 }</pre>	<pre>{ "result": 1, "code": 9004, "message": "SESSION_NOT_FOUND", "body": null }</pre>
로그인 한 사용자 (유효한 JWT)	<pre>POST /api/players/login { "playerId": "postman", "playerPassword": "1234" }</pre>	<pre>{ "result": 0, "code": 0, "message": null, "body": {...} }</pre> Cookie 에 JWT 생성
	<pre>POST /api/players/buy { "playerId": "postman", "stockId": 1, "stockQuantity": 1 }</pre> API 요청 시 Cookie 자동 전달	<pre>{ "result": 0, "code": 0, "message": null, "body": null }</pre>

❖ 구현 예시는 기능 설명을 위해 서버에서 세션 관리 없이 Cookie에 설정된 JWT 인증 방식으로 단순화

SKALA-STOCK-API의 JWT 적용 예시

1. Maven Dependency 설정

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

2. JWT 인코딩 및 디코딩

```
# com.sk.skala.stockapi.tools.JwtTool
generateToken(String id, Object payload)
getValidPayload(String token)
```

3. Cookie에 JWT 설정 및 설정 정보(playerId) 확인

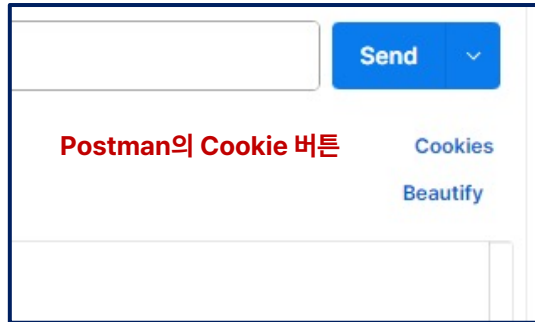
```
# com.sk.skala.stockapi.service.SessionHandler
storeAccessToken(PlayerSession playerSession)
getPlayerId()
```

4. 로그인시 Cookie로 전달한 JWT에서 주식 거래과정에서 playerId 검증

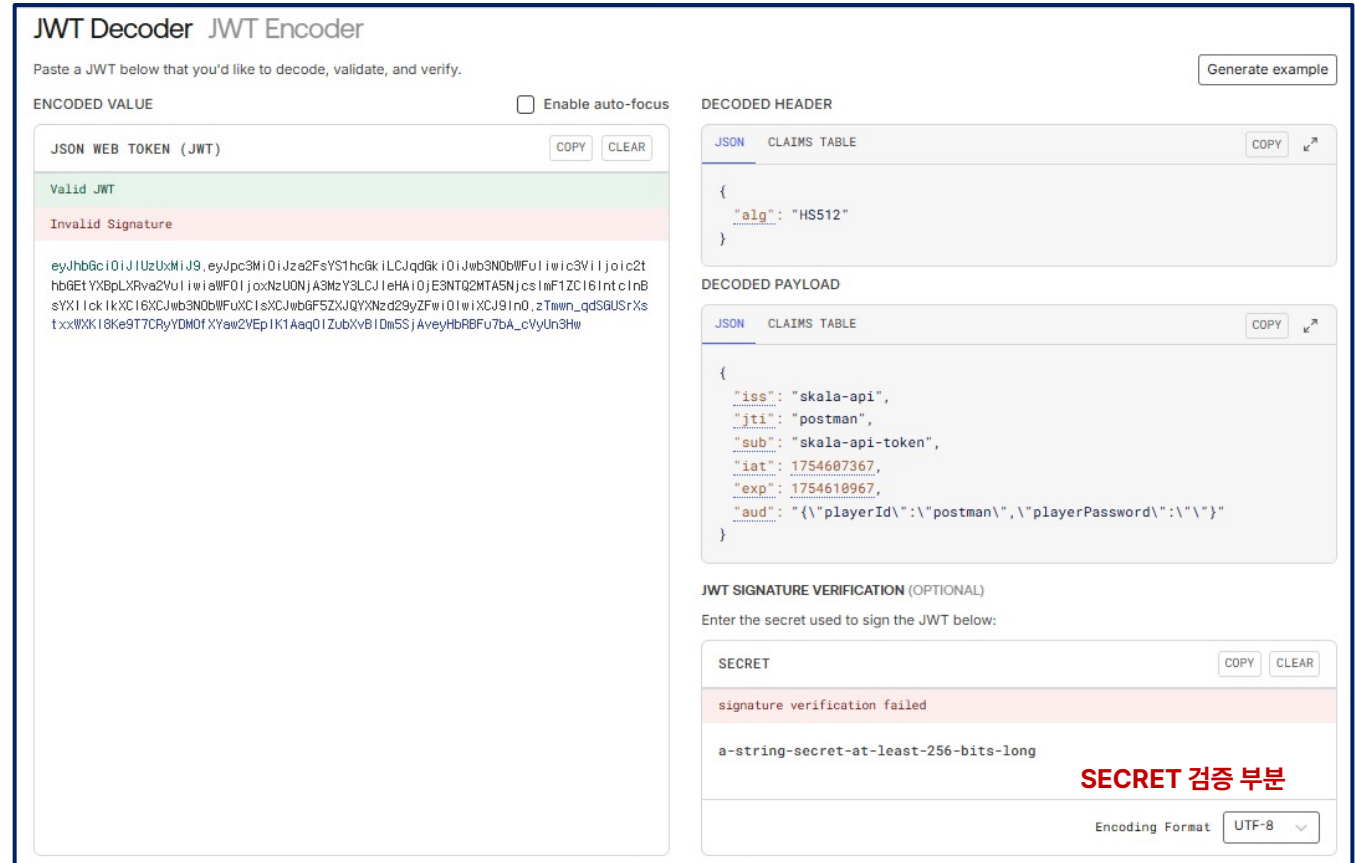
```
# com.sk.skala.stockapi.service.PlayerService
loginPlayer(PlayerSession playerSession)
buyPlayerStock(StockOrder order)
sellPlayerStock(StockOrder order)
```

[참고] Cookie로 전달된 JWT 검증 방법

- 플레이어가 로그인에 성공하면 Cookie에 "bff-access"라는 이름의 JWT가 전달됨
- Cookie의 Value를 복사하여 <https://www.jwt.io/>에서 토큰의 내용 확인 (aud 필드에 플레이어ID 존재)
- SECRET은 Constant 클래스에 ***JWT_SECRET*** 상수로 저장되어 있음.



❖ Cookie 팝업 화면을 통해 발행된
JWT 삭제도 가능



[샘플코드] JWT 인증

@Component

```
public class JwtProvider {  
    private final Key key =  
        Keys.hmacShaKeyFor("my-secret-key-must-be-long...".getBytes());  
  
    public String createToken(String customerId) {        // 토큰 생성  
        return Jwts.builder()  
            .setSubject(customerId)  
            .setExpiration(new Date(System.currentTimeMillis() + 3600000))  
            .signWith(key, SignatureAlgorithm.HS256)        // HS256 서명  
            .compact();  
    }  
    public String getCustomerId(String token) {        // 토큰 검증/파싱  
        return Jwts.parserBuilder().setSigningKey(key).build()  
            .parseClaimsJws(token).getBody().getSubject();  
    }  
}
```

[핵심 요약] JWT (JSON Web Token)

핵심 포인트	반드시 숙지할 내용
JWT	토큰 기반 인증 방식 — Header.Payload.Signature 3부분
구성	Header(alg , 예 HS256) · Payload(Claims) · Signature(서명)
Spring Security	인증(Authentication)·인가(Authorization) 처리 프레임워크
CORS	브라우저의 교차 출처 요청 차단 정책 — 서버에서 허용 설정 필요

[실습] 스칼라 온라인 쇼핑몰 API

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

실습 시나리오 ① – 온라인 쇼핑몰 'SKALA-SHOP' 개요

항목	내용
서비스	SKALA-SHOP — 온라인 쇼핑몰 백엔드 REST API
목표	상품·고객·주문을 관리하는 API를 계층형 구조(Controller–Service–Repository)로 구현
핵심 도메인	상품(Product) · 고객(Customer) · 주문상품(OrderItem) — 고객이 주문한 상품(1:N)
사용자	쇼핑몰 고객 — 회원가입·로그인 후 상품을 주문
기술 스택	Spring Boot · JPA / H2 · JWT 인증 · Gradle
기능 범위	상품 CRUD · 고객 관리/인증 · 주문(답기) · 주문 취소

실습 시나리오 ② – 고객의 상품 주문 여정 (User Journey)

단계	고객 행동	API
1. 회원가입	ID·비밀번호로 가입, 초기 포인트 지급	POST /api/customers
2. 로그인	로그인 성공 시 JWT 토큰 발급	POST /api/customers/login
3. 상품 조회	판매 상품 목록·상세 확인	GET /api/products
4. 상품 주문	원하는 상품을 수량만큼 주문(포인트 차감)	POST /api/customers/order
5. 주문 확인	내가 주문한 상품 목록 조회	GET /api/customers/{id}
6. 주문 취소	주문 취소(포인트 환급)	POST /api/customers/cancel

실습 시나리오 ③ – 비즈니스 규칙 & 예외 처리

구분	규칙
포인트	고객은 보유 포인트 로 주문 → 부족 시 주문 거부(INSUFFICIENT_FUNDS)
주문 수량	같은 상품 재주문 시 수량 누적, 취소 시 수량 차감(0이면 삭제)
인증(JWT)	주문·취소는 로그인 필수 → 토큰(Cookie)에서 고객을 식별
입력 검증	상품명·가격 등 필수값 검증 실패 시 ParameterException
예외 처리	없는 상품·고객 조회 시 DATA_NOT_FOUND (전역 예외 처리)
트랜잭션	주문·취소는 @Transactional 로 원자적 처리(포인트·수량 일관성)

실습 시나리오 ④ – API 호출 흐름 예시 (End-to-End)

1) 회원가입 (초기 포인트 지급)

POST /api/customers

{ "customerId": "skala01", "customerPassword": "pw1234" }

2) 로그인 -> JWT 발급 (이후 요청에 Cookie 자동 첨부)

POST /api/customers/login

{ "customerId": "skala01", "customerPassword": "pw1234" }

-> Set-Cookie: bff-access=<JWT>

3) 상품 목록 조회

GET /api/products

-> [{ "id":1, "name":"무선마우스", "price":15000 }, ...]

4) 상품 주문 - 무선마우스 2개 (JWT 필요, 포인트 차감)

POST /api/customers/order

{ "productId": 1, "quantity": 2 } -> 200 OK, 남은 포인트 970000

5) 내 주문 확인

GET /api/customers/skala01

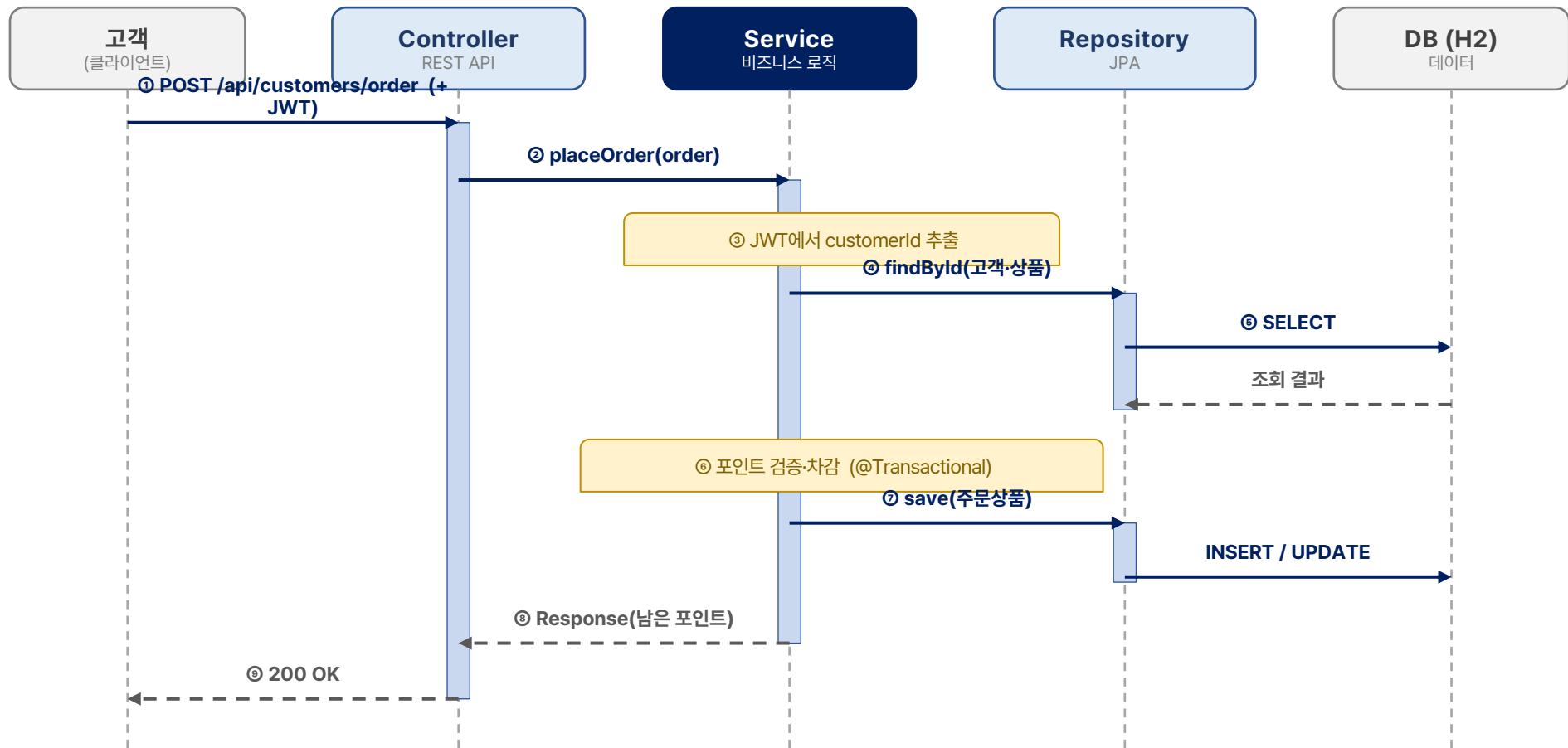
-> { "customerPoint":970000, "products":[{ "무선마우스", "수량":2 }] }

6) 주문 취소 - 1개 취소 (포인트 환급)

POST /api/customers/cancel

{ "productId": 1, "quantity": 1 } -> 200 OK, 남은 포인트 985000

실습 시나리오 - 시퀀스 다이어그램 : 상품 주문 처리 흐름

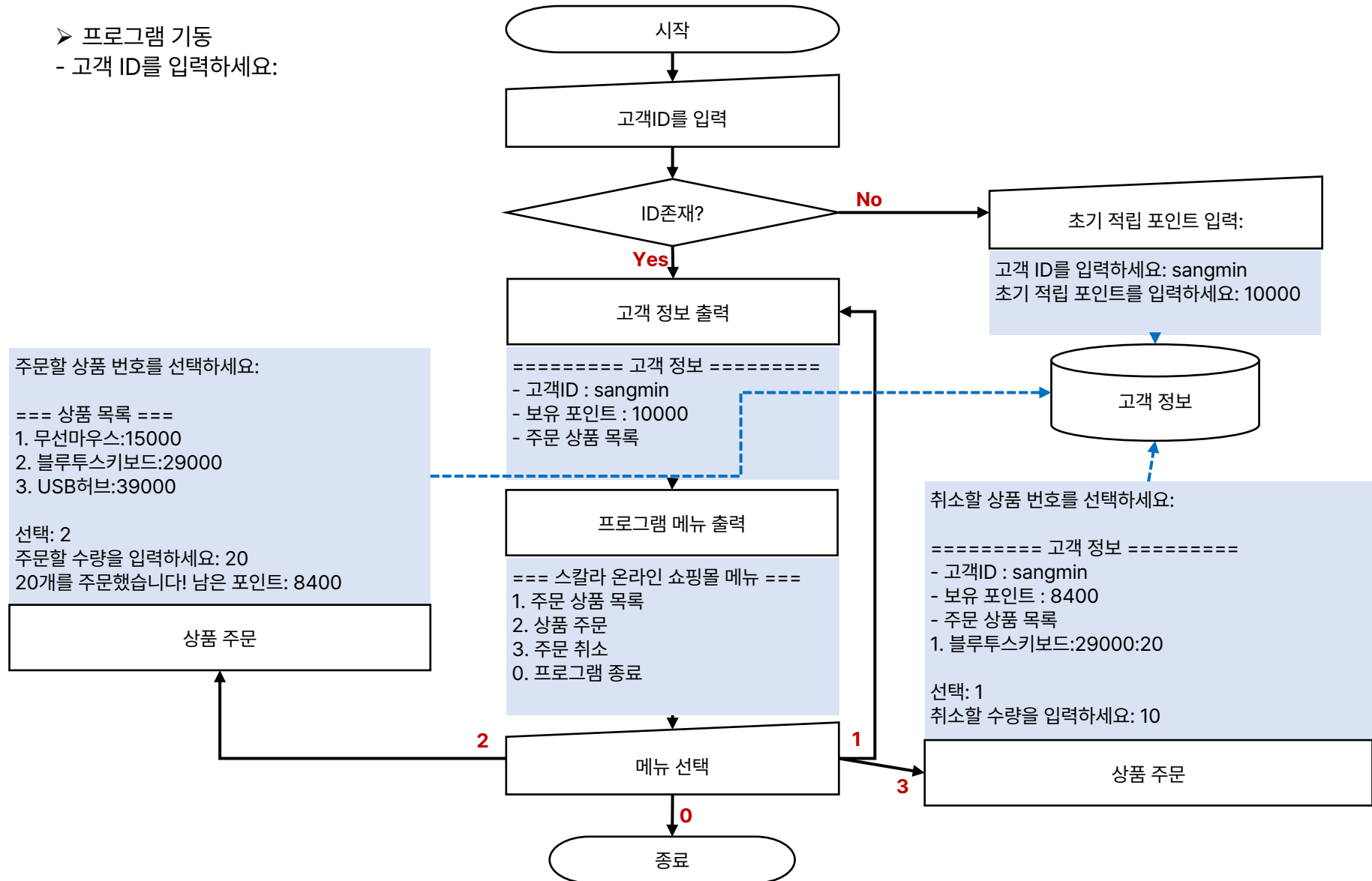


시나리오 : 상품 주문 처리 (POST /api/customers/order)

실선 화살표 = 요청/호출, 점선 화살표 = 응답/반환 | 포인트 부족 시 INSUFFICIENT_FUNDS 로 주문 거부

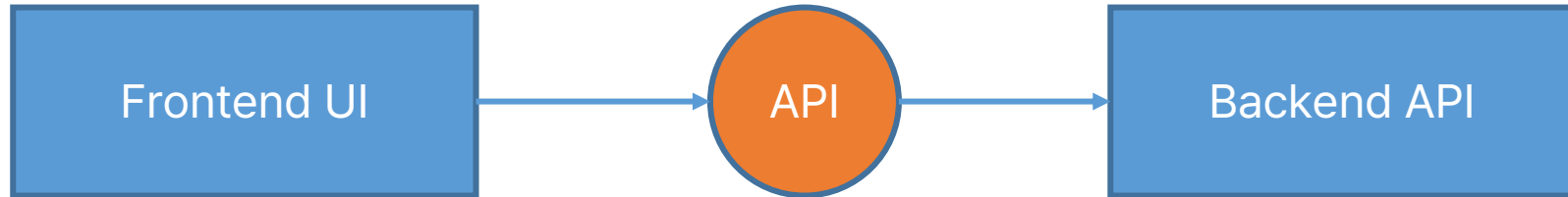
[리뷰] JAVA 콘솔 (skala-shop-console) 시나리오

- 프로그램 기동
- 고객 ID를 입력하세요:



Frontend UI 와 Backend API 관계

- Frontend : 화면 구성(렌더링)을 위해 필요한 정보를 API를 통해 얻음
- Backend : API를 통해 Storage(DB 등)에 저장된 정보를 CRUD



스칼라 온라인 쇼핑몰 (skala-shop-api) 패키지 구조

구분	폴더/파일	설명
src/main/java		
	com.sk.skala.shopapi	어플리케이션 기본 폴더(패키지)
	com.sk.skala.shopapi.aop	AOP 클래스 – API 로그
	com.sk.skala.shopapi.config	어플리케이션 초기 설정 정보 – DB 접속, 보안 등
	com.sk.skala.shopapi.controller	API 컨트롤러 – API 인입 포인트
	com.sk.skala.shopapi.data	데이터 객체 – 테이블, DTO 등
	com.sk.skala.shopapi.exception	에러(exception) 클래스
	com.sk.skala.shopapi.repository	DB 레파지토리
	com.sk.skala.shopapi.service	API 서비스 – 컨트롤러와 레파지토리 연결
	com.sk.skala.shopapi.tools	유틸리티 도구 – 문자열, Json, JWT 등
src/main/resources	application.yml	어플리케이션 구성정보 – DB 접속 등

Gradle 프로젝트 구조 (skala-shop-api)

구분	파일 / 폴더	설명
빌드 스크립트	<code>build.gradle</code>	플러그인·의존성·버전 선언 (Maven pom.xml 대체)
	<code>settings.gradle</code>	루트 프로젝트 이름 설정 – <code>rootProject.name = 'skala-shop-api'</code>
Gradle Wrapper	<code>gradlew / gradlew.bat</code>	Gradle 설치 없이 지정 버전으로 빌드 (Linux·Mac / Windows)
	<code>gradle/wrapper/gradle-wrapper.properties</code>	사용할 Gradle 버전 지정
	<code>gradle/wrapper/gradle-wrapper.jar</code>	Wrapper 실행용 라이브러리
소스	<code>src/main/java</code>	애플리케이션 소스 코드 (com.sk.skala.shopapi ...)
	<code>src/main/resources</code>	application.yml 등 구성 파일
	<code>src/test/java</code>	테스트 코드
산출물	<code>build/</code>	컴파일·패키징 결과물 (Maven target/ 에 해당)

스칼라 온라인 쇼핑몰 (skala-shop-api) API 목록

Resource	Http Method	API URI	설명
상품(product)	GET	~/api/products	상품 전체 목록 조회(페이지 단위 처리)
	GET	~/api/products/{id}	ID에 해당하는 개별 상품 상세 정보
	POST	~/api/products	상품 정보 추가
	PUT	~/api/products	상품 정보 변경
	DELETE	~/api/products	상품 정보 삭제
고객(customer)	GET	~/api/customers	고객 전체 목록 조회(페이지 단위 처리)
	GET	~/api/customers/{customerId}	ID에 해당하는 고객 상세 정보
	GET	~/api/customers/{customerName}	Name에 해당하는 고객 상세 정보
	POST	~/api/customers	고객 정보 추가
	POST	~/api/customers/login	고객 로그인
	PUT	~/api/customers	고객 정보 변경
	DELETE	~/api/customers/{customerId}	고객 정보 삭제
	GET	~/customers/{customerId}/products	고객의 상품 정보 조회
	POST	~/api/customers/order	고객의 상품 주문
	POST	~/api/customers/cancel	고객의 주문 취소

프로젝트 정보(Gradle build.gradle)

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '3.3.0'  
    id 'io.spring.dependency-management' version '1.1.5'  
}  
group = 'com.sk.skala'  
version = '0.0.1-SNAPSHOT'  
description = 'SKALA SHOP REST API'  
java {  
    sourceCompatibility = '17'  
}  
repositories {  
    mavenCentral()  
}
```

프로젝트 의존성 (build.gradle)

Configuration	Dependency (group:artifact)	설명
implementation	org.springframework.boot:spring-boot-starter-data-jpa	JPA 기반 DB 연동(Spring Data JPA)
implementation	org.springframework.boot:spring-boot-starter-aop	AOP(관점지향 프로그래밍) 기능 제공
implementation	org.springframework.boot:spring-boot-starter-actuator	Actuator(운영 및 모니터링 Endpoints)
implementation	org.springframework.boot:spring-boot-starter-web	Web(REST API 포함) 개발에 필요한 기본 구성 제공
annotationProcessor	org.springframework.boot:spring-boot-configuration-processor	@ConfigurationProperties 자동 처리(메타데이터 생성 등)
compileOnly / annotationProcessor	org.projectlombok:lombok	Lombok(코드 자동 생성: getter/setter, 빌더 등)
testImplementation	org.springframework.boot:spring-boot-starter-test	테스트(JUnit, Mockito 등 포함)
runtimeOnly	com.h2database:h2	H2 인메모리 DB(개발/테스트용)
implementation	io.jsonwebtoken:jjwt-api:0.11.5	JWT 인증 토큰 생성/검증용 API
runtimeOnly	io.jsonwebtoken:jjwt-impl:0.11.5	JWT 구현체(Runtime시에 필요)
runtimeOnly	io.jsonwebtoken:jjwt-jackson:0.11.5	JWT 처리에 Jackson(JSON) 사용 지원

- ❖ **Plugins** : plugins 블록에 org.springframework.boot, io.spring.dependency-management 선언 (Maven의 spring-boot-maven-plugin 대체)
- ❖ **Lombok** : compileOnly + annotationProcessor 로 선언 → 컴파일 시에만 사용되고 빌드 산출물(jar)에는 포함되지 않음

[실습 1-1] JPA 엔터티 - 상품

[Product.java]

- JPA 테이블 **product**의 엔터티로 ID는 순차적으로 자동 증가
- 속성
 - Long id;
 - String productName;
 - Double productPrice;
- 생성자
 - 빈 생성자
 - productName과 productPrice를 인자로 받는 생성자
- 메서드
 - Setter 및 Getter

[실습 1-2] JPA 엔터티 - 고객

[Customer.java]

- JPA 테이블 **customer**의 엔터티로 ID는 문자열 형식의 **customerId**
- 속성
 - String customerId;
 - String customerPassword;
 - Double customerPoint;
- 생성자
 - 빈 생성자
 - 고객 ID와 초기 적립 포인트를 인자로 받는 생성자
- 메서드
 - Setter 및 Getter

[실습 1-3] JPA 엔터티 - 고객이 주문한 상품 매핑

[OrderItem.java]

- Customer가 보유한 Product 정보를 관리하기 위해 두 엔터티를 연결하는 중간 엔터티(매핑 엔터티)
- 엔터티의 ID는 순차적으로 자동 증가
- 속성
 - Long id;
 - Customer customer; // Customer 테이블의 PK를 참조 - @JoinColumn
 - Product product; // Product 테이블의 PK를 참조 - @JoinColumn
 - Integer quantity;
- 생성자
 - 빈 생성자
 - Customer와 Product 그리고 주문 상품 수량을 인자로 받는 생성자
- 메서드
 - Setter 및 Getter

❖ @ManyToOne(fetch = FetchType.LAZY) : 연관 엔티티를 DB에서 바로 조회하지 않고, 필요한 시점에 가져오는 방식

[실습 1-4] DTO – 고객이 주문한 상품

[OrderItemDto.java]

- **Customer가 보유하는 상품 정보 (상품 엔터티에 수량을 추가) – Builder 패턴으로 메서드 체이닝 제공**
- **속성**
 - Long productId;
 - String productName;
 - Double productPrice;
 - Integer quantity;
- **생성자**
 - 빈 생성자
- **메서드**
 - Setter 및 Getter

[실습 1-5] DTO – 고객 주문 상품 목록 조회 응답

[OrderListDto.java]

- **Customer가 주문한 상품 목록 조회 응답 – Builder 패턴으로 메서드 체이닝 제공**
- **속성**
 - String customerId;
 - Double customerPoint;
 - List<OrderItemDto> products
- **생성자**
 - 빈 생성자
- **메서드**
 - Setter 및 Getter

[실습 2-1] Repository – 상품 데이터 저장소

[ProductRepository.java]

- **Product** 엔티티를 관리하기 위해 **JpaRepository** 상속한 인터페이스 구현
- 사용자 정의 메서드
 - `findByProductName` : 상품 이름과 동일한 상품 데이터를 조회 (존재 여부 확인 목적 → Optional)

[실습 2-2] Repository – 고객 데이터 저장소

[CustomerRepository.java]

- Customer 엔티티를 관리하기 위해 JpaRepository 상속한 인터페이스 구현
- 사용자 정의 메서드 : 필요 없음

[실습 2-3] Repository – 고객 주문 상품 매핑 저장소

[CustomerProductRepository.java]

- **OrderItem** 엔티티를 관리하기 위해 **JpaRepository** 상속한 인터페이스 구현
- **사용자 정의 메서드**
 - `findByCustomer_CustomerId` : **OrderItem** 엔티티의 `customer` 필드가 참조하는 **Customer** 엔티티의 `customerId` 필드 값과 일치하는 **OrderItem** 목록 조회
 - `findByCustomerAndProduct` : 특정 고객(**Customer**)가 특정 상품(**Product**)을 보유하고 있는지 검색

❖ `findByCustomer_CustomerId` 메서드는 **속성 순회(Property Traversal)** 활용 : 연관된 엔티티의 속성'을 이용해 쿼리 조건을 만드는 것
→ 고객 객체를 전달 받는 방식 형식도 구현 가능 : `List<OrderItem> findItemsByCustomer(Customer customer)`

[참고] JPA 쿼리 메소드 작명 가이드

- JPA가 쿼리를 생성할 때 메소드 이름을 분석하는 규칙

find(조회) + { Subject } + By + { Predicate }

- find...By* : *find, get, read, query, search* 등 조회 동작을 나타내는 접두사와 *By* 키워드를 구분
- { Subject }* : *find*와 *By* 사이에 들어가는 부분은 쿼리를 생성하는 데 아무런 영향을 주지 않으며, 오직 개발자의 가독성을 위해 존재
- { Predicate }* : *By* 뒤에 오는 부분으로 실제 쿼리 조건(WHERE 절)을 만드는 데 사용

※ 다음 두 메서드는 동일하게 동작

```
List<OrderItem> findByCustomer(Customer customer)
```

```
List<OrderItem> findItemsByCustomer(Customer customer)
```

기능	명명 규칙 키워드	설명
정렬	...OrderBy<필드><Asc Desc>	특정 필드를 기준으로 결과를 정렬
개수 세기	countBy<필드>	특정 조건을 만족하는 엔티티의 개수
중복 제거	...Distinct...	조회 결과에서 중복을 제거
그룹화(GROUP BY)	없음	메소드 이름만으로는 GROUP BY를 지원하지 않음.
복잡한 쿼리	@Query 어노테이션	JPQL을 직접 작성하는 것이 표준적인 방법

[실습 3-1] Service – 상품 관리 서비스 (1/2)

[ProductService.java]

▪ 클래스 선언 및 필드 정의

```
@Service
@RequiredArgsConstructor
public class CustomerService {
    private final ProductRepository productRepository;
}
```

▪ 메서드

// 전체 상품 목록 조회

```
public Response getAllProducts(int offset, int count)
```

- Pageable 객체 생성 (페이징 및 정렬)
- productRepository.findAll(pageable)로 페이지 단위 데이터 조회
- 결과를 PagedList 객체로 가공
- Response 객체에 담아 반환

// 개별 상품 상세 조회

```
public Response getProductById(Long id)
```

- ID로 상품(Product) 조회, Optional로 존재 여부 확인 : 오류시 ResponseException(Error.DATA_NOT_FOUND)
- Response 객체에 담아 반환

[실습 3-1] Service – 상품 관리 서비스 (2/2)

▪ 메서드

// 상품 등록 (생성)

```
public Response createProduct(Product product)
```

- 입력값 검증 (productName 비어있음 or price ≤ 0) : 오류시 ParameterException("productName", "productPrice")
- 이름 중복 체크 (findByProductName) : 중복시 ResponseException(Error.DATA_DUPLICATED)
- 신규 Product의 ID는 0L로 세팅(=JPA가 자동 생성)
- 저장 후 Response 반환

// 상품 정보 수정

```
public Response updateProduct(Product product)
```

- 입력값 검증 (productName, productPrice) : 오류시 ParameterException("productName", "productPrice")
- 해당 ID의 Product이 존재하는지 확인 : 오류시 ResponseException(Error.DATA_NOT_FOUND)
- 있으면 수정(저장) 후 Response 반환

// 상품 삭제

```
public Response deleteProduct(Product product)
```

- ID로 조회해서 존재하면 삭제, 없으면 예외 처리 : ResponseException(Error.DATA_NOT_FOUND)
- 저장 후 Response 반환

[실습 3-2] Service – 고객 관리 서비스 (1/3)

[CustomerService.java]

▪ 클래스 선언 및 필드 정의

```
@Service
@RequiredArgsConstructor
public class CustomerService {
    private final ProductRepository productRepository;
    private final CustomerRepository customerRepository;
    private final OrderItemRepository orderItemRepository;
    private final SessionHandler sessionHandler;
}
```

▪ 메서드

// 전체 고객 목록 조회

```
public Response getAllCustomers(int offset, int count)
```

- Pageable 객체 생성 (페이지네이션)
- customerRepository.findAll()로 페이지 단위 조회
- PagedList로 결과 변환 후 Response로 감싸 반환

// 단일 고객 및 상품 목록 조회

```
@Transactional(readOnly = true)
```

```
public Response getCustomerById(String customerId)
```

- Customer 엔티티 존재 여부 검증, 예외 처리 (ResponseException(Error.DATA_NOT_FOUND, "Customer not found"))
- 고객이 보유한 OrderItem 리스트 조회
- Stream API로 DTO 리스트 변환
- DTO를 Response에 세팅해 반환

[실습 3-2] Service – 고객 관리 서비스 (2/3)

▪ 메서드

// 고객 생성

```
public Response createCustomer(Customer customerSession)
```

- 입력 값 검증 (isEmpty) : 오류시 ParameterException("customerId", "customerPassword")
- 중복 아이디 체크 : 오류시 ResponseException(Error.DATA_DUPLICATED)
- Customer 객체 생성, 초기 적립 포인트 세팅
- 저장 후 Response 반환

// 고객 로그인

```
public Response loginCustomer(CustomerSession customerSession)
```

- 입력 값 검증 : 오류시 ParameterException("customerId", "customerPassword")
- ID로 조회, 비밀번호 검증 : 오류시 ResponseException(Error.DATA_NOT_FOUND), (Error.NOT_AUTHENTICATED)
- 인증 성공시 sessionHandler.storeAccessToken 호출
- 고객 정보 Response의 body에 담아 반환 (패스워드 null 처리)

// 고객 정보 업데이트

```
public Response updateCustomer(Customer customer)
```

- customerId와 customerPoint 유효성 체크 : 오류시 ResponseException(Error.DATA_NOT_FOUND)
- 존재 확인 후, 포인트 업데이트
- 저장 후 Response 반환

// 고객 삭제

```
public Response deleteCustomer(Customer customer)
```

- customerId로 존재 확인 후 삭제 : 오류시 ResponseException(Error.DATA_NOT_FOUND)
- 저장 후 Response 반환

[실습 3-2] Service – 고객 관리 서비스 (3/3)

▪ 메서드

// 상품 주문

@Transactional

public Response placeOrder(OrderRequest order)

- 현재 로그인된 customerId 가져오기 (sessionHandler)
- customer, product 엔터티 조회 및 검증 : 오류시 ResponseException(Error.DATA_NOT_FOUND)
- 포인트 충분성 체크 및 차감 : 오류시 ResponseException(Error.INSUFFICIENT_FUNDS)
- OrderItem에 이미 주문한 상품이면 수량 추가, 없으면 신규 생성
- 요청 처리 결과 Response 로 성공 응답

// 주문 취소

@Transactional

public Response cancelOrder(OrderRequest order)

- 취소할 Customer, Product 엔터티 조회 : 오류시 ResponseException(Error.DATA_NOT_FOUND)
- OrderItem 보유수량 검증 : 오류시 ResponseException(Error.INSUFFICIENT_QUANTITY)
- 수량 감소 또는 삭제 처리
- 취소 금액만큼 고객 포인트 증가
- 요청 처리 결과 Response 로 성공 응답

[실습 4-1] Controller – 상품 API (1/2)

[ProductController.java]

▪ 클래스 선언 및 필드 정의

```
@RestController
@RequestMapping("/api/products") //이 컨트롤러의 기본 URL 경로를 /api/products로 지정
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;
}
```

▪ 메서드

// 전체 상품 목록 조회 API

```
@GetMapping("/list")
public Response getAllProducts(@RequestParam(defaultValue = "0") Integer offset,
                               @RequestParam(defaultValue = "10") Integer count)
- 쿼리 파라미터로 offset, count(기본 0, 10)
- Service에서 페이징 적용된 상품 목록을 받아 반환
```

// 개별 상품 상세 조회 API

```
@GetMapping("/{id}")
public Response getProductById(@PathVariable Long id)
- 서비스에서 해당 상품 정보 반환
```

[실습 4-1] Controller – 상품 API (2/2)

▪ 메서드

// 상품 등록 API

@PostMapping

public Response createProduct(@RequestBody Product product)

- JSON 바디에 상품 정보(Product 엔터티) 입력
- 서비스를 통해 새 상품 등록, 결과 반환

// 상품 정보 수정 API

@PutMapping

public Response updateProduct(@RequestBody Product product)

- 서비스를 통해 상품 정보 업데이트

// 상품 삭제 API

@DeleteMapping

public Response deleteProduct(@RequestBody Product product)

- 서비스로 삭제할 상품 정보 전달(일반적으로 id만 필요)

[실습 4-2] Controller – 고객 API (1/2)

[CustomerController.java]

▪ 클래스 선언 및 필드 정의

```
@RestController
@RequestMapping("/api/customers") //이 컨트롤러의 기본 URL 경로를 지정
@RequiredArgsConstructor
public class CustomerController {
    private final CustomerService customerService;
}
```

▪ 메서드

// 전체 고객 목록 조회 API

```
@GetMapping("/list")
public Response getAllCustomers(@RequestParam(value = "offset", defaultValue = "0") int offset,
                                @RequestParam(value = "count", defaultValue = "10") int count)
- 쿼리 파라미터로 offset, count(기본 0, 10)
- CustomerService에서 페이징 적용된 고객 목록을 받아 반환
```

// 단일 고객 상세 조회 API

```
@GetMapping("/{customerId}")
public Response getCustomerById(@PathVariable String customerId)
- 경로변수(@PathVariable)로 customerId 전달
- 해당 고객 정보 + 주문 상품 리스트 반환
```

[실습 4-2] Controller – 고객 API (2/2)

▪ 메서드

```
//고객 등록
@PostMapping
public Response createCustomer(@RequestBody Customer customer)
- 서비스로 Customer 정보 전달
- 신규 고객 등록, Response 반환

// 고객 로그인
@PostMapping("/login")
public Response loginCustomer(@RequestBody CustomerSession customerSession)
- 서비스로 CustomerSession(id, pw) 전달
- 로그인 성공 시 토큰/세션 부여 및 고객 정보 반환

// 고객 정보 수정
@PutMapping
public Response updateCustomer(@RequestBody Customer customer)
- 서비스로 수정할 Customer 정보 전달
- 고객 정보(주로 포인트) 업데이트

// 고객 삭제
@DeleteMapping
public Response deleteCustomer(@RequestBody Customer customer)
- 서비스로 삭제할 Customer 정보 전달 (id 필요)
- 해당 고객 삭제

// 고객 상품 주문
@PostMapping("/order")
public Response placeOrder(@RequestBody OrderRequest order)
HTTP POST /api/customers/order
- 서비스로 OrderRequest(productId, quantity 등) 전달
- 고객이 상품 주문

// 고객 주문 취소
@PostMapping("/cancel")
public Response cancelOrder(@RequestBody OrderRequest order)
- 서비스로 OrderRequest 전달
```

온라인 쇼핑몰 API (skala-shop-api) – 전체 폴더 구조

```

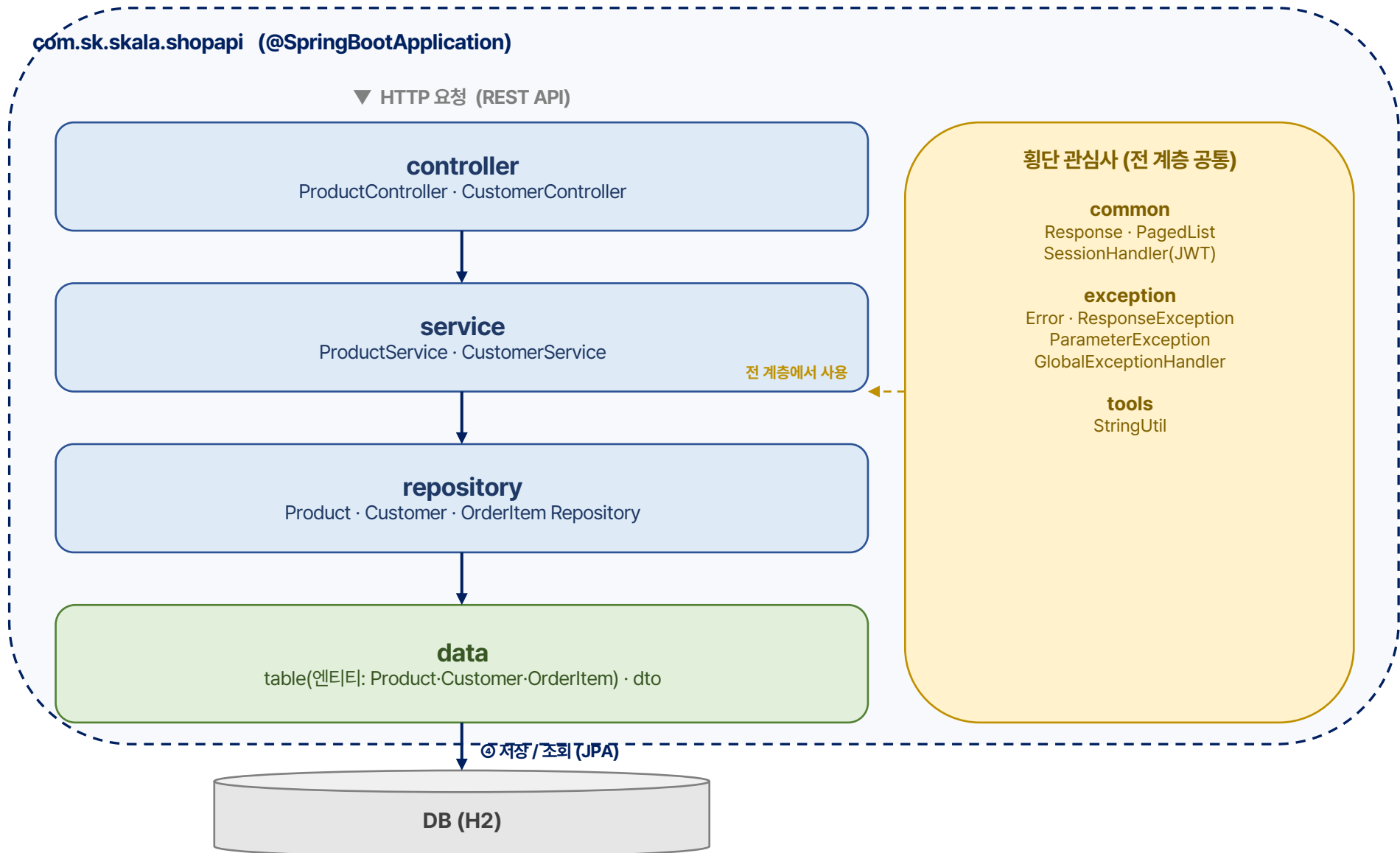
scala-shop-api/
├─ build.gradle           # Gradle 빌드 스크립트(플러그인·의존성)
├─ settings.gradle       # rootProject.name = 'skala-shop-api'
├─ src/main/java/com/sk/skala/shopapi/
│   ├── ShopApiApplication.java    @SpringBootApplication (메인)
│   ├── controller/               ProductController · CustomerController ← REST API
│   ├── service/                  ProductService · CustomerService ← 비즈니스 로직
│   ├── repository/               Product·Customer·OrderItem Repository ← JPA 접근
│   ├── data/table/               Product · Customer · OrderItem ← JPA 엔티티
│   ├── data/dto/                 OrderItemDto · OrderListDto · OrderRequest · CustomerSession
│   ├── common/                  Response · PagedList · SessionHandler(JWT)
│   ├── exception/               Error · ResponseException · ParameterException · GlobalExceptionHandler
│   └─ tools/                     StringUtil
├─ src/main/resources/
│   ├── application.yml          # H2 · JPA · Actuator 설정
│   └─ data.sql                  # 초기 상품 시드 데이터
└─ README.md                    # 실행법 · API · 시나리오

```

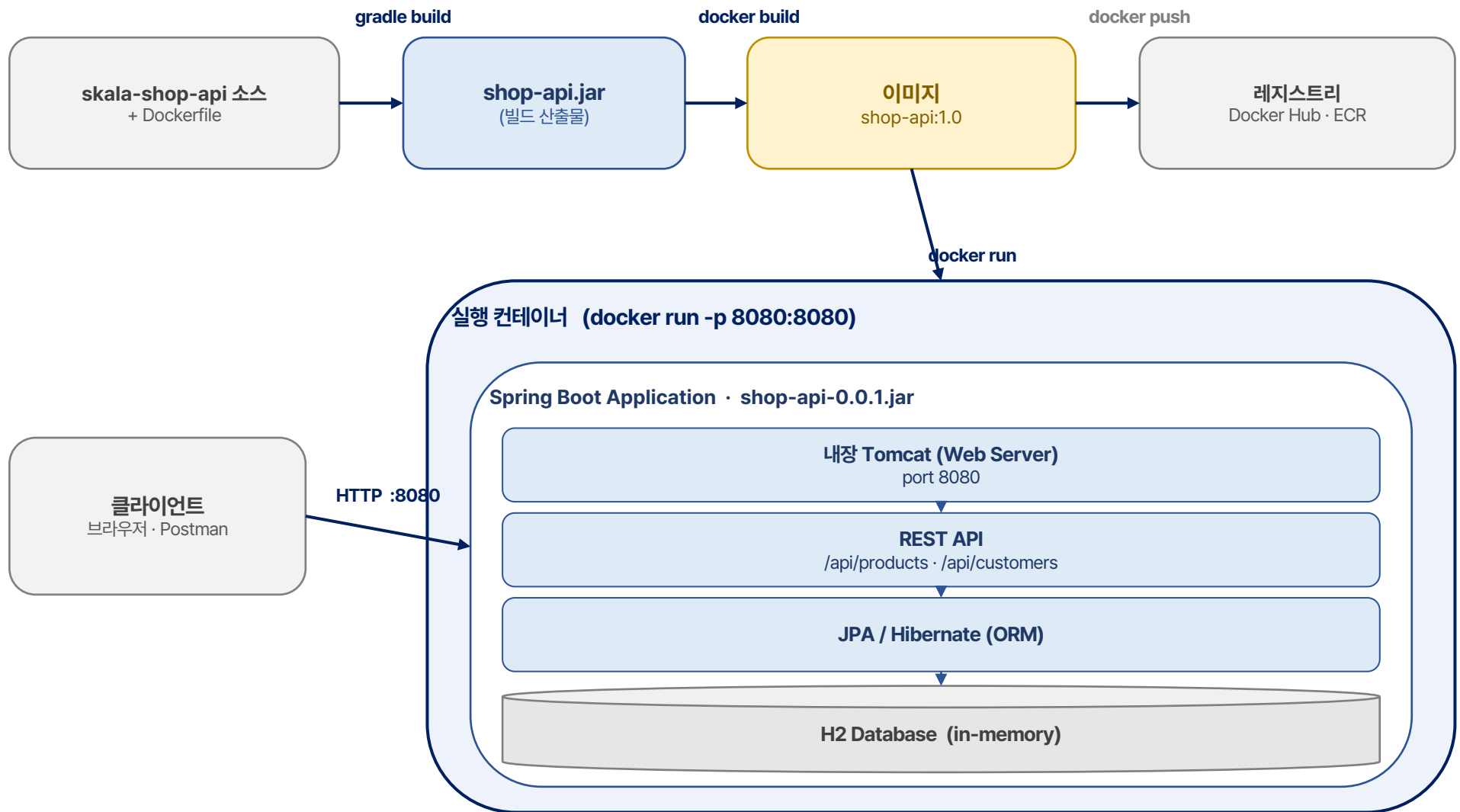
계층 구조: Controller(요청) → Service(로직) → Repository(JPA) → DB

* 전체 완성 코드는 별도 Spring Boot 프로젝트 파일(skala-shop-api)로 제공

계층형 패키지 구조 - 중첩 박스 다이어그램



skala-shop-api Docker 배포 다이어그램



빌드 : gradle build → jar → docker build → 이미지 실행 : docker run -p 8080:8080 (내장 Tomcat·H2 포함 단일 컨테이너)

[실습] 마무리

- **[추가 도전 과제]** → 혹시, 완성되면 구현 방식을 “슬랙 다이렉트 메시지”로 전달!
 - 입력값 검증(`@Valid`)과 `GlobalExceptionHandler`를 이용해서 요청 파라미터 예외 처리하기
 - H2 DB 데이터를 로컬 파일로 저장해서 유지하기
 - OAS 기반으로 REST API 문서 자동화를 구성하기

API 테스트 – Postman

- SKALA : API 검증 collection
- API 호출

SKALA

GET stocks/list

POST stocks

PUT stocks

DEL stocks

GET players/list

POST players

POST players/buy

POST players/sell

PUT players

DEL players

GET localhost:9080/api/stocks/list?offset=0&count=2

Send

Params Authorization Headers (6) Body Scripts Tests Settings Cookies

Headers 6 hidden

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results

200 OK 156 ms 444 B Save Response

Pretty Raw Preview Visualize JSON

```

1  {
2    "result": 0,
3    "code": 0,
4    "message": null,
5    "body": {
6      "total": 4,
7      "count": 2,
8      "offset": 0,
9      "list": [
10       {
11         "id": 2,
12         "stockName": "GreenEnergy",
13         "stockPrice": 80.0
14       },
15       {
16         "id": 3,
17         "stockName": "HealthPlus",
18         "stockPrice": 120.0
19       }
20     ]
21   }
22 }
```

Postbot Ctrl Alt P

[핵심 요약] [실습] 스칼라 온라인 쇼핑몰 API

핵심 포인트	반드시 숙지할 내용
통합 실습	Entity → Repository → Service → Controller 순으로 계층별 구현
구조 이해	Frontend UI ↔ Backend API 관계, 패키지 구조, build.gradle 의존성
도메인 매핑	상품·고객·주문상품(1:N 매핑) 엔티티와 DTO 설계
비즈니스 로직	Service에 @Transactional 적용, Repository 쿼리 메서드 활용
검증	Postman 으로 전체 API 요청·응답 테스트

컨테이너 이해 및 적용

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

왜 컨테이너가 필요할까? – ‘제 컴퓨터에선 됐는데요?’

상황	설명
문제 상황	내 PC에선 잘 되던 앱이 다른 PC·서버에선 오류 (Java 버전·OS·라이브러리·환경변수 차이)
근본 원인	앱 실행에 필요한 환경이 컴퓨터마다 제각각
기존 방식	설치 문서 보고 수동 세팅 → 실수 잦고 시간 소모(“환경 지옥”)
컨테이너 방식	앱 + 필요한 환경을 하나의 상자에 통째로 담아 어디서나 똑같이 실행
결과	내 PC = 테스트 = 운영 환경 일치 → 배포가 쉽고 안전해짐
한 줄 정의	컨테이너 = 앱 + 실행 환경을 격리해 담은 표준 실행 단위

컨테이너 쉽게 이해하기 – 택배(운송) 컨테이너 비유

구분	운송 컨테이너	소프트웨어 컨테이너
내용물	냉장고·옷·가구 등 다양한 화물	앱·라이브러리·설정 파일
표준 규격	어떤 배·트럭·항구든 동일하게 취급	어떤 PC·서버·클라우드든 동일 실행
격리(밀봉)	다른 화물과 섞이지 않음	다른 앱과 격리(포트·파일·프로세스)
이동	배→트럭→기차로 그대로 운반	개발→테스트→운영으로 그대로 배포
핵심	“한 번 실으면 어디든 그대로”	Build once, Run anywhere

이미지 vs 컨테이너 – ‘붕어빵 틀’과 ‘붕어빵’

개념	비유	실제 의미
이미지(Image)	붕어빵 틀	실행에 필요한 것을 담은 설계도(읽기 전용), 재사용
컨테이너(Container)	구워낸 붕어빵	이미지를 실행한 실체 — 틀 하나로 여러 개 생성
관계	틀 1개 → 붕어빵 여러 개	이미지 1개 → 컨테이너 N개 실행
레지스트리	틀 보관 창고	이미지를 저장·공유 (Docker Hub, ECR)
비슷한 개념	설계도와 완제품	Class(이미지) vs Object/Instance (컨테이너)

컨테이너란? – 가상머신(VM) vs 컨테이너

구분	가상머신(VM)	컨테이너
격리 방식	게스트 OS 전체 + Hypervisor	프로세스 단위 (호스트 OS 커널 공유)
크기	수 GB (OS 포함)	수십~수백 MB (앱+라이브러리)
기동 속도	수십 초 ~ 분	수 초 이내
자원 효율	낮음 (OS 중복)	높음 (커널 공유)
대표 기술	VMware, VirtualBox	Docker , containerd
이식성	낮음	Build once, Run anywhere

Docker 아키텍처 & 핵심 개념

구성요소	설명
이미지(Image)	앱 실행에 필요한 파일·라이브러리·설정을 담은 읽기 전용 템플릿
컨테이너(Container)	이미지를 실행한 격리된 인스턴스 (실행 중인 프로세스)
Dockerfile	이미지를 만드는 빌드 스크립트 (명령 나열)
레지스트리(Registry)	이미지를 저장·공유하는 저장소 (Docker Hub , AWS ECR)
Docker Daemon	이미지·컨테이너를 관리하는 백그라운드 서비스
이미지 계층(Layer)	명령별로 쌓이는 캐시 계층 → 빌드 속도·용량 최적화

Docker 핵심 명령어

명령어	설명
<code>docker build -t app:1.0 .</code>	Dockerfile로 이미지 빌드
<code>docker run -d -p 8080:8080 app:1.0</code>	컨테이너 실행 (백그라운드, 포트 매핑)
<code>docker ps -a</code>	컨테이너 목록(실행 중/전체)
<code>docker logs -f <id></code>	컨테이너 로그 실시간 확인
<code>docker exec -it <id> sh</code>	실행 중 컨테이너 내부 접속
<code>docker stop / rm <id></code>	컨테이너 중지 / 삭제
<code>docker push / pull</code>	레지스트리에 이미지 업로드 / 다운로드

Dockerfile 이해 – Spring Boot 앱 이미지화

지시어	설명
FROM	베이스 이미지 지정 (예: eclipse-temurin:17-jre)
WORKDIR	컨테이너 내 작업 디렉토리 설정
COPY	빌드 산출물(jar 등)을 이미지로 복사
EXPOSE	컨테이너가 사용하는 포트 명시
ENTRYPOINT	컨테이너 시작 시 실행 명령 (java -jar)
멀티 스테이지	빌드 단계 / 실행 단계 분리 → 이미지 경량화

[샘플코드] Dockerfile (멀티 스테이지)

```
# 1) 빌드 스테이지 (Gradle 빌드)
FROM gradle:8.5-jdk17 AS build
WORKDIR /app
COPY . .
RUN gradle clean build -x test

# 2) 실행 스테이지 (경량 JRE 이미지)
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

# 빌드: docker build -t shop-api:1.0 .
# 실행: docker run -d -p 8080:8080 shop-api:1.0
```

Docker Compose – 여러 컨테이너 함께 실행

항목	설명
목적	앱 + DB 등 여러 컨테이너를 한 번에 정의·실행
파일	docker-compose.yml (서비스·네트워크·볼륨 선언)
주요 명령	docker compose up -d (실행) / down (종료)
services	각 컨테이너(앱, mysql 등)를 정의
depends_on	서비스 기동 순서 지정
volumes	데이터 영속화 (컨테이너 삭제돼도 유지)

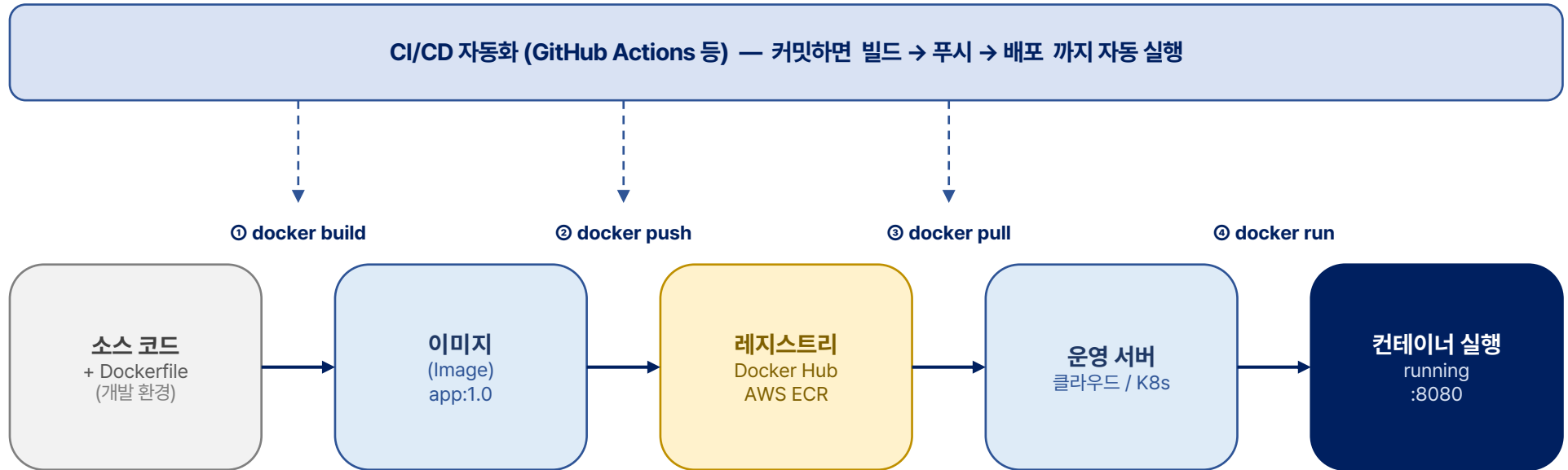
[샘플코드] docker-compose.yml

```
version: "3.8"
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=prod
    depends_on:
      - db
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: shop
      MYSQL_ROOT_PASSWORD: secret
    volumes:
      - db-data:/var/lib/mysql
volumes:
  db-data:
```


컨테이너 배포 흐름 & 레지스트리

단계	설명
1. 빌드	소스 → jar → docker build 로 이미지 생성
2. 태깅	docker tag app:1.0 <레지스트리>/app:1.0
3. 푸시	docker push → Docker Hub / AWS ECR 저장
4. 배포	운영 서버·쿠버네티스에서 docker pull → run
이미지 태그	버전 관리(1.0, latest) → 롤백·재현성 확보
CI/CD	GitHub Actions 등으로 빌드~푸시~배포 자동화

컨테이너 배포 흐름 다이어그램 - 빌드에서 실행까지



배포 흐름 4단계

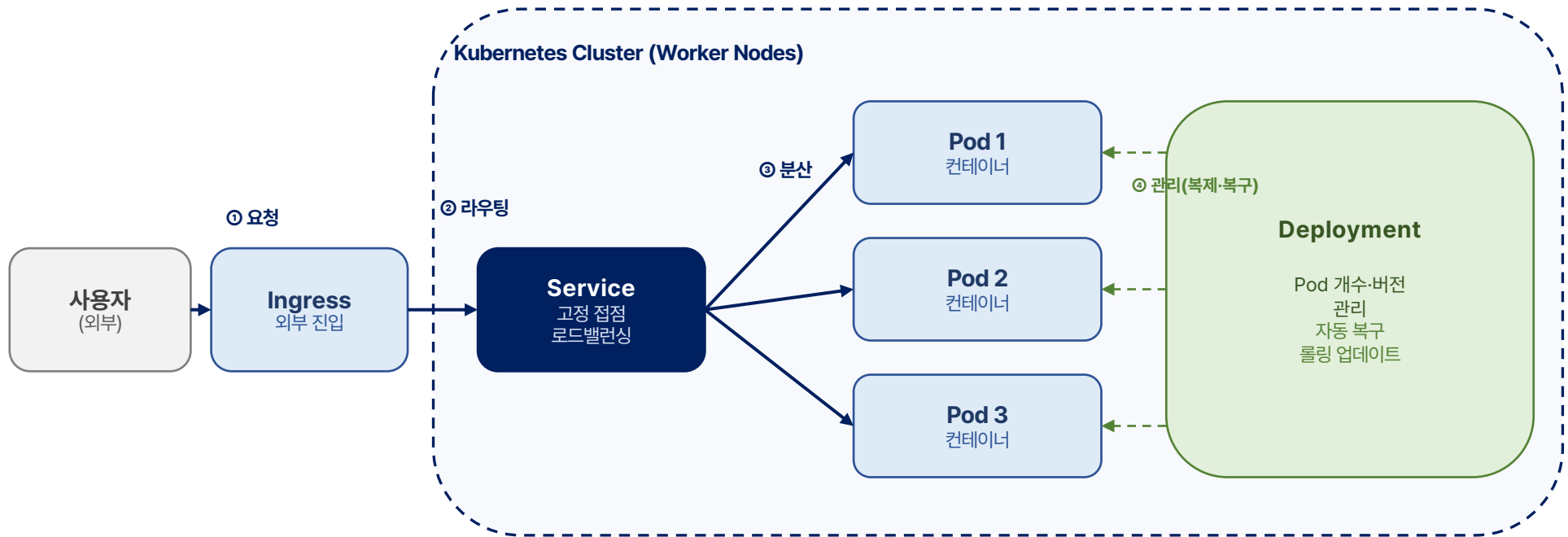
① docker build : Dockerfile로 소스 → 이미지 생성 ② docker push : 이미지를 레지스트리에 업로드(버전 태그로 관리)

③ docker pull : 운영 환경에서 이미지 다운로드 ④ docker run : 컨테이너로 실행(포트 매핑, 예 : 8080)

핵심 : 한 번 만든 이미지를 개발·테스트·운영에 그대로 배포 → 환경 일치 · 손쉬운 롤백

쿠버네티스(Kubernetes) 개념도 - 컨테이너 오케스트레이션

Control Plane (마스터) : API Server · Scheduler · etcd · Controller — 원하는 상태(desired state) 유지·관리



쿠버네티스(Kubernetes) 핵심 오브젝트

Pod : 컨테이너를 실행하는 최소 배포 단위 **Deployment** : Pod 개수·버전을 선언적으로 관리(자동 복구·롤링 업데이트)

Service : 여러 Pod의 고정 진입점 + 로드밸런싱 **Ingress** : 외부 HTTP(S) 요청을 Service로 라우팅

Control Plane : 원하는 상태(desired state)를 유지하도록 클러스터를 자동 제어 (컨테이너를 대규모로 운영·관리)

[핵심 요약] 컨테이너 이해 및 적용

핵심 포인트	반드시 숙지할 내용
컨테이너 vs VM	OS 커널 공유로 가볍고 빠름(초 단위 기동)
Docker 3요소	이미지 · 컨테이너 · 레지스트리
Dockerfile	이미지 빌드 스크립트 — 멀티 스테이지로 경량화
Compose	앱+DB 등 여러 컨테이너를 한 번에 실행
이식성	Build once, Run anywhere — 개발=운영 환경 일치
배포	build → push → pull → run, CI/CD 자동화

MSA 이해 및 적용

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

왜 MSA가 등장했을까? – 모놀리식의 '성장통'

상황	설명
처음(소규모)	기능이 적어 하나의 앱(모놀리식) 으로 충분 → 개발·배포가 단순
커지면서	코드 수십만 줄·팀 수십 명 → 작은 수정에도 전체를 다시 배포
문제 ①	일부 기능만 트래픽 폭증해도 앱 전체를 늘려야 함(비효율)
문제 ②	한 모듈의 오류가 전체 서비스 장애 로 전파
문제 ③	새 기술 도입·부분 교체가 어려움(코드가 강하게 얽힘)
그래서 MSA	앱을 작고 독립적인 서비스들로 분리 하자

MSA 쉽게 이해하기 – ‘대형 식당’ vs ‘푸드코트’

구분	모놀리식 (대형 식당)	MSA (푸드코트)
구성	한 주방에서 모든 메뉴 조리	매장(서비스)마다 독립 주방
확장	손님 몰리면 식당 전체를 키움	인기 매장만 규모 확대
장애	주방에 불나면 식당 전체 마비	한 매장 닫아도 나머지는 영업
메뉴 변경	식당 전체 리모델링 필요	매장 하나만 바꾸면 됨
관리	단순하지만 커지면 한계	유연하지만 매장 간 조율(운영) 필요

MSA는 항상 정답일까? – 장점과 대가(Trade-off)

구분	내용
장점	독립 배포·확장 , 장애 격리, 기술 자유도, 팀별 병렬 개발
대가(비용)	서비스 간 통신 복잡 , 분산 트랜잭션·데이터 일관성 관리 어려움
운영 부담	배포 대상↑, 모니터링·로그 추적↑ (Gateway·Eureka 등 인프라 필요)
모놀리식이 유리	초기·소규모·소수 팀에는 모놀리식이 오히려 단순·효율적
MSA가 유리	대규모·트래픽 편차 큼·잦은 배포·여러 팀 협업
현실적 조언	“작게 시작(모놀리식)하고, 필요할 때 분리” 가 안전한 전략

모놀리식(Monolith) vs MSA

구분	모놀리식	MSA
구조	하나의 앱에 전 기능 포함	작은 서비스들로 분리
배포	전체를 한 번에 배포	서비스별 독립 배포
확장	전체를 스케일아웃	필요한 서비스만 스케일
장애 영향	전체로 전파 가능	서비스 단위로 격리
기술 스택	단일 스택	서비스별 자유 선택
데이터	단일 DB 공유	서비스별 DB(독립성)
적합	소규모·초기	대규모·복잡·빠른 변화

MSA 핵심 구성요소

구성요소	역할
API Gateway	모든 요청의 단일 진입점 (라우팅·인증·로깅)
Service Discovery	서비스 위치(주소) 를 자동 등록·조회
Config Server	설정을 중앙 집중 관리
서비스 간 통신	REST / OpenFeign / 메시지 큐(Kafka 등)
Circuit Breaker	장애 전파를 차단하는 회복성 장치
분산 추적/로깅	서비스 넘나드는 요청 흐름 추적(Zipkin)

Spring Cloud 생태계

컴포넌트	설명
Spring Cloud Gateway	API Gateway (라우팅·필터)
Netflix Eureka	Service Discovery 서버/클라이언트
Spring Cloud Config	중앙 설정 관리 서버
OpenFeign	선언적 REST 클라이언트
Resilience4j	Circuit Breaker ·재시도·타임아웃
Sleuth / Zipkin	분산 추적 (요청 흐름 시각화)

API Gateway & 서비스 디스커버리

항목	설명
API Gateway	클라이언트 요청의 단일 진입점 → 각 서비스로 라우팅
Gateway 기능	Path/Host 라우팅, 인증, Rate Limit , 공통 필터
Service Discovery	동적으로 늘고 주는 인스턴스의 위치를 자동 관리
Eureka Server	서비스들의 레지스트리 (등록소)
등록/조회	서비스가 기동 시 자신을 등록, 이름으로 조회·호출
로드밸런싱	여러 인스턴스에 부하 분산 (client-side LB)

서비스 간 통신 · 설정 · 회복성

항목	설명
OpenFeign	선언적 인터페이스로 REST 호출 (보일러플레이트 감소)
이름 기반 호출	Eureka 결합 시 서비스 이름으로 호출
Config Server	모든 서비스 설정을 Git 등에서 중앙 관리 (재배포 없이 반영)
Circuit Breaker	장애 서비스 호출을 차단 해 전파 방지
상태 전이	Closed → Open(차단) → Half-Open(재시도)
Fallback	실패 시 대체 응답 반환(사용자 경험 유지)

[샘플코드] Spring Cloud (Gateway · Eureka · Feign)

1) API Gateway 라우팅 (**gateway** 서비스의 application.yml)

```
spring:
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://PRODUCT-SERVICE      # Eureka 이름으로 라우팅
          predicates:
            - Path=/api/products/**
```

// 2) 서비스 등록 (Eureka Client)

@SpringBootApplication

@EnableDiscoveryClient

```
public class ProductServiceApp { }
```

// 3) 선언적 통신 + 회복성 (OpenFeign + Resilience4j)

@FeignClient(name = "product-service")

```
public interface ProductClient {
```

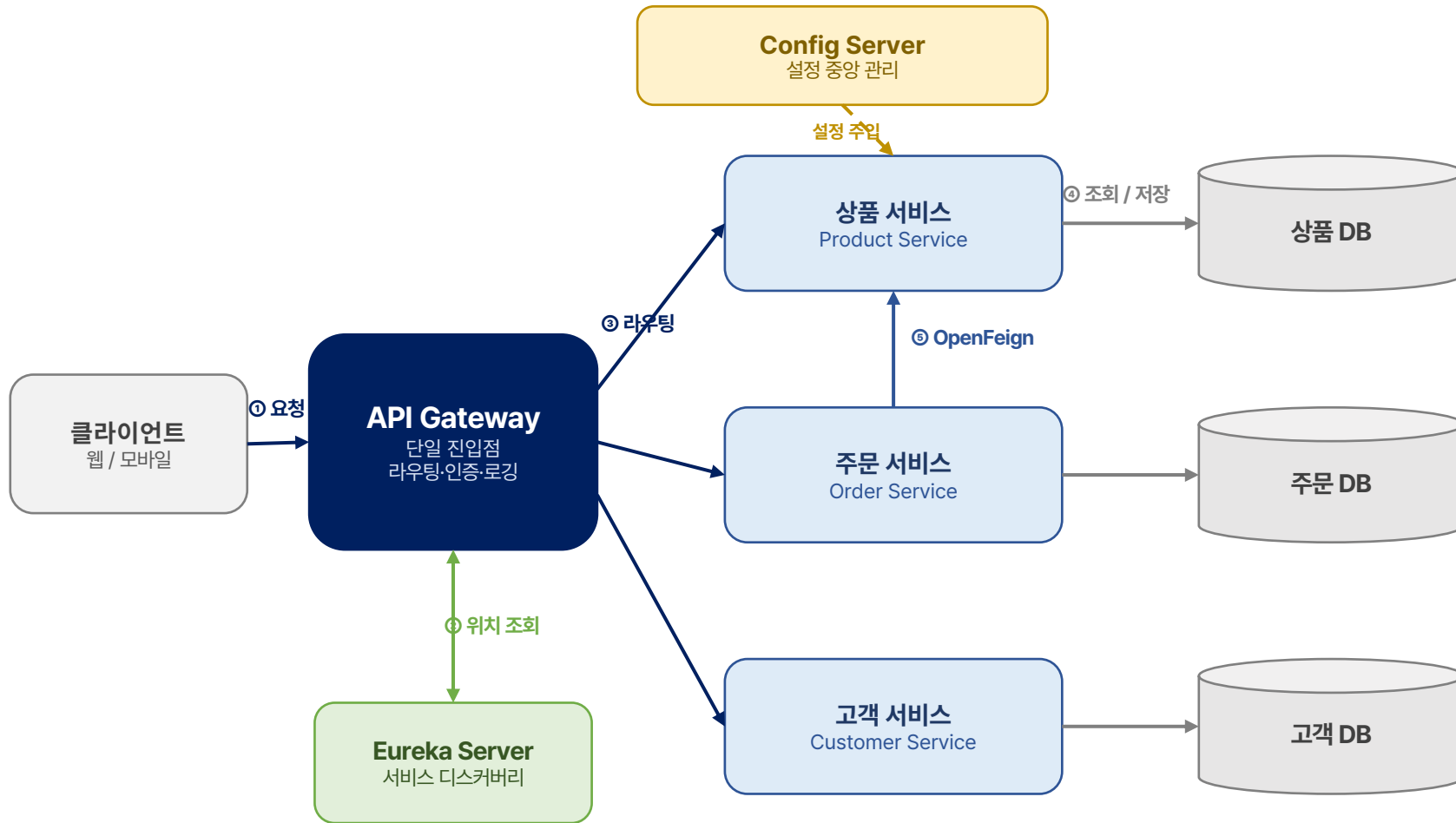
```
    @GetMapping("/api/products/{id}")
```

```
    @CircuitBreaker(name = "productCB", fallbackMethod = "fallback")
```

```
    Product getProduct(@PathVariable Long id);
```

```
}
```

MSA 통신 흐름 예시 - 요청 처리 과정



요청 처리 흐름

- ① 클라이언트가 API Gateway로 요청 → ② Gateway가 Eureka에서 서비스 위치 조회 → ③ 해당 서비스로 라우팅
 ④ 각 서비스가 자신의 DB 처리 → ⑤ 서비스 간 통신은 OpenFeign(+Circuit Breaker로 장애 격리)

[핵심 요약] MSA 이해 및 적용

핵심 포인트	반드시 숙지할 내용
모놀리식 vs MSA	기능을 작은 서비스로 분리 → 독립 배포·확장
API Gateway	요청의 단일 진입점 (라우팅·인증·공통 처리)
Service Discovery	Eureka 로 서비스 위치 자동 등록·조회
서비스 통신	OpenFeign 선언적 REST 호출
설정/회복성	Config Server 중앙관리 + Circuit Breaker(Resilience4j)
트레이드오프	유연·확장성 ↑, 분산 복잡성 (운영·데이터 일관성) ↑



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.